Software design: FaceAttend EDU

Jonattan Steven Rizo Solano

Diego Andrés Gutiérrez Núñez

Juan David Arboleda Perdomo

Centro de la Industria, la Empresa y los Servicios, Servicio Nacional de Aprendizaje

Technologist in Software Analysis and Development, Cohort 3145556

November 4, 2026

Abstract

This document details the technical architecture of the FaceAttendEDU system, describing the architectural model, the implemented design patterns, the data model, and the technological stack used. It explains the application of Hexagonal Architecture and Domain-Driven Design (DDD) principles to ensure the scalability, maintainability, and security of the system. The document serves as a technical guide for developers, auditors, and software architects, ensuring that the technical implementation is aligned with the strategic and functional objectives of the project.

Keywords: architectural design, hexagonal architecture, design patterns, data model, microservices, DDD

Contents

Abstract 2
Introduction 6
- Document Objective 6
- Document Scope 6

Architectural Model 6
- Hexagonal Architecture (Ports and Adapters) 6
- Information Flow 7

Design Patterns 7
- Structural and Behavioral Patterns 7
- Data and Messaging Patterns 7

Data Model 8
- Main Entities and Relationships 8
- Persistence Strategy 8

Technical Information and Technological Stack 8
- Technological Stack 8
- Technical Success Factors 9

Conclusions 9
References 9
Graphical interfaces 10
Data model: conceptual, logical and physical 16
Data model guide 43
- 1. Overview 43
- 2. Bounded Contexts 43
- 3. Context: Identity 45
- 4. Context: Authorization 45
- 5. Context: Academic 46
- 6. Context: Scheduling 46
- 7. Context: Attendance 47
- 8. Context: Biometric 48
- 9. Context: Configuration 48
- 9.5 Context: Quality 48
- 10. Context: Notification 49
- 11. Most important rule: No FKs between contexts 49
- 12. Relationship diagram (simplified) 50
- 13. Model ENUMs 50
- 14. Auditing and cross-cutting best practices 51
- 15. Physical layer (implementation) 51

Class diagrams 53
Data dictionary: identity 60
- 1. Table: city — DEPRECATED (legacy, still deployed) 60
- 2. Table: person 60
- 3. Table: appuser 61
- 4. Table: usersession 62

5. Table: passwordpolicy 63
6. Relationship Diagram 65
7. Cross-Context Relationships 65
8. Correlations 66
Data dictionary: authorization 67
1. Table: role 67
2. Table: permission 68
3. Table: rolepermission 68
4. Table: userrole 69
5. Relationship Diagram 70
6. Cross-Context Relationships 70
7. Correlations 70
Data dictionary: academic 72
1. Table: school 72
2. Table: program 73
3. Table: academicperiod 74
4. Table: cohort 75
5. Table: course 75
6. Table: academicactortype 76
7. Table: academicactor 77
8. Table: enrollment 78
9. Relationship Diagram 80
10. Cross-Context Relationships 80
11. Correlations 81
Data dictionary: scheduling 82
1. Table: environment 82
2. Table: scheduleblock 83
3. Table: classsession 84
4. Relationship Diagram 86
5. Cross-Context Relationships 86
6. Correlations 87
Data dictionary: attendance 88
1. Table: attendancerecord 88
2. Table: justificationtype 89
3. Table: justification 90
4. Table: supportingdocument 91
5. Table: attendancereport (derived projection, outside 3NF core) 91
6. Relationship Diagram 93
7. Cross-Context Relationships 93
8. Correlations 94
Data dictionary: biometric 95
1. Collection: facialembeddings 95
2. Collection: fingerprintembeddings 96
3. Cross-Context Relationships 97

4. Correlations 98
Data dictionary: notification 99
1. Table: alerttype 99
2. Table: alert 100
3. Relationship Diagram 101
4. Cross-Context Relationships 101
5. Correlations 101
Data dictionary: configuration 103
1. Table: academicconfiguration 103
2. Table: securityconfiguration 104
3. Table: biometricupdatecase — logical owner Biometric, physical host Configuration (transient) 104
4. Relationship Diagram 106
5. Cross-Context Relationships 106
6. Correlations 106

Software design: FaceAttend EDU

Introduction

The architectural design of FaceAttendEDU represents the translation of functional and non-functional requirements into a robust technical structure. Unlike the software analysis, which defines the "what" and "why", this document focuses on the "how": code organization, data management, communication between components, and technology selection.

The system has been conceived under a **Polyglot Modular Monolith** approach (organized in a monorepo), where each module operates as an independent microservice with its own bounded context, allowing the coexistence of multiple programming languages based on technical needs (Java, TypeScript, Python, and Go).

Document Objective

To provide a detailed and formal description of the system architecture, the applied design patterns, and the data model, establishing the technical base for implementation, deployment, and software evolution.

Document Scope

This document covers the back-end services architecture, the front-end structure, the data persistence strategy, API management via Gateway, and the information flow between layers.

Architectural Model

Hexagonal Architecture (Ports and Adapters)

FaceAttendEDU implements **Hexagonal Architecture**, whose primary goal is to isolate the business logic (the core) from external technologies (frameworks, databases, APIs). This separation ensures that the system is testable and that changes in the infrastructure do not affect the business rules.

The internal structure of each service is divided into three main layers:

1. Domain Layer (The Core)

The center of the hexagon and the most stable part of the system.

- **Entities and Value Objects**: Represent business concepts (e.g., `User`, `AttendanceRecord`).
- **Ports**: Interfaces that define how the domain interacts with the outside. There are input ports (for use cases) and output ports (for persistence or external services).
- **Business Rules**: Pure logic that does not depend on any external library.

2. Application Layer

Acts as the system orchestrator.

- **Use Cases**: Implement the application logic. They receive data from input adapters, coordinate the domain, and return a response.
- **Orchestration**: They do not contain complex business rules but instead direct the data flow between the domain and infrastructure.

3. Infrastructure Layer (Adapters)

Contains the concrete implementations of the ports.

- **Input Adapters (Driving Adapters)**: REST Controllers (Spring Boot, FastAPI, etc.) that receive HTTP requests and translate them into use case calls.
- **Output Adapters (Driven Adapters)**: Persistence implementations (JPA, MongoDB, pgx) that translate domain needs into database queries.

Information Flow

A typical request flow follows this route: `Client` → `Kong Gateway` → `Controller (Infrastructure)` → `Use Case (Application)` → `Domain Entity/Port (Domain)` → `Persistence Adapter (Infrastructure)` → `Database`.

Design Patterns

To ensure code quality and ease of maintenance, the following patterns have been applied:

Structural and Behavioral Patterns

- **Repository Pattern**: Used to abstract the data layer. The domain defines an interface (port) and the infrastructure implements it, allowing the database to be changed without affecting the business logic.
- **Use Case / Interactor**: Each system functionality is encapsulated in a unique use case class, facilitating traceability and unit testing.
- **Data Transfer Object (DTO)**: Specific objects are used for data transport between the API and the application, avoiding exposing domain entities directly to the client.
- **Mapper Pattern**: Components responsible for transforming domain entities into DTOs and vice versa, maintaining the purity of the domain layer.
- **API Gateway**: Implemented using **Kong 3.6**, centralizing routing, rate limiting, CORS and edge authentication: a Lua *pre-function* requires `Authorization: Bearer <sessionId>` (opaque UUID, not a JWT) and validates it against ms-identity (`GET /api/v1/sessions/{id}`, 10 s cache). Permission checks are evaluated in each microservice.

Data and Messaging Patterns

- **Database-per-Service**: Each microservice has its own PostgreSQL schema, eliminating physical dependencies (FKs) between contexts and ensuring loose coupling.
- **Event-Driven Architecture (EDA)**: Use of **Apache Kafka** for asynchronous communication. Services publish domain events (e.g., `attendance-events`) that can be consumed by other modules in the future.
- **Polyglot Persistence**: Selection of the database based on the data type:
    - **PostgreSQL**: For structured and relational data.
    - **MongoDB**: For storing biometric embeddings (facial and fingerprint vectors).

- **Redis**: For caching and traffic control in the Gateway.

Data Model

The system uses a distributed data architecture based on logical schemas linked by UUIDs.

Main Entities and Relationships

Identity and Authorization Context

- **User**: Central entity linked to credentials and sessions.
- **Role & Permission**: Implementation of **RBAC (Role-Based Access Control)**. A role groups multiple permissions, and a user can have one or more roles.

Academic and Attendance Context

- **Academic Actor**: Represents the relationship between a person and their role in the school (student, instructor).
- **Course (Ficha) & Program**: Hierarchical structure of the academic offer.
- **Attendance Record**: Registers entry/exit, linking the user, the IoT device, and the timestamp.
- **Justification**: Entity linked to an attendance record, allowing the upload of digital supporting documents.

Persistence Strategy

Table 1

Persistence Strategy

Data Type	Technology	Justification
Relational Data	PostgreSQL 17	ACID consistency and support for complex schemas.
Biometric Embeddings	MongoDB 7	High efficiency in searching and storing high-dimensional vectors.
Sessions and Rate Limit	Redis 7	Low latency for real-time validations.

Technical Information and Technological Stack

The success of the FaceAttendEDU project is based on a technological selection oriented towards performance and specialization.

Technological Stack

Table 2

Technological Stack

Component	Technology	Version	Purpose
Backend (Core)	Java / Spring Boot	21 / 4.1.1	Identity and authorization services.
Backend (Biometrics)	Python / FastAPI	3.12 / 0.110	Biometric vector processing.
Backend (Notifications)	TypeScript / Fastify (Node.js)	5.4 / 4.26 (Node 20)	High concurrency in alerts (async I/O).
Backend (TS)	TypeScript / Fastify	5.4 / 4.26.0	Lightweight and fast services.
Frontend	React Native / Expo	RN 0.86 / Expo 57	Multiplatform mobile app.
Gateway	Kong	3.6	API Orchestration and Security.
Messaging	Apache Kafka	3.8.0	Asynchronous communication.
Migrations	Liquibase	4.29.0	Database version control.

Technical Success Factors

1. **Total Decoupling**: Thanks to Hexagonal Architecture, the system can evolve its frameworks without rewriting the business logic.
2. **Robust Security**: Opaque sessions (`sessionId` UUID persisted in `identity.user_session`) are validated twice: at Kong and again in each microservice (session against ms-identity and permissions against ms-authorization), so no request reaches business logic without being validated.
3. **Polyglot Persistence**: The use of MongoDB for biometrics and PostgreSQL for administration optimizes performance according to the nature of the data.
4. **Modular Scalability**: The microservices structure allows independent scaling of the attendance module (high load) and the configuration module (low load).

Conclusions

The architectural design of FaceAttendEDU is not just a technical choice, but a strategy to mitigate the risks identified in the analysis phase. The adoption of Hexagonal Architecture and DDD allows the system to be resilient to change and easy to audit. The combination of specialized languages (Java for robustness, Python for AI/Biometrics, Go for speed) ensures that each component is implemented with the most efficient tool for its purpose.

Author's Note. Jonas is the author and technical lead of the architectural design. Technical correspondence can be directed to jonas@consultoria.example.

References

American Psychological Association. (2020). *Publication manual of the American Psychological Association* (7th ed.). https://doi.org/10.1037/0000165-000

International Organization for Standardization & International Electrotechnical Commission. (2011). *Systems and software engineering — Systems and software quality requirements and evaluation (SQuaRE) — System and software quality models* (ISO/IEC 25010:2011). https://www.iso.org/standard/35733.html

Evans, E. (2003). *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley.

Alistair Cockburn. (2005). *Hexagonal Architecture (Ports and Adapters)*.

Graphical interfaces

Prototypes of the main screens. The editable HTML sources are in `source/mockups/`.

Web

Figure 1

AdministrationPanel

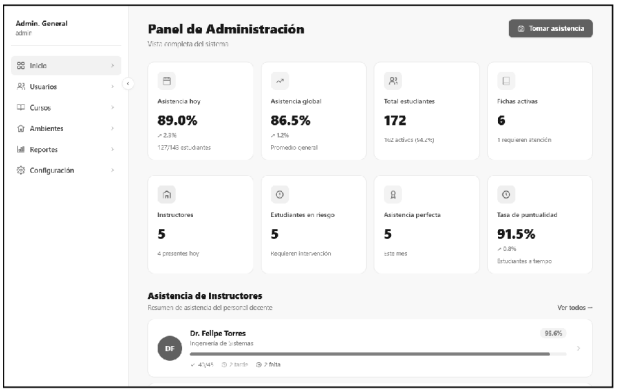


Figure 2

Enviroments

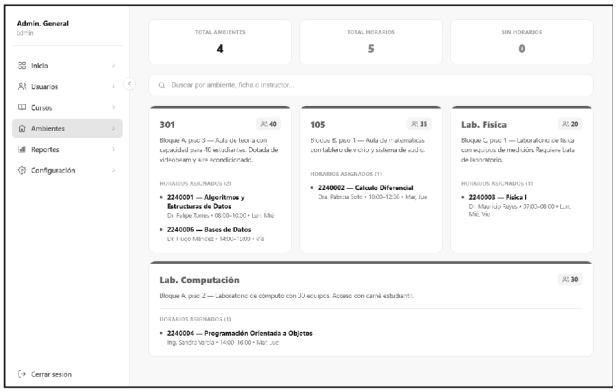


Figure 3

Courses

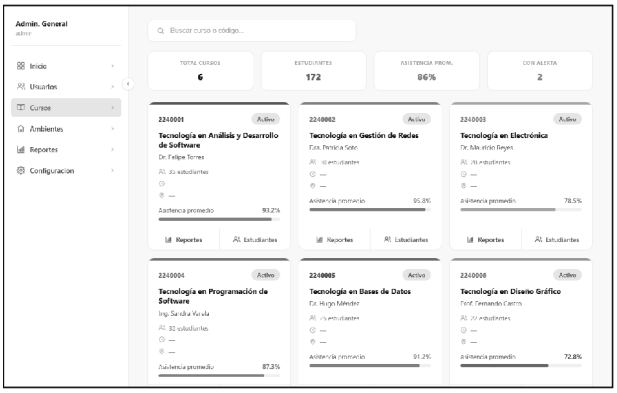


Figure 4

login

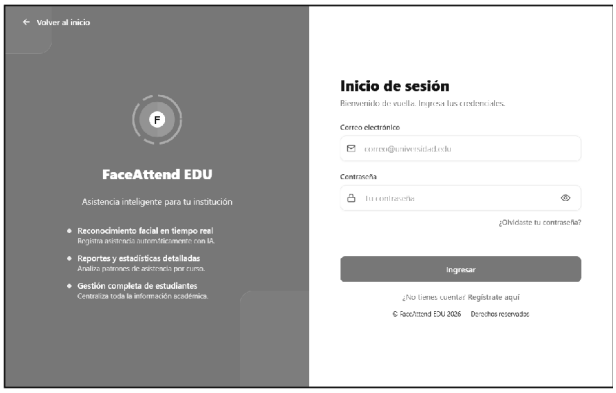


Figure 5

RecognitionConfiguration

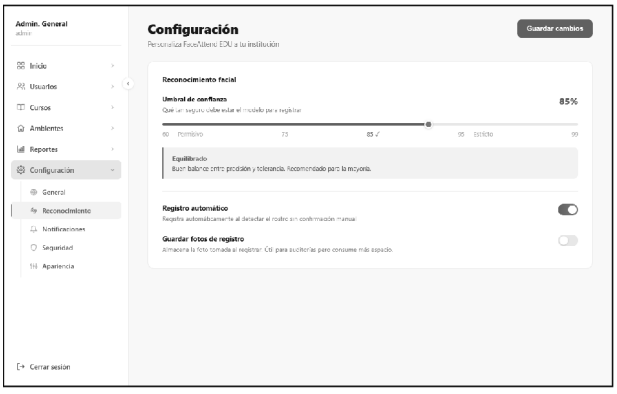


Figure 6

Configuration

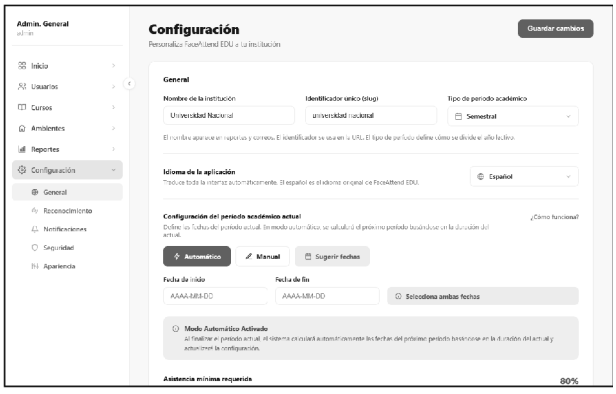


Figure 7

UsersAndStudent

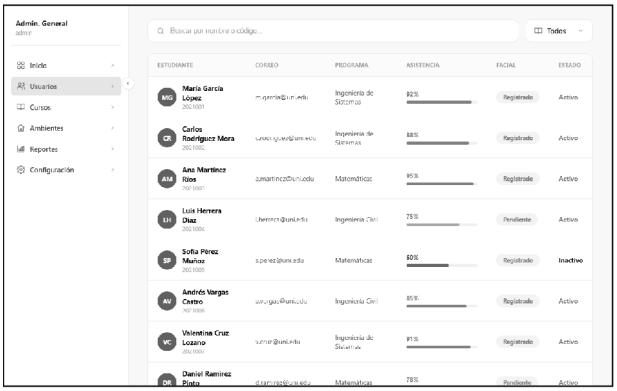


Figure 8

landing



Mobile

Figure 9

AttendanceRecords

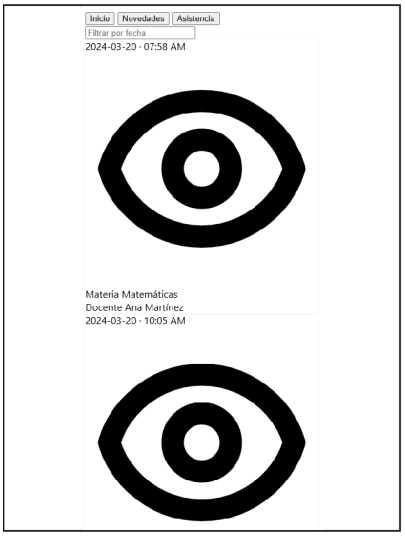


Figure 10

Dashboard

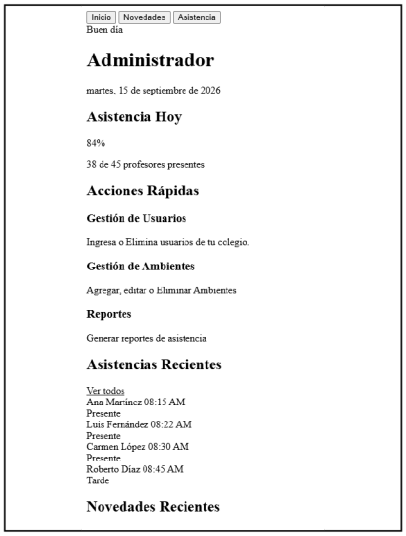


Figure 11

Faceregistration

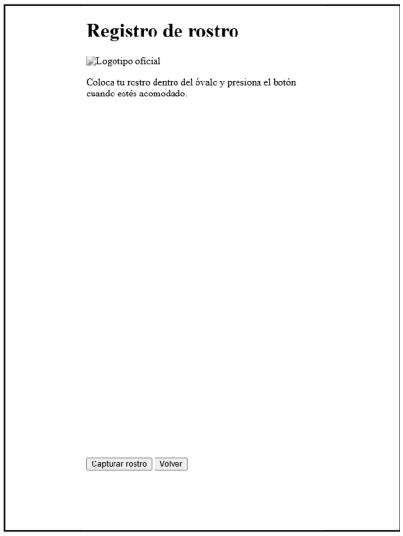


Figure 12

FacialRegistrationValidation

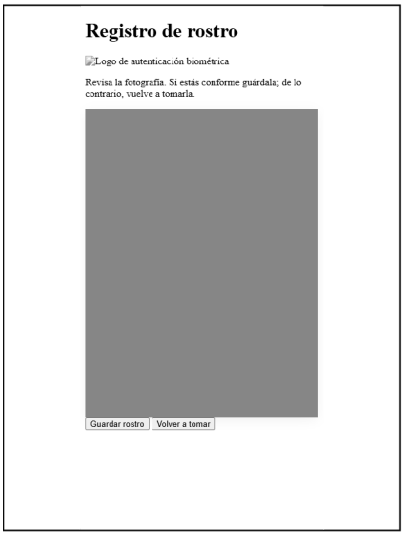


Figure 13

facialregistration

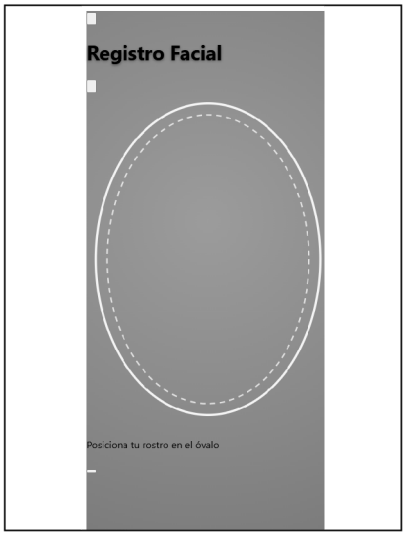


Figure 14

UpdatefacePhoto



Figure 15

Checkphoto



Figure 16

FacialFail

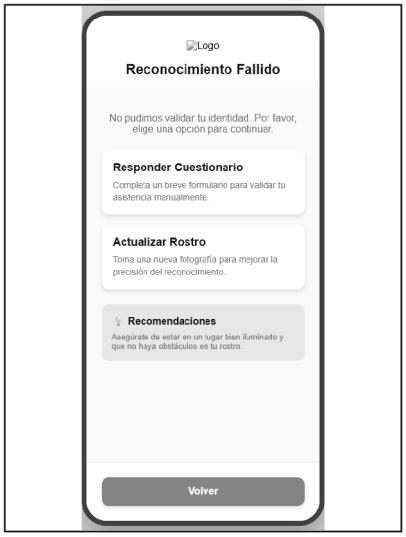


Figure 17

AttendanceHistory

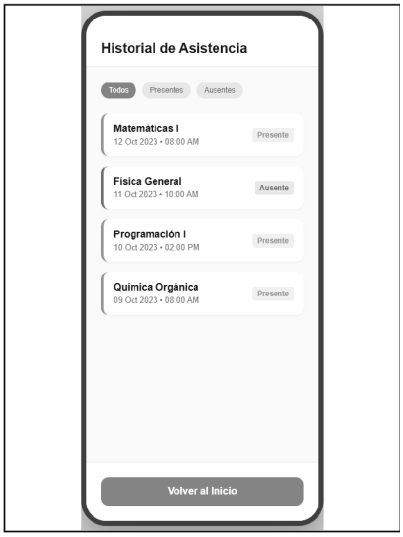


Figure 18

PendingJustifications



Figure 19

JustificationResults

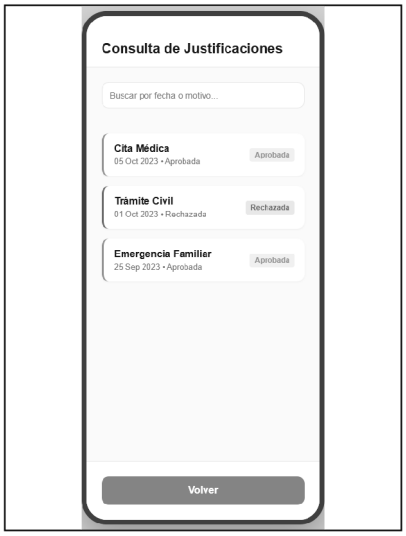


Figure 20

SubmitJustification

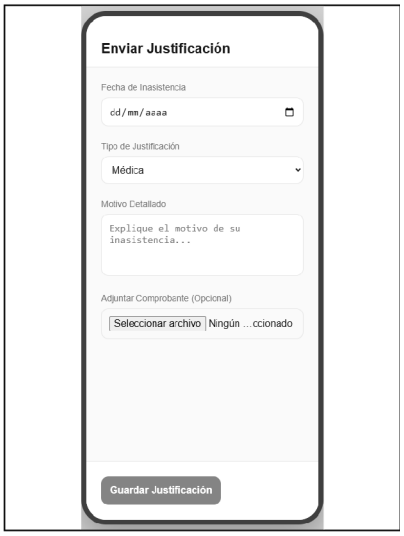


Figure 21

SubmittedJustifications



Figure 22

ValidateJustification

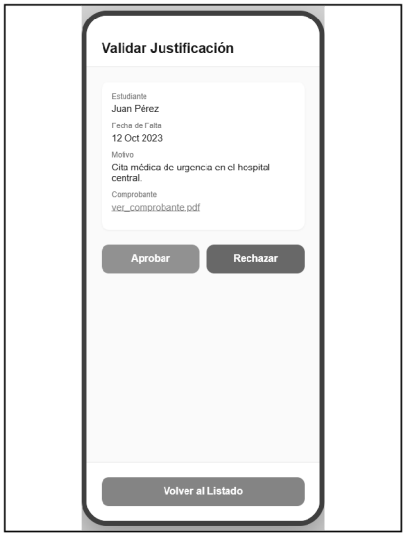


Figure 23

Login

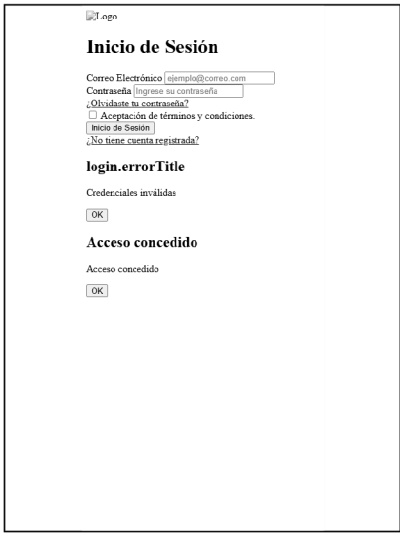


Figure 24

ManageEnvironments

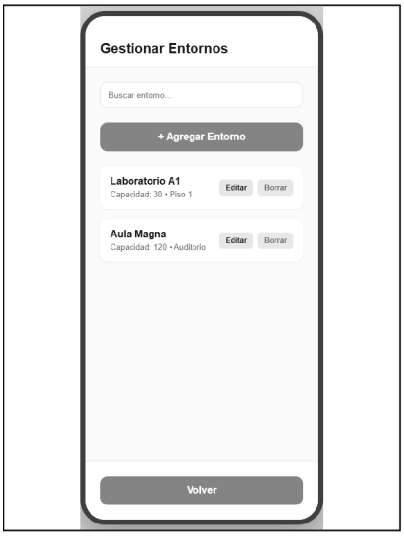


Figure 25

ManageUsers

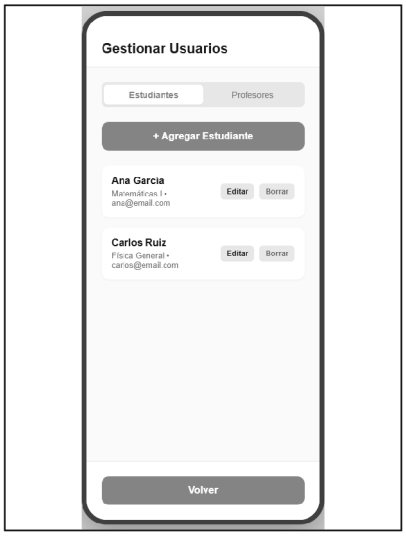


Figure 26

Menu

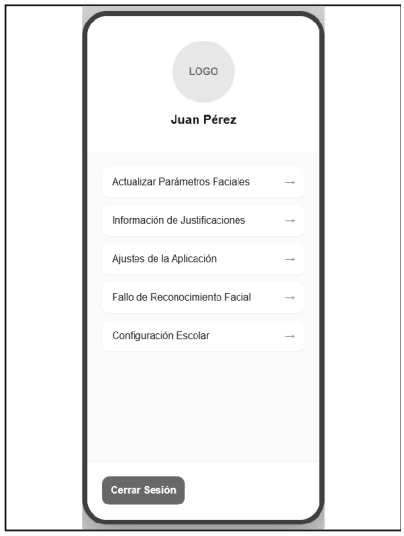


Figure 27

News

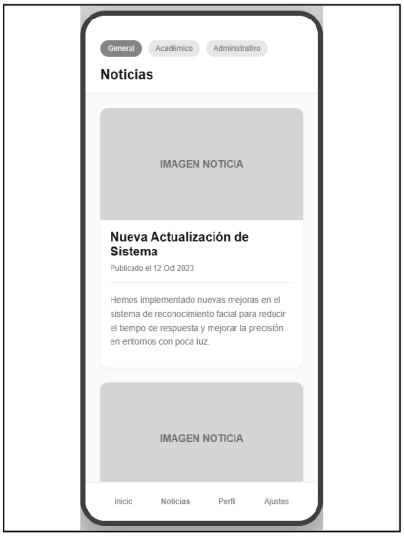


Figure 28

ChangePassword

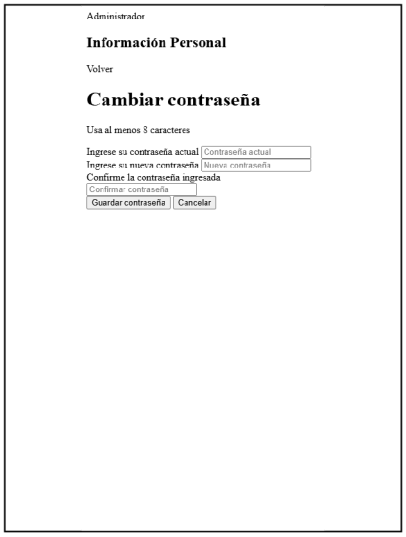


Figure 29

Profile

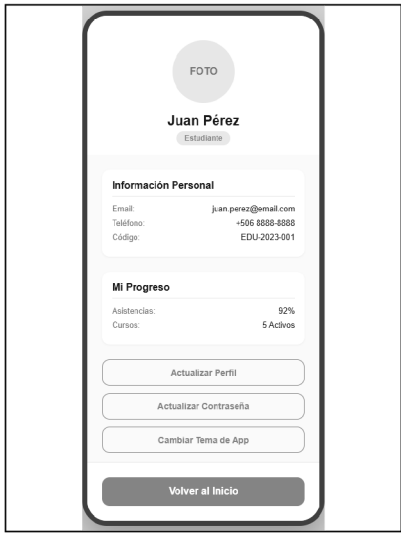


Figure 30

SchoolConfiguration

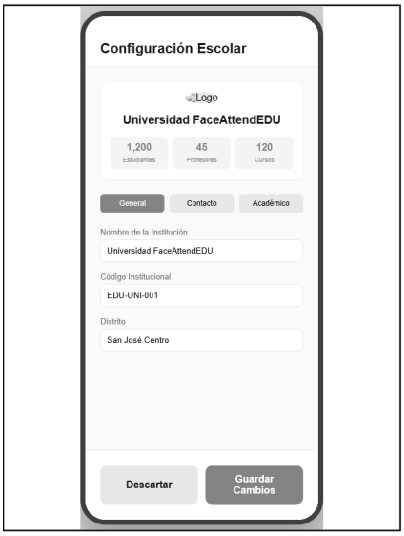


Figure 31

Settings



Data model: conceptual, logical and physical

Source

This document was generated from `database/faceattend_edu_mr_v4.dbml` in the main repository, the sole authority of the relational model. Biometric embeddings are stored in MongoDB, so they do not appear here as tables.

References between contexts are not foreign keys: the application validates them. Only relationships inside one context have a database constraint.

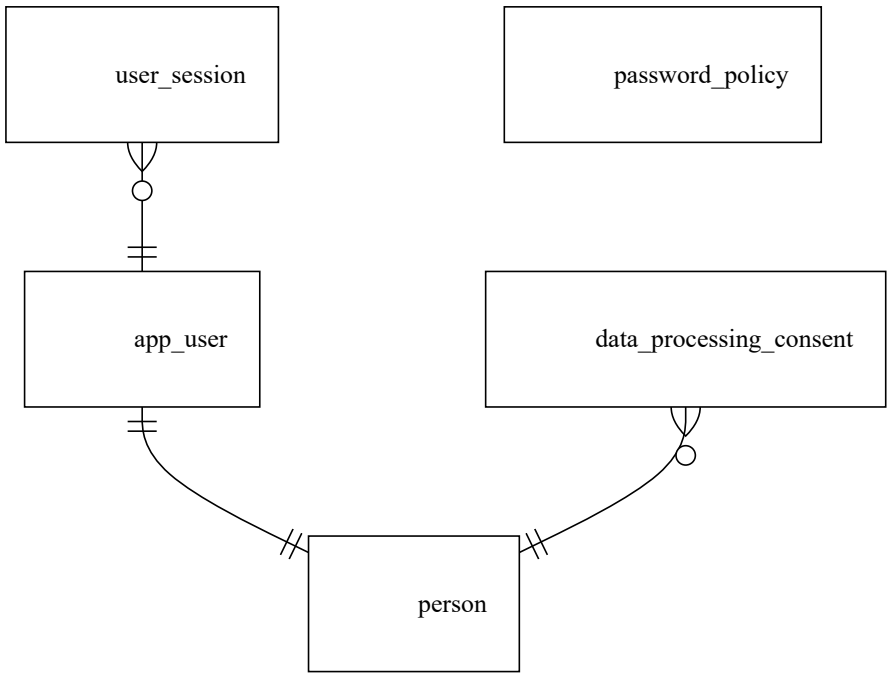
Conceptual model

Entities and relationships of each context, without attributes.

Context: Identity

Figure 32

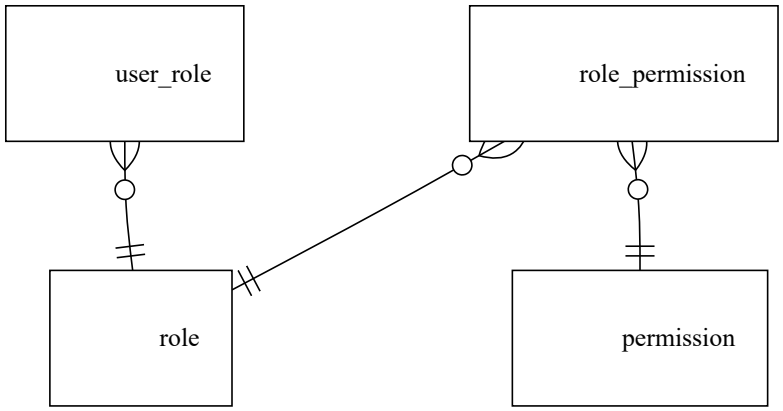
Context: Identity



Context: Authorization

Figure 33

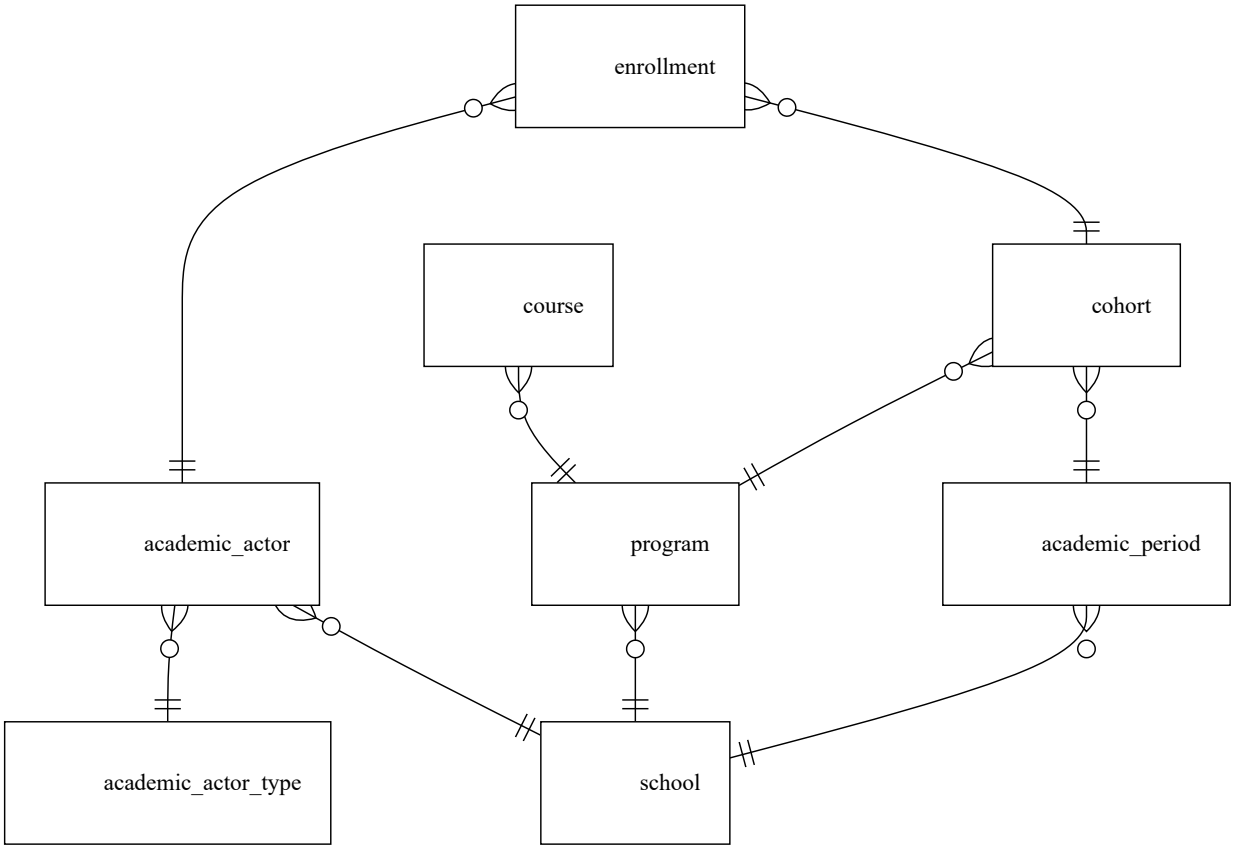
Context: Authorization



Context: Academic

Figure 34

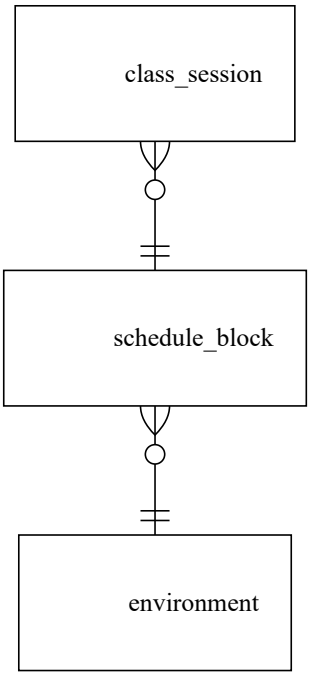
Context: Academic



Context: Scheduling

Figure 35

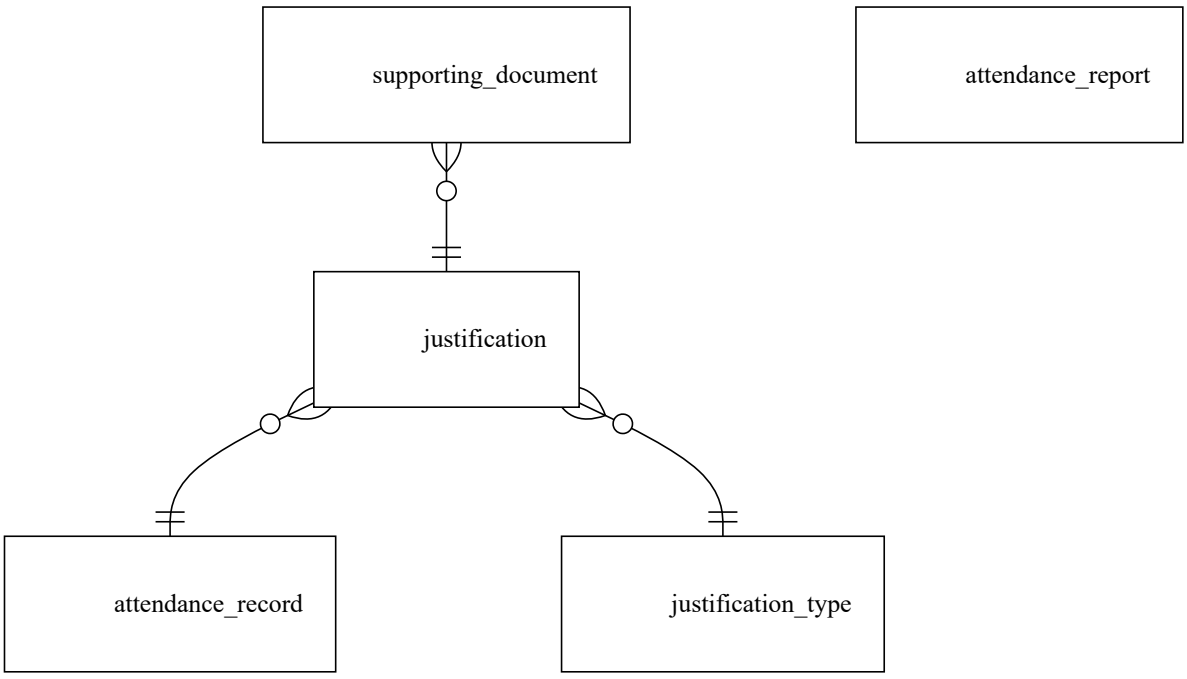
Context: Scheduling



Context: Attendance

Figure 36

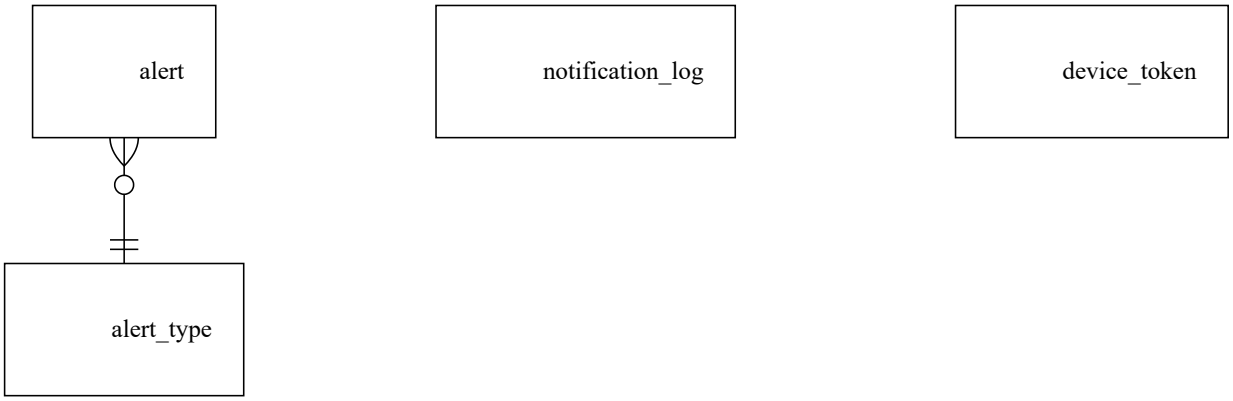
Context: Attendance



Context: Notification

Figure 37

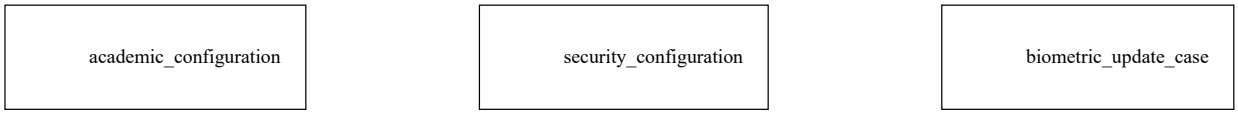
Context: Notification



Context: Configuration

Figure 38

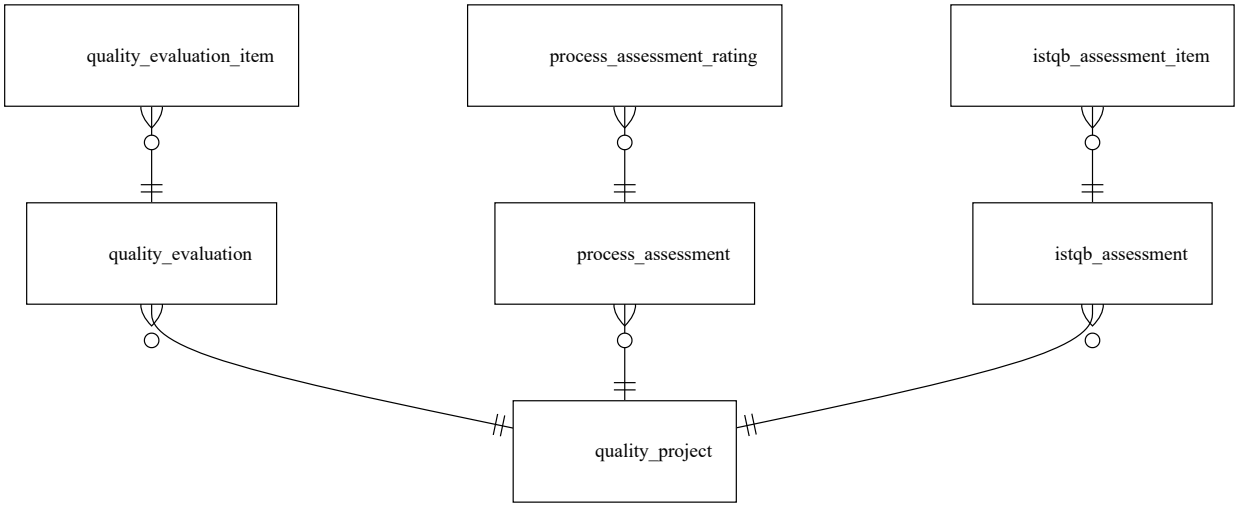
Context: Configuration



Context: Quality

Figure 39

Context: Quality



Logical model

Entities with attributes, types and keys of each context.

Identity

Table 3

person

Attribute	Type	Key
person_id	uuid	PK
document_number	varchar	
name	varchar	
last_name	varchar	
email	varchar	
phone	varchar	
document_type	varchar	
blood_type	varchar	
birth_date	date	
address	varchar	
status	boolean	
created_at	datetime	
updated_at	datetime	
deleted_at	datetime	
created_by	uuid	
updated_by	uuid	
deleted_by	uuid	
row_version	bigint	

Table 4

app_user

Attribute	Type	Key
user_id	uuid	PK
person_id	uuid	FK
username	varchar	
password_hash	varchar	
authentication_type	varchar	
status	boolean	
must_change_password	boolean	
two_factor_enabled	boolean	
last_access	datetime	
created_at	datetime	
updated_at	datetime	
deleted_at	datetime	
created_by	uuid	
updated_by	uuid	
deleted_by	uuid	
row_version	bigint	

Table 5

user_session

Attribute	Type	Key
session_id	uuid	PK
user_id	uuid	FK
start_date	datetime	
end_date	datetime	
source_ip	varchar	
session_status	varchar	
created_at	datetime	
updated_at	datetime	
deleted_at	datetime	
created_by	uuid	
updated_by	uuid	
deleted_by	uuid	
row_version	bigint	

Table 6

password_policy

Attribute	Type	Key
policy_id	int	PK
min_length	int	
max_length	int	
requires_uppercase	boolean	
requires_numbers	boolean	
requires_symbols	boolean	
expiration_days	int	
created_at	datetime	
updated_at	datetime	
deleted_at	datetime	
created_by	uuid	
updated_by	uuid	
deleted_by	uuid	
row_version	bigint	

Table 7

data_processing_consent

Attribute	Type	Key
consent_id	uuid	PK
person_id	uuid	FK
policy_version	varchar(20)	
accepted_at	datetime	
biometric_accepted	boolean	
biometric_revoked_at	datetime	
channel	varchar(10)	
recorded_by	uuid	
created_at	datetime	
updated_at	datetime	
deleted_at	datetime	
created_by	uuid	
updated_by	uuid	
deleted_by	uuid	
row_version	bigint	

Authorization

Table 8

role

Attribute	Type	Key
role_id	int	PK
role_name	varchar	UK
description	varchar	
created_at	datetime	
updated_at	datetime	
deleted_at	datetime	
created_by	uuid	
updated_by	uuid	
deleted_by	uuid	
row_version	bigint	

Table 9

permission

Attribute	Type	Key
permission_id	int	PK
permission_name	varchar	UK
description	varchar	
created_at	datetime	
updated_at	datetime	
deleted_at	datetime	
created_by	uuid	
updated_by	uuid	
deleted_by	uuid	
row_version	bigint	

Table 10

role_permission

Attribute	Type	Key
role_id	int	FK
permission_id	int	FK
assignment_date	datetime	
created_at	datetime	
updated_at	datetime	
deleted_at	datetime	
created_by	uuid	
updated_by	uuid	
deleted_by	uuid	
row_version	bigint	

Table 11

user_role

Attribute	Type	Key
user_id	uuid	
role_id	int	FK
assignment_date	datetime	
created_at	datetime	
updated_at	datetime	
deleted_at	datetime	
created_by	uuid	
updated_by	uuid	
deleted_by	uuid	
row_version	bigint	

Academic

Table 12

school

Attribute	Type	Key
school_id	int	PK
code	varchar	
name	varchar	
city_id	varchar	
address	varchar	
district	varchar	
country	varchar	
city_name	varchar	
phone	varchar	
email	varchar	
status	boolean	
created_at	datetime	
updated_at	datetime	
deleted_at	datetime	
created_by	uuid	
updated_by	uuid	
deleted_by	uuid	
row_version	bigint	

Table 13

program

Attribute	Type	Key
program_id	int	PK
school_id	int	FK
code	varchar	
name	varchar	
status	boolean	
created_at	datetime	
updated_at	datetime	
deleted_at	datetime	
created_by	uuid	
updated_by	uuid	
deleted_by	uuid	
row_version	bigint	

Table 14

academic_period

Attribute	Type	Key
academic_period_id	int	PK
school_id	int	FK
name	varchar	
starts_on	date	
ends_on	date	
is_active	boolean	
created_at	datetime	
updated_at	datetime	
deleted_at	datetime	
created_by	uuid	
updated_by	uuid	
deleted_by	uuid	
row_version	bigint	

Table 15

cohort

Attribute	Type	Key
cohort_id	bigint	PK
program_id	int	FK
academic_period_id	int	FK
code	varchar	
status	boolean	
created_at	datetime	
updated_at	datetime	
deleted_at	datetime	
created_by	uuid	
updated_by	uuid	
deleted_by	uuid	
row_version	bigint	

Table 16

course

Attribute	Type	Key
course_id	int	PK
program_id	int	FK
code	varchar	
name	varchar	
credit_hours	smallint	
status	boolean	
created_at	datetime	
updated_at	datetime	
deleted_at	datetime	
created_by	uuid	
updated_by	uuid	
deleted_by	uuid	
row_version	bigint	

Table 17

academic_actor_type

Attribute	Type	Key
actor_type_id	smallint	PK
code	varchar	UK
name	varchar	
created_at	datetime	
updated_at	datetime	
deleted_at	datetime	
created_by	uuid	
updated_by	uuid	
deleted_by	uuid	
row_version	bigint	

Table 18

academic_actor

Attribute	Type	Key
academic_actor_id	bigint	PK
person_id	uuid	
actor_type_id	smallint	FK
school_id	int	FK
actor_code	varchar	
started_on	date	
ended_on	date	
status	boolean	
created_at	datetime	
updated_at	datetime	
deleted_at	datetime	
created_by	uuid	
updated_by	uuid	
deleted_by	uuid	
row_version	bigint	

Table 19

enrollment

Attribute	Type	Key
enrollment_id	bigint	PK
academic_actor_id	bigint	FK
cohort_id	bigint	FK
enrolled_on	date	
enrollment_status	varchar	
created_at	datetime	
updated_at	datetime	
deleted_at	datetime	
created_by	uuid	
updated_by	uuid	
deleted_by	uuid	
row_version	bigint	

Scheduling

Table 20

environment

Attribute	Type	Key
environment_id	int	PK
school_id	int	
code	varchar	
name	varchar	
capacity	smallint	
status	boolean	
created_at	datetime	
updated_at	datetime	
deleted_at	datetime	
created_by	uuid	
updated_by	uuid	
deleted_by	uuid	
row_version	bigint	

Table 21

schedule_block

Attribute	Type	Key
schedule_block_id	bigint	PK
cohort_id	bigint	
course_id	int	
environment_id	int	FK
instructor_actor_id	bigint	
day_of_week	smallint	
starts_at	time	
ends_at	time	
created_at	datetime	
updated_at	datetime	
deleted_at	datetime	
created_by	uuid	
updated_by	uuid	
deleted_by	uuid	
row_version	bigint	

Table 22

class_session

Attribute	Type	Key
class_session_id	bigint	PK
schedule_block_id	bigint	FK
session_date	date	
opened_by	bigint	
opened_at	datetime	
closed_by	bigint	
closed_at	datetime	
session_status	varchar	
created_at	datetime	
updated_at	datetime	
deleted_at	datetime	
created_by	uuid	
updated_by	uuid	
deleted_by	uuid	
row_version	bigint	

Attendance

Table 23

attendance_record

Attribute	Type	Key
attendance_record_id	bigint	PK
class_session_id	bigint	
academic_actor_id	bigint	
attendance_status	varchar	
captured_at	datetime	
capture_method	varchar	
match_score	decimal(5,4)	
created_at	datetime	
updated_at	datetime	
deleted_at	datetime	
created_by	uuid	
updated_by	uuid	
deleted_by	uuid	
row_version	bigint	

Table 24

justification_type

Attribute	Type	Key
justification_type_id	int	PK
school_id	int	
name	varchar	
description	varchar	
requires_attachment	boolean	
status	boolean	
created_at	datetime	
updated_at	datetime	
deleted_at	datetime	
created_by	uuid	
updated_by	uuid	
deleted_by	uuid	
row_version	bigint	

Table 25

justification

Attribute	Type	Key
justification_id	bigint	PK
attendance_record_id	bigint	FK
justification_type_id	int	FK
reason	text	
submitted_at	datetime	
reviewed_by	uuid	
reviewed_at	datetime	
review_status	varchar	
resolution_notes	text	
created_at	datetime	
updated_at	datetime	
deleted_at	datetime	
created_by	uuid	
updated_by	uuid	
deleted_by	uuid	
row_version	bigint	

Table 26

supporting_document

Attribute	Type	Key
supporting_document_id	bigint	PK
justification_id	bigint	FK
file_name	varchar	
storage_uri	varchar	
mime_type	varchar	
size_bytes	bigint	
created_at	datetime	
updated_at	datetime	
deleted_at	datetime	
created_by	uuid	
updated_by	uuid	
deleted_by	uuid	
row_version	bigint	

Table 27

attendance_report

Attribute	Type	Key
report_id	uuid	PK
school_id	int	
cohort_id	bigint	
generated_by	uuid	
report_type	varchar	
filters	jsonb	
result	jsonb	
generated_at	datetime	
created_at	datetime	
updated_at	datetime	
deleted_at	datetime	
created_by	uuid	
updated_by	uuid	
deleted_by	uuid	
row_version	bigint	

Notification

Table 28

alert_type

Attribute	Type	Key
alert_type_id	smallint	PK
code	varchar	UK
name	varchar	
severity	varchar	
channel	varchar	
created_at	datetime	
updated_at	datetime	
deleted_at	datetime	
created_by	uuid	
updated_by	uuid	
deleted_by	uuid	
row_version	bigint	

Table 29

alert

Attribute	Type	Key
alert_id	bigint	PK
academic_actor_id	bigint	
alert_type_id	smallint	FK
raised_at	datetime	
resolved_at	datetime	
created_at	datetime	
updated_at	datetime	
deleted_at	datetime	
created_by	uuid	
updated_by	uuid	
deleted_by	uuid	
row_version	bigint	

Table 30

notification_log

Attribute	Type	Key
log_id	bigint	PK
kind	varchar	
dedupe_key	varchar	
school_id	int	
sent_at	datetime	
created_at	datetime	
updated_at	datetime	
deleted_at	datetime	
created_by	uuid	
updated_by	uuid	
deleted_by	uuid	
row_version	bigint	

Table 31

device_token

Attribute	Type	Key
device_token_id	bigint	PK
academic_actor_id	bigint	
token	varchar(255)	
platform	varchar(10)	
locale	varchar(5)	
created_at	datetime	
updated_at	datetime	
deleted_at	datetime	
created_by	uuid	
updated_by	uuid	
deleted_by	uuid	
row_version	bigint	
revoked_at	datetime	

Configuration

Table 32

academic_configuration

Attribute	Type	Key
configuration_id	int	PK
school_id	int	
configuration_name	varchar	
configuration_value	varchar	
description	varchar	
created_at	datetime	
updated_at	datetime	
deleted_at	datetime	
created_by	uuid	
updated_by	uuid	
deleted_by	uuid	
row_version	bigint	

Table 33

security_configuration

Attribute	Type	Key
configuration_id	int	PK
configuration_name	varchar	
configuration_value	varchar	
description	varchar	
created_at	datetime	
updated_at	datetime	
deleted_at	datetime	
created_by	uuid	
updated_by	uuid	
deleted_by	uuid	
row_version	bigint	

Table 34

biometric_update_case

Attribute	Type	Key
case_id	uuid	PK
person_id	uuid	
biometric_type	varchar	
finger_number	smallint	
chk_finger_number	finger_number	
current_embedding_ref	varchar	
reason	text	
update_status	varchar	
requested_by	uuid	
requested_at	datetime	
reviewed_by	uuid	
reviewed_at	datetime	
resolution_notes	text	
created_at	datetime	
updated_at	datetime	
deleted_at	datetime	
created_by	uuid	
updated_by	uuid	
deleted_by	uuid	
row_version	bigint	

Quality

Table 35

quality_project

Attribute	Type	Key
project_id	bigint	PK
project_name	varchar	
description	varchar	
customer	varchar	
planned_start_on	date	
planned_end_on	date	
chk_quality_project_dates	planned_start_on	
project_status	varchar	
created_at	datetime	
updated_at	datetime	
deleted_at	datetime	
created_by	uuid	
updated_by	uuid	
deleted_by	uuid	
row_version	bigint	

Table 36

quality_evaluation

Attribute	Type	Key
evaluation_id	bigint	PK
service_name	varchar	
evaluator_name	varchar	
evaluation_scope	varchar	
project_id	bigint	FK
evaluation_status	varchar	
comments	text	
global_score	decimal	
percentage	decimal	
quality_level	varchar	
created_at	datetime	
updated_at	datetime	
deleted_at	datetime	
created_by	uuid	
updated_by	uuid	
deleted_by	uuid	
row_version	bigint	

Table 37

quality_evaluation_item

Attribute	Type	Key
evaluation_item_id	bigint	PK
evaluation_id	bigint	FK
subcharacteristic_code	varchar	
score	smallint	
created_at	datetime	
updated_at	datetime	
deleted_at	datetime	
created_by	uuid	
updated_by	uuid	
deleted_by	uuid	
row_version	bigint	

Table 38

process_assessment

Attribute	Type	Key
assessment_id	bigint	PK
project_id	bigint	FK
process_code	varchar	
assessor_name	varchar	
assessment_status	varchar	
comments	text	
process_score	decimal	
process_rating	varchar	
created_at	datetime	
updated_at	datetime	
deleted_at	datetime	
created_by	uuid	
updated_by	uuid	
deleted_by	uuid	
row_version	bigint	

Table 39

process_assessment_rating

Attribute	Type	Key
process_rating_id	bigint	PK
assessment_id	bigint	FK
objective_code	varchar	
objective_rating	varchar	
created_at	datetime	
updated_at	datetime	
deleted_at	datetime	
created_by	uuid	
updated_by	uuid	
deleted_by	uuid	
row_version	bigint	

Table 40

istqb_assessment

Attribute	Type	Key
istqb_assessment_id	bigint	PK
service_name	varchar	
evaluator_name	varchar	
assessment_scope	varchar	
project_id	bigint	FK
assessment_status	varchar	
comments	text	
global_score	decimal	
percentage	decimal	
quality_level	varchar	
created_at	datetime	
updated_at	datetime	
deleted_at	datetime	
created_by	uuid	
updated_by	uuid	
deleted_by	uuid	
row_version	bigint	

Table 41

istqb_assessment_item

Attribute	Type	Key
istqb_assessment_item_id	bigint	PK
istqb_assessment_id	bigint	FK
item_code	varchar	
score	smallint	
created_at	datetime	
updated_at	datetime	
deleted_at	datetime	
created_by	uuid	
updated_by	uuid	
deleted_by	uuid	
row_version	bigint	

Physical model

PostgreSQL script exported from the DBML. The DBML groups tables by context and does not declare the schema, so the script does not include it. In the real database each context has its own schema, and the migrations that run in production are the Liquibase changelogs in the main repository.

```
CREATE TABLE "person" (
  "person_id" uuid PRIMARY KEY NOT NULL,
  "document_number" varchar NOT NULL,
  "name" varchar NOT NULL,
  "last_name" varchar NOT NULL,
  "email" varchar,
  "phone" varchar,
  "document_type" varchar,
  "blood_type" varchar,
  "birth_date" date,
  "address" varchar,
  "status" boolean NOT NULL DEFAULT true,
  "created_at" datetime NOT NULL,
  "updated_at" datetime NOT NULL,
  "deleted_at" datetime,
  "created_by" uuid,
  "updated_by" uuid,
  "deleted_by" uuid,
  "row_version" bigint NOT NULL DEFAULT 1
);

CREATE TABLE "app_user" (
  "user_id" uuid PRIMARY KEY NOT NULL,
  "person_id" uuid UNIQUE NOT NULL,
  "username" varchar NOT NULL,
  "password_hash" varchar NOT NULL,
  "authentication_type" varchar NOT NULL,
  "status" boolean NOT NULL DEFAULT true,
  "must_change_password" boolean NOT NULL DEFAULT false,
  "two_factor_enabled" boolean NOT NULL DEFAULT false,
  "last_access" datetime,
  "created_at" datetime NOT NULL,
  "updated_at" datetime NOT NULL,
  "deleted_at" datetime,
  "created_by" uuid,
  "updated_by" uuid,
  "deleted_by" uuid,
  "row_version" bigint NOT NULL DEFAULT 1
);

CREATE TABLE "guardian" (
  "guardian_id" uuid PRIMARY KEY NOT NULL,
  "minor_person_id" uuid NOT NULL,
  "full_name" varchar NOT NULL,
  "document_type" varchar NOT NULL,
  "document_number" varchar NOT NULL,
  "email" varchar,
  "phone" varchar,
```

```
  "relationship" varchar NOT NULL,
  "school_id" bigint,
  "consent_document_uri" varchar,
  "consent_document_name" varchar,
  "consent_document_mime" varchar,
  "consent_document_size" bigint,
  "consent_document_uploaded_at" datetime,
  "created_at" datetime NOT NULL,
  "updated_at" datetime NOT NULL,
  "deleted_at" datetime,
  "created_by" uuid,
  "updated_by" uuid,
  "deleted_by" uuid,
  "row_version" bigint NOT NULL DEFAULT 1
);

CREATE TABLE "terms_acceptance" (
  "acceptance_id" uuid PRIMARY KEY NOT NULL,
  "person_id" uuid NOT NULL,
  "terms_version" varchar NOT NULL,
  "user_type" varchar NOT NULL,
  "accepted_by" varchar NOT NULL,
  "guardian_id" uuid,
  "channel" varchar NOT NULL,
  "recorded_by" uuid,
  "school_id" bigint,
  "accepted_at" datetime NOT NULL,
  "created_at" datetime NOT NULL,
  "updated_at" datetime NOT NULL,
  "deleted_at" datetime,
  "created_by" uuid,
  "updated_by" uuid,
  "deleted_by" uuid,
  "row_version" bigint NOT NULL DEFAULT 1
);

CREATE TABLE "user_session" (
  "session_id" uuid PRIMARY KEY NOT NULL,
  "user_id" uuid NOT NULL,
  "start_date" datetime NOT NULL,
  "end_date" datetime,
  "source_ip" varchar,
  "session_status" varchar NOT NULL,
  "created_at" datetime NOT NULL,
  "updated_at" datetime NOT NULL,
  "deleted_at" datetime,
  "created_by" uuid,
  "updated_by" uuid,
  "deleted_by" uuid,
  "row_version" bigint NOT NULL DEFAULT 1
);

CREATE TABLE "password_policy" (
  "policy_id" INT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  "min_length" int NOT NULL DEFAULT 8,
  "max_length" int NOT NULL DEFAULT 20,
  "requires_uppercase" boolean NOT NULL DEFAULT true,
  "requires_numbers" boolean NOT NULL DEFAULT true,
  "requires_symbols" boolean NOT NULL DEFAULT true,
  "expiration_days" int NOT NULL DEFAULT 90,
  "created_at" datetime NOT NULL,
  "updated_at" datetime NOT NULL,
  "deleted_at" datetime,
  "created_by" uuid,
  "updated_by" uuid,
  "deleted_by" uuid,
  "row_version" bigint NOT NULL DEFAULT 1
);

CREATE TABLE "data_processing_consent" (
  "consent_id" uuid PRIMARY KEY NOT NULL,
  "person_id" uuid NOT NULL,
  "policy_version" varchar(20) NOT NULL,
  "accepted_at" datetime NOT NULL,
  "biometric_accepted" boolean NOT NULL DEFAULT false,
  "biometric_revoked_at" datetime,
  "channel" varchar(10) NOT NULL,
```

```
  "recorded_by" uuid,
  "created_at" datetime NOT NULL,
  "updated_at" datetime NOT NULL,
  "deleted_at" datetime,
  "created_by" uuid,
  "updated_by" uuid,
  "deleted_by" uuid,
  "row_version" bigint NOT NULL DEFAULT 1
);

CREATE TABLE "role" (
  "role_id" INT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  "role_name" varchar UNIQUE NOT NULL,
  "description" varchar,
  "created_at" datetime NOT NULL,
  "updated_at" datetime NOT NULL,
  "deleted_at" datetime,
  "created_by" uuid,
  "updated_by" uuid,
  "deleted_by" uuid,
  "row_version" bigint NOT NULL DEFAULT 1
);

CREATE TABLE "permission" (
  "permission_id" INT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  "permission_name" varchar UNIQUE NOT NULL,
  "description" varchar,
  "created_at" datetime NOT NULL,
  "updated_at" datetime NOT NULL,
  "deleted_at" datetime,
  "created_by" uuid,
  "updated_by" uuid,
  "deleted_by" uuid,
  "row_version" bigint NOT NULL DEFAULT 1
);

CREATE TABLE "role_permission" (
  "role_id" int NOT NULL,
  "permission_id" int NOT NULL,
  "assignment_date" datetime NOT NULL,
  "created_at" datetime NOT NULL,
  "updated_at" datetime NOT NULL,
  "deleted_at" datetime,
  "created_by" uuid,
  "updated_by" uuid,
  "deleted_by" uuid,
  "row_version" bigint NOT NULL DEFAULT 1,
  PRIMARY KEY ("role_id", "permission_id")
);

CREATE TABLE "user_role" (
  "user_id" uuid NOT NULL,
  "role_id" int NOT NULL,
  "assignment_date" datetime NOT NULL,
  "created_at" datetime NOT NULL,
  "updated_at" datetime NOT NULL,
  "deleted_at" datetime,
  "created_by" uuid,
  "updated_by" uuid,
  "deleted_by" uuid,
  "row_version" bigint NOT NULL DEFAULT 1,
  PRIMARY KEY ("user_id", "role_id")
);

CREATE TABLE "school" (
  "school_id" INT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  "code" varchar NOT NULL,
  "name" varchar NOT NULL,
  "city_id" varchar,
  "address" varchar,
  "district" varchar,
  "country" varchar,
  "city_name" varchar,
  "phone" varchar,
  "email" varchar,
  "status" boolean NOT NULL DEFAULT true,
  "created_at" datetime NOT NULL,
```

```
  "updated_at" datetime NOT NULL,
  "deleted_at" datetime,
  "created_by" uuid,
  "updated_by" uuid,
  "deleted_by" uuid,
  "row_version" bigint NOT NULL DEFAULT 1
);

CREATE TABLE "program" (
  "program_id" INT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  "school_id" int NOT NULL,
  "code" varchar NOT NULL,
  "name" varchar NOT NULL,
  "status" boolean NOT NULL DEFAULT true,
  "created_at" datetime NOT NULL,
  "updated_at" datetime NOT NULL,
  "deleted_at" datetime,
  "created_by" uuid,
  "updated_by" uuid,
  "deleted_by" uuid,
  "row_version" bigint NOT NULL DEFAULT 1
);

CREATE TABLE "academic_period" (
  "academic_period_id" INT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  "school_id" int NOT NULL,
  "name" varchar NOT NULL,
  "starts_on" date NOT NULL,
  "ends_on" date NOT NULL,
  "is_active" boolean NOT NULL DEFAULT false,
  "created_at" datetime NOT NULL,
  "updated_at" datetime NOT NULL,
  "deleted_at" datetime,
  "created_by" uuid,
  "updated_by" uuid,
  "deleted_by" uuid,
  "row_version" bigint NOT NULL DEFAULT 1
);

CREATE TABLE "cohort" (
  "cohort_id" BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  "program_id" int NOT NULL,
  "academic_period_id" int NOT NULL,
  "code" varchar NOT NULL,
  "status" boolean NOT NULL DEFAULT true,
  "created_at" datetime NOT NULL,
  "updated_at" datetime NOT NULL,
  "deleted_at" datetime,
  "created_by" uuid,
  "updated_by" uuid,
  "deleted_by" uuid,
  "row_version" bigint NOT NULL DEFAULT 1
);

CREATE TABLE "course" (
  "course_id" INT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  "program_id" int NOT NULL,
  "code" varchar NOT NULL,
  "name" varchar NOT NULL,
  "credit_hours" smallint NOT NULL,
  "status" boolean NOT NULL DEFAULT true,
  "created_at" datetime NOT NULL,
  "updated_at" datetime NOT NULL,
  "deleted_at" datetime,
  "created_by" uuid,
  "updated_by" uuid,
  "deleted_by" uuid,
  "row_version" bigint NOT NULL DEFAULT 1
);

CREATE TABLE "academic_actor_type" (
  "actor_type_id" SMALLINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  "code" varchar UNIQUE NOT NULL,
  "name" varchar NOT NULL,
  "created_at" datetime NOT NULL,
  "updated_at" datetime NOT NULL,
  "deleted_at" datetime,
```

```
  "created_by" uuid,
  "updated_by" uuid,
  "deleted_by" uuid,
  "row_version" bigint NOT NULL DEFAULT 1
);

CREATE TABLE "academic_actor" (
  "academic_actor_id" BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  "person_id" uuid NOT NULL,
  "actor_type_id" smallint NOT NULL,
  "school_id" int NOT NULL,
  "actor_code" varchar NOT NULL,
  "started_on" date NOT NULL,
  "ended_on" date,
  "status" boolean NOT NULL DEFAULT true,
  "created_at" datetime NOT NULL,
  "updated_at" datetime NOT NULL,
  "deleted_at" datetime,
  "created_by" uuid,
  "updated_by" uuid,
  "deleted_by" uuid,
  "row_version" bigint NOT NULL DEFAULT 1
);

CREATE TABLE "enrollment" (
  "enrollment_id" BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  "academic_actor_id" bigint NOT NULL,
  "cohort_id" bigint NOT NULL,
  "enrolled_on" date NOT NULL,
  "enrollment_status" varchar NOT NULL,
  "created_at" datetime NOT NULL,
  "updated_at" datetime NOT NULL,
  "deleted_at" datetime,
  "created_by" uuid,
  "updated_by" uuid,
  "deleted_by" uuid,
  "row_version" bigint NOT NULL DEFAULT 1
);

CREATE TABLE "environment" (
  "environment_id" INT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  "school_id" int NOT NULL,
  "code" varchar NOT NULL,
  "name" varchar NOT NULL,
  "capacity" smallint NOT NULL,
  "status" boolean NOT NULL DEFAULT true,
  "created_at" datetime NOT NULL,
  "updated_at" datetime NOT NULL,
  "deleted_at" datetime,
  "created_by" uuid,
  "updated_by" uuid,
  "deleted_by" uuid,
  "row_version" bigint NOT NULL DEFAULT 1
);

CREATE TABLE "schedule_block" (
  "schedule_block_id" BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  "cohort_id" bigint NOT NULL,
  "course_id" int NOT NULL,
  "environment_id" int NOT NULL,
  "instructor_actor_id" bigint NOT NULL,
  "day_of_week" smallint NOT NULL,
  "starts_at" time NOT NULL,
  "ends_at" time NOT NULL,
  "created_at" datetime NOT NULL,
  "updated_at" datetime NOT NULL,
  "deleted_at" datetime,
  "created_by" uuid,
  "updated_by" uuid,
  "deleted_by" uuid,
  "row_version" bigint NOT NULL DEFAULT 1
);

CREATE TABLE "class_session" (
  "class_session_id" BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  "schedule_block_id" bigint NOT NULL,
  "session_date" date NOT NULL,
```

```
  "opened_by" bigint,
  "opened_at" datetime,
  "closed_by" bigint,
  "closed_at" datetime,
  "session_status" varchar NOT NULL,
  "created_at" datetime NOT NULL,
  "updated_at" datetime NOT NULL,
  "deleted_at" datetime,
  "created_by" uuid,
  "updated_by" uuid,
  "deleted_by" uuid,
  "row_version" bigint NOT NULL DEFAULT 1
);

CREATE TABLE "attendance_record" (
  "attendance_record_id" BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  "class_session_id" bigint NOT NULL,
  "academic_actor_id" bigint NOT NULL,
  "attendance_status" varchar NOT NULL,
  "captured_at" datetime,
  "capture_method" varchar NOT NULL,
  "match_score" decimal(5,4),
  "created_at" datetime NOT NULL,
  "updated_at" datetime NOT NULL,
  "deleted_at" datetime,
  "created_by" uuid,
  "updated_by" uuid,
  "deleted_by" uuid,
  "row_version" bigint NOT NULL DEFAULT 1
);

CREATE TABLE "justification_type" (
  "justification_type_id" INT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  "school_id" int,
  "name" varchar NOT NULL,
  "description" varchar,
  "requires_attachment" boolean NOT NULL DEFAULT false,
  "status" boolean NOT NULL DEFAULT true,
  "created_at" datetime NOT NULL,
  "updated_at" datetime NOT NULL,
  "deleted_at" datetime,
  "created_by" uuid,
  "updated_by" uuid,
  "deleted_by" uuid,
  "row_version" bigint NOT NULL DEFAULT 1
);

CREATE TABLE "justification" (
  "justification_id" BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  "attendance_record_id" bigint NOT NULL,
  "justification_type_id" int NOT NULL,
  "reason" text NOT NULL,
  "submitted_at" datetime NOT NULL,
  "reviewed_by" uuid,
  "reviewed_at" datetime,
  "review_status" varchar NOT NULL,
  "resolution_notes" text,
  "created_at" datetime NOT NULL,
  "updated_at" datetime NOT NULL,
  "deleted_at" datetime,
  "created_by" uuid,
  "updated_by" uuid,
  "deleted_by" uuid,
  "row_version" bigint NOT NULL DEFAULT 1
);

CREATE TABLE "supporting_document" (
  "supporting_document_id" BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  "justification_id" bigint NOT NULL,
  "file_name" varchar NOT NULL,
  "storage_uri" varchar NOT NULL,
  "mime_type" varchar NOT NULL,
  "size_bytes" bigint NOT NULL,
  "created_at" datetime NOT NULL,
  "updated_at" datetime NOT NULL,
  "deleted_at" datetime,
  "created_by" uuid,
```

```
  "updated_by" uuid,
  "deleted_by" uuid,
  "row_version" bigint NOT NULL DEFAULT 1
);

CREATE TABLE "attendance_report" (
  "report_id" uuid PRIMARY KEY NOT NULL,
  "school_id" int,
  "cohort_id" bigint,
  "generated_by" uuid,
  "report_type" varchar NOT NULL,
  "filters" jsonb,
  "result" jsonb,
  "generated_at" datetime NOT NULL,
  "created_at" datetime NOT NULL,
  "updated_at" datetime NOT NULL,
  "deleted_at" datetime,
  "created_by" uuid,
  "updated_by" uuid,
  "deleted_by" uuid,
  "row_version" bigint NOT NULL DEFAULT 1
);

CREATE TABLE "alert_type" (
  "alert_type_id" SMALLINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  "code" varchar UNIQUE NOT NULL,
  "name" varchar NOT NULL,
  "severity" varchar NOT NULL,
  "channel" varchar NOT NULL DEFAULT 'DASHBOARD',
  "created_at" datetime NOT NULL,
  "updated_at" datetime NOT NULL,
  "deleted_at" datetime,
  "created_by" uuid,
  "updated_by" uuid,
  "deleted_by" uuid,
  "row_version" bigint NOT NULL DEFAULT 1
);

CREATE TABLE "alert" (
  "alert_id" BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  "academic_actor_id" bigint NOT NULL,
  "alert_type_id" smallint NOT NULL,
  "raised_at" datetime NOT NULL,
  "resolved_at" datetime,
  "created_at" datetime NOT NULL,
  "updated_at" datetime NOT NULL,
  "deleted_at" datetime,
  "created_by" uuid,
  "updated_by" uuid,
  "deleted_by" uuid,
  "row_version" bigint NOT NULL DEFAULT 1
);

CREATE TABLE "notification_log" (
  "log_id" BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  "kind" varchar NOT NULL,
  "dedupe_key" varchar NOT NULL,
  "school_id" int,
  "sent_at" datetime NOT NULL,
  "created_at" datetime NOT NULL,
  "updated_at" datetime NOT NULL,
  "deleted_at" datetime,
  "created_by" uuid,
  "updated_by" uuid,
  "deleted_by" uuid,
  "row_version" bigint NOT NULL DEFAULT 1
);

CREATE TABLE "device_token" (
  "device_token_id" BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  "academic_actor_id" bigint NOT NULL,
  "token" varchar(255) NOT NULL,
  "platform" varchar(10) NOT NULL,
  "locale" varchar(5) NOT NULL DEFAULT 'es',
  "created_at" datetime NOT NULL DEFAULT (now()),
  "updated_at" datetime NOT NULL DEFAULT (now()),
  "deleted_at" datetime,
```

```
  "created_by" uuid,
  "updated_by" uuid,
  "deleted_by" uuid,
  "row_version" bigint NOT NULL DEFAULT 1,
  "revoked_at" datetime
);

CREATE TABLE "academic_configuration" (
  "configuration_id" INT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  "school_id" int NOT NULL,
  "configuration_name" varchar NOT NULL,
  "configuration_value" varchar NOT NULL,
  "description" varchar,
  "created_at" datetime NOT NULL,
  "updated_at" datetime NOT NULL,
  "deleted_at" datetime,
  "created_by" uuid,
  "updated_by" uuid,
  "deleted_by" uuid,
  "row_version" bigint NOT NULL DEFAULT 1
);

CREATE TABLE "security_configuration" (
  "configuration_id" INT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  "configuration_name" varchar NOT NULL,
  "configuration_value" varchar NOT NULL,
  "description" varchar,
  "created_at" datetime NOT NULL,
  "updated_at" datetime NOT NULL,
  "deleted_at" datetime,
  "created_by" uuid,
  "updated_by" uuid,
  "deleted_by" uuid,
  "row_version" bigint NOT NULL DEFAULT 1
);

CREATE TABLE "biometric_update_case" (
  "case_id" uuid PRIMARY KEY NOT NULL,
  "person_id" uuid NOT NULL,
  "biometric_type" varchar NOT NULL,
  "finger_number" smallint,
  "chk_finger_number" finger_number,
  "current_embedding_ref" varchar,
  "reason" text NOT NULL,
  "update_status" varchar NOT NULL DEFAULT 'Pending',
  "requested_by" uuid NOT NULL,
  "requested_at" datetime NOT NULL,
  "reviewed_by" uuid,
  "reviewed_at" datetime,
  "resolution_notes" text,
  "created_at" datetime NOT NULL,
  "updated_at" datetime NOT NULL,
  "deleted_at" datetime,
  "created_by" uuid,
  "updated_by" uuid,
  "deleted_by" uuid,
  "row_version" bigint NOT NULL DEFAULT 1
);

CREATE TABLE "quality_project" (
  "project_id" BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  "project_name" varchar NOT NULL,
  "description" varchar,
  "customer" varchar,
  "planned_start_on" date,
  "planned_end_on" date,
  "chk_quality_project_dates" planned_start_on,
  "project_status" varchar NOT NULL DEFAULT 'Planned',
  "created_at" datetime NOT NULL,
  "updated_at" datetime NOT NULL,
  "deleted_at" datetime,
  "created_by" uuid,
  "updated_by" uuid,
  "deleted_by" uuid,
  "row_version" bigint NOT NULL DEFAULT 1
);
```

```
CREATE TABLE "quality_evaluation" (
  "evaluation_id" BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  "service_name" varchar NOT NULL,
  "evaluator_name" varchar NOT NULL,
  "evaluation_scope" varchar NOT NULL,
  "project_id" bigint,
  "evaluation_status" varchar NOT NULL DEFAULT 'Draft',
  "comments" text,
  "global_score" decimal,
  "percentage" decimal,
  "quality_level" varchar,
  "created_at" datetime NOT NULL,
  "updated_at" datetime NOT NULL,
  "deleted_at" datetime,
  "created_by" uuid,
  "updated_by" uuid,
  "deleted_by" uuid,
  "row_version" bigint NOT NULL DEFAULT 1
);

CREATE TABLE "quality_evaluation_item" (
  "evaluation_item_id" BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  "evaluation_id" bigint NOT NULL,
  "subcharacteristic_code" varchar NOT NULL,
  "score" smallint NOT NULL,
  "created_at" datetime NOT NULL,
  "updated_at" datetime NOT NULL,
  "deleted_at" datetime,
  "created_by" uuid,
  "updated_by" uuid,
  "deleted_by" uuid,
  "row_version" bigint NOT NULL DEFAULT 1
);

CREATE TABLE "process_assessment" (
  "assessment_id" BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  "project_id" bigint NOT NULL,
  "process_code" varchar NOT NULL,
  "assessor_name" varchar NOT NULL,
  "assessment_status" varchar NOT NULL DEFAULT 'Draft',
  "comments" text,
  "process_score" decimal,
  "process_rating" varchar,
  "created_at" datetime NOT NULL,
  "updated_at" datetime NOT NULL,
  "deleted_at" datetime,
  "created_by" uuid,
  "updated_by" uuid,
  "deleted_by" uuid,
  "row_version" bigint NOT NULL DEFAULT 1
);

CREATE TABLE "process_assessment_rating" (
  "process_rating_id" BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  "assessment_id" bigint NOT NULL,
  "objective_code" varchar NOT NULL,
  "objective_rating" varchar NOT NULL,
  "created_at" datetime NOT NULL,
  "updated_at" datetime NOT NULL,
  "deleted_at" datetime,
  "created_by" uuid,
  "updated_by" uuid,
  "deleted_by" uuid,
  "row_version" bigint NOT NULL DEFAULT 1
);

CREATE TABLE "istqb_assessment" (
  "istqb_assessment_id" BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  "service_name" varchar NOT NULL,
  "evaluator_name" varchar NOT NULL,
  "assessment_scope" varchar NOT NULL,
  "project_id" bigint,
  "assessment_status" varchar NOT NULL DEFAULT 'Draft',
  "comments" text,
  "global_score" decimal,
  "percentage" decimal,
  "quality_level" varchar,
```

```
  "created_at" datetime NOT NULL,
  "updated_at" datetime NOT NULL,
  "deleted_at" datetime,
  "created_by" uuid,
  "updated_by" uuid,
  "deleted_by" uuid,
  "row_version" bigint NOT NULL DEFAULT 1
);

CREATE TABLE "istqb_assessment_item" (
  "istqb_assessment_item_id" BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  "istqb_assessment_id" bigint NOT NULL,
  "item_code" varchar NOT NULL,
  "score" smallint NOT NULL,
  "created_at" datetime NOT NULL,
  "updated_at" datetime NOT NULL,
  "deleted_at" datetime,
  "created_by" uuid,
  "updated_by" uuid,
  "deleted_by" uuid,
  "row_version" bigint NOT NULL DEFAULT 1
);

CREATE UNIQUE INDEX "uq_person_document" ON "person" ("document_type", "document_number");

CREATE UNIQUE INDEX "uq_person_email" ON "person" ("email") WHERE deleted_at IS NULL;

CREATE INDEX "idx_person_deleted" ON "person" ("deleted_at");

CREATE UNIQUE INDEX "uq_user_username" ON "app_user" ("username");

CREATE UNIQUE INDEX "uq_guardian_minor_active" ON "guardian" ("minor_person_id") WHERE deleted_at IS NULL;

CREATE INDEX "idx_terms_person_accepted" ON "terms_acceptance" ("person_id", "accepted_at");

CREATE INDEX "idx_session_user" ON "user_session" ("user_id");

CREATE INDEX "idx_session_status" ON "user_session" ("session_status");

CREATE INDEX "idx_consent_person_accepted" ON "data_processing_consent" ("person_id", "accepted_at");

CREATE UNIQUE INDEX "uq_school_code" ON "school" ("code") WHERE deleted_at IS NULL;

CREATE UNIQUE INDEX "uq_program_code_school" ON "program" ("school_id", "code") WHERE deleted_at IS NULL;

CREATE UNIQUE INDEX "uq_period_name_school" ON "academic_period" ("school_id", "name") WHERE deleted_at IS
NULL;

CREATE UNIQUE INDEX "uq_cohort_code" ON "cohort" ("code") WHERE deleted_at IS NULL;

CREATE UNIQUE INDEX "uq_course_code_program" ON "course" ("program_id", "code") WHERE deleted_at IS NULL;

CREATE UNIQUE INDEX "uq_actor_code_school_type" ON "academic_actor" ("school_id", "actor_type_id",
"actor_code") WHERE deleted_at IS NULL;

CREATE UNIQUE INDEX "uq_actor_person_type_school" ON "academic_actor" ("person_id", "actor_type_id",
"school_id") WHERE deleted_at IS NULL;

CREATE INDEX "idx_actor_deleted" ON "academic_actor" ("deleted_at");

CREATE UNIQUE INDEX "uq_enrollment_actor_cohort" ON "enrollment" ("academic_actor_id", "cohort_id") WHERE
deleted_at IS NULL;

CREATE INDEX "idx_enrollment_cohort" ON "enrollment" ("cohort_id");

CREATE INDEX "idx_enrollment_deleted" ON "enrollment" ("deleted_at");

CREATE UNIQUE INDEX "uq_environment_code_school" ON "environment" ("school_id", "code") WHERE deleted_at IS
NULL;

CREATE UNIQUE INDEX "uq_block_environment_slot" ON "schedule_block" ("environment_id", "day_of_week",
"starts_at") WHERE deleted_at IS NULL;

CREATE UNIQUE INDEX "uq_block_instructor_slot" ON "schedule_block" ("instructor_actor_id", "day_of_week",
"starts_at") WHERE deleted_at IS NULL;

CREATE INDEX "idx_block_cohort" ON "schedule_block" ("cohort_id");
```

```
CREATE INDEX "idx_block_course" ON "schedule_block" ("course_id");

CREATE UNIQUE INDEX "uq_session_block_date" ON "class_session" ("schedule_block_id", "session_date") WHERE
deleted_at IS NULL;

CREATE INDEX "idx_session_date" ON "class_session" ("session_date");

CREATE UNIQUE INDEX "uq_attendance_session_actor" ON "attendance_record" ("class_session_id",
"academic_actor_id") WHERE deleted_at IS NULL;

CREATE INDEX "idx_attendance_actor" ON "attendance_record" ("academic_actor_id");

CREATE INDEX "idx_attendance_deleted" ON "attendance_record" ("deleted_at");

CREATE UNIQUE INDEX "uq_justification_type_school_name" ON "justification_type" ("school_id", "name") WHERE
deleted_at IS NULL;

CREATE INDEX "idx_justification_type_school" ON "justification_type" ("school_id");

CREATE UNIQUE INDEX "uq_justification_type_global_name" ON "justification_type" ("name") WHERE school_id IS
NULL AND deleted_at IS NULL;

CREATE UNIQUE INDEX "uq_justification_attendance_record" ON "justification" ("attendance_record_id") WHERE
deleted_at IS NULL;

CREATE INDEX "idx_justification_type" ON "justification" ("justification_type_id");

CREATE INDEX "idx_justification_reviewer" ON "justification" ("reviewed_by");

CREATE INDEX "idx_document_justification" ON "supporting_document" ("justification_id");

CREATE INDEX "idx_report_school" ON "attendance_report" ("school_id");

CREATE INDEX "idx_report_cohort" ON "attendance_report" ("cohort_id");

CREATE INDEX "idx_alert_actor" ON "alert" ("academic_actor_id");

CREATE INDEX "idx_alert_type" ON "alert" ("alert_type_id");

CREATE UNIQUE INDEX "uq_notification_log_dedupe" ON "notification_log" ("kind", "dedupe_key") WHERE
deleted_at IS NULL;

CREATE INDEX "idx_notification_log_school" ON "notification_log" ("school_id");

CREATE UNIQUE INDEX "uq_device_token_token" ON "device_token" ("token");

CREATE INDEX "idx_device_token_actor_active" ON "device_token" ("academic_actor_id");

CREATE UNIQUE INDEX "uq_academic_configuration" ON "academic_configuration" ("school_id",
"configuration_name") WHERE deleted_at IS NULL;

CREATE UNIQUE INDEX "uq_security_configuration_name" ON "security_configuration" ("configuration_name")
WHERE deleted_at IS NULL;

CREATE INDEX "idx_biometric_case_person" ON "biometric_update_case" ("person_id");

CREATE INDEX "idx_biometric_case_person_type" ON "biometric_update_case" ("person_id", "biometric_type");

CREATE INDEX "idx_biometric_case_status" ON "biometric_update_case" ("update_status");

CREATE INDEX "idx_quality_project_status" ON "quality_project" ("project_status");

CREATE INDEX "idx_quality_project_deleted" ON "quality_project" ("deleted_at");

CREATE INDEX "idx_quality_evaluation_service" ON "quality_evaluation" ("service_name");

CREATE INDEX "idx_quality_evaluation_status" ON "quality_evaluation" ("evaluation_status");

CREATE INDEX "idx_quality_evaluation_project" ON "quality_evaluation" ("project_id");

CREATE INDEX "idx_quality_evaluation_deleted" ON "quality_evaluation" ("deleted_at");

CREATE UNIQUE INDEX "uq_quality_evaluation_item_code" ON "quality_evaluation_item" ("evaluation_id",
"subcharacteristic_code");

CREATE INDEX "idx_process_assessment_project" ON "process_assessment" ("project_id");
```

```
CREATE INDEX "idx_process_assessment_process" ON "process_assessment" ("process_code");

CREATE INDEX "idx_process_assessment_deleted" ON "process_assessment" ("deleted_at");

CREATE UNIQUE INDEX "uq_process_assessment_rating_objective" ON "process_assessment_rating"
("assessment_id", "objective_code");

CREATE INDEX "idx_istqb_assessment_service" ON "istqb_assessment" ("service_name");

CREATE INDEX "idx_istqb_assessment_status" ON "istqb_assessment" ("assessment_status");

CREATE INDEX "idx_istqb_assessment_project" ON "istqb_assessment" ("project_id");

CREATE INDEX "idx_istqb_assessment_deleted" ON "istqb_assessment" ("deleted_at");

CREATE UNIQUE INDEX "uq_istqb_assessment_item_code" ON "istqb_assessment_item" ("istqb_assessment_id",
"item_code");

COMMENT ON COLUMN "person"."document_type" IS 'CC / TI / CE / PP';

COMMENT ON COLUMN "person"."blood_type" IS 'A+, A-, B+, B-, AB+, AB-, O+, O-';

COMMENT ON COLUMN "app_user"."authentication_type" IS 'Local, Windows, External';

COMMENT ON COLUMN "app_user"."must_change_password" IS 'true cuando las credenciales iniciales se
entregaron por correo; se apaga al cambiar la contraseña';

COMMENT ON COLUMN "app_user"."two_factor_enabled" IS 'true cuando la persona activó el código por correo
como segundo factor al iniciar sesión';

COMMENT ON COLUMN "guardian"."minor_person_id" IS 'person menor de edad (FK identity.person)';

COMMENT ON COLUMN "guardian"."full_name" IS 'cifrado AES-256-GCM';

COMMENT ON COLUMN "guardian"."document_type" IS 'CC / CE / PP';

COMMENT ON COLUMN "guardian"."document_number" IS 'cifrado AES-256-GCM';

COMMENT ON COLUMN "guardian"."email" IS 'cifrado; correo o teléfono obligatorio';

COMMENT ON COLUMN "guardian"."phone" IS 'cifrado';

COMMENT ON COLUMN "guardian"."school_id" IS 'sede donde se registró (academic.school, sin FK)';

COMMENT ON COLUMN "guardian"."consent_document_uri" IS 'autorización firmada del acudiente en MinIO
(<bucket>/<clave>); obligatoria para habilitar al menor';

COMMENT ON COLUMN "guardian"."consent_document_name" IS 'nombre de presentación del archivo (sin ruta)';

COMMENT ON COLUMN "guardian"."consent_document_mime" IS 'application/pdf, image/jpeg o image/png (detectado
por firma, no por extensión)';

COMMENT ON COLUMN "guardian"."consent_document_size" IS 'bytes (máx. 5 MB)';

COMMENT ON COLUMN "terms_acceptance"."user_type" IS 'ADULT / MINOR';

COMMENT ON COLUMN "terms_acceptance"."accepted_by" IS 'SELF / GUARDIAN';

COMMENT ON COLUMN "terms_acceptance"."guardian_id" IS 'obligatorio si accepted_by = GUARDIAN';

COMMENT ON COLUMN "terms_acceptance"."channel" IS 'self / assisted';

COMMENT ON COLUMN "terms_acceptance"."recorded_by" IS 'usuario que registró la aceptación';

COMMENT ON COLUMN "user_session"."session_status" IS 'Active or Closed';

COMMENT ON TABLE "data_processing_consent" IS 'Una fila por aceptación. La PK se llama
data_processing_consent_pkey (default de PostgreSQL) en lugar de pk_data_processing_consent.';

COMMENT ON COLUMN "data_processing_consent"."person_id" IS 'Titular que autoriza; FK intra-contexto (ON
DELETE CASCADE)';

COMMENT ON COLUMN "data_processing_consent"."policy_version" IS 'Versión de la política de privacidad
aceptada';

COMMENT ON COLUMN "data_processing_consent"."accepted_at" IS 'Ley 1581 de 2012, arts. 9 y 17-b';
```

```
COMMENT ON COLUMN "data_processing_consent"."biometric_accepted" IS 'Autorización opcional del tratamiento
de datos biométricos (art. 6-a)';

COMMENT ON COLUMN "data_processing_consent"."biometric_revoked_at" IS 'Revocatoria de la parte biométrica';

COMMENT ON COLUMN "data_processing_consent"."channel" IS 'CHECK ck_consent_channel: self (el titular) /
assisted (personal con el titular presente)';

COMMENT ON COLUMN "data_processing_consent"."recorded_by" IS 'Cross-context: identity.app_user.user_id de
quien registra (NO FK)';

COMMENT ON COLUMN "school"."city_id" IS 'External catalog via countriesnow API, no FK';

COMMENT ON COLUMN "academic_actor_type"."code" IS 'STUDENT / INSTRUCTOR / ADMIN (administrador de sede)';

COMMENT ON COLUMN "academic_actor"."actor_code" IS 'Código de aprendiz, instructor o administrador';

COMMENT ON COLUMN "enrollment"."enrollment_status" IS 'ENUM: Active / Withdrawn / Completed';

COMMENT ON COLUMN "schedule_block"."day_of_week" IS '1..7';

COMMENT ON COLUMN "class_session"."opened_by" IS 'academic_actor_id (instructor) que abrió la clase';

COMMENT ON COLUMN "class_session"."closed_by" IS 'academic_actor_id (instructor) que cerró la clase';

COMMENT ON COLUMN "class_session"."session_status" IS 'ENUM: Open / Closed / Cancelled';

COMMENT ON COLUMN "attendance_record"."attendance_status" IS 'ENUM: Present / Absent / Late / Justified';

COMMENT ON COLUMN "attendance_record"."capture_method" IS 'ENUM: FACIAL / MANUAL / FINGERPRINT / IMPORT';

COMMENT ON COLUMN "attendance_record"."match_score" IS 'Score del reconocimiento facial o dactilar';

COMMENT ON COLUMN "justification_type"."school_id" IS 'NULL = global type, NOT NULL = per-school custom
type';

COMMENT ON COLUMN "justification"."reviewed_by" IS 'app_user.user_id (cross-context)';

COMMENT ON COLUMN "justification"."review_status" IS 'ENUM: Pending / Approved / Rejected';

COMMENT ON COLUMN "attendance_report"."school_id" IS 'Cross-context to Academic.school';

COMMENT ON COLUMN "attendance_report"."cohort_id" IS 'Cross-context to Academic.cohort';

COMMENT ON COLUMN "attendance_report"."generated_by" IS 'Cross-context to Identity.app_user';

COMMENT ON COLUMN "alert_type"."severity" IS 'INFO / WARNING / CRITICAL';

COMMENT ON COLUMN "alert_type"."channel" IS 'DASHBOARD / EMAIL / PUSH';

COMMENT ON COLUMN "notification_log"."kind" IS 'ABSENCE_REPORT / AT_RISK_REPORT';

COMMENT ON COLUMN "notification_log"."dedupe_key" IS 'What it was about, e.g. session:1216; no personal
data';

COMMENT ON COLUMN "notification_log"."school_id" IS 'academic.school.school_id (cross-context, NO FK)';

COMMENT ON TABLE "device_token" IS 'Bloque de auditoría completo (AUD-33);
trg_device_token_audit_timestamps mantiene updated_at y row_version. La revocación del token sigue siendo
revoked_at; deleted_at es el borrado lógico y las lecturas filtran deleted_at IS NULL.';

COMMENT ON COLUMN "device_token"."academic_actor_id" IS 'academic.academic_actor.academic_actor_id (cross-
context, NO FK)';

COMMENT ON COLUMN "device_token"."token" IS 'ExponentPushToken[...]; registrarlo de nuevo lo reactiva para
el actor actual';

COMMENT ON COLUMN "device_token"."platform" IS 'CHECK ck_device_token_platform: ios / android';

COMMENT ON COLUMN "device_token"."locale" IS 'CHECK ck_device_token_locale: es / en / pt / fr';

COMMENT ON COLUMN "device_token"."revoked_at" IS 'Los tokens revocados se conservan y nunca reciben push';

COMMENT ON COLUMN "academic_configuration"."configuration_name" IS 'e.g., tardy_tolerance_minutes';

COMMENT ON COLUMN "biometric_update_case"."biometric_type" IS 'ENUM: FACIAL / FINGERPRINT';
```

```
COMMENT ON COLUMN "biometric_update_case"."finger_number" IS 'Obligatorio solo si biometric_type =
FINGERPRINT (1..10); nulo si es FACIAL';

COMMENT ON COLUMN "biometric_update_case"."chk_finger_number" IS 'CHECK: (biometric_type = FINGERPRINT AND
finger_number BETWEEN 1 AND 10) OR (biometric_type = FACIAL AND finger_number IS NULL)';

COMMENT ON COLUMN "biometric_update_case"."current_embedding_ref" IS '_id lógico del documento activo en
facial_embedding o fingerprint_embedding (NoSQL). Sin FK real: cross-paradigm.';

COMMENT ON COLUMN "biometric_update_case"."update_status" IS 'ENUM: Pending / In_Review / Approved /
Rejected';

COMMENT ON COLUMN "biometric_update_case"."requested_by" IS 'app_user.user_id (cross-context)';

COMMENT ON COLUMN "biometric_update_case"."reviewed_by" IS 'app_user.user_id (cross-context)';

COMMENT ON COLUMN "quality_project"."customer" IS 'Cliente/usuario que acepta la entrega (evidencia WP10)';

COMMENT ON COLUMN "quality_project"."chk_quality_project_dates" IS 'CHECK: planned_start_on <=
planned_end_on';

COMMENT ON COLUMN "quality_project"."project_status" IS 'ENUM: Planned / Active / Closed';

COMMENT ON COLUMN "quality_evaluation"."service_name" IS 'Microservicio evaluado, ej. 03-ms-academic (sin
FK)';

COMMENT ON COLUMN "quality_evaluation"."project_id" IS 'Trazabilidad ISO/IEC 29110 SI.O2';

COMMENT ON COLUMN "quality_evaluation"."evaluation_status" IS 'ENUM: Draft / Completed';

COMMENT ON COLUMN "quality_evaluation"."global_score" IS 'Puntaje ponderado 1-5, derivado de
quality_evaluation_item';

COMMENT ON COLUMN "quality_evaluation"."quality_level" IS 'ENUM: Deficiente / En proceso / Aceptable /
Bueno / Excelente';

COMMENT ON COLUMN "quality_evaluation_item"."subcharacteristic_code" IS 'Catálogo ISO/IEC 25010 en código,
sin FK';

COMMENT ON COLUMN "quality_evaluation_item"."score" IS 'CHECK: 1..5';

COMMENT ON COLUMN "process_assessment"."process_code" IS 'ENUM: PM (Gestión de Proyecto) / SI
(Implementación de Software)';

COMMENT ON COLUMN "process_assessment"."assessment_status" IS 'ENUM: Draft / Completed';

COMMENT ON COLUMN "process_assessment"."process_score" IS '% de logro 0-100, derivado de
process_assessment_rating';

COMMENT ON COLUMN "process_assessment"."process_rating" IS 'ENUM: N / P / L / F';

COMMENT ON COLUMN "process_assessment_rating"."objective_code" IS 'Catálogo ISO/IEC 29110 en código, ej.
PM.O1, sin FK';

COMMENT ON COLUMN "process_assessment_rating"."objective_rating" IS 'ENUM: N / P / L / F';

COMMENT ON COLUMN "istqb_assessment"."service_name" IS 'Microservicio evaluado (sin FK)';

COMMENT ON COLUMN "istqb_assessment"."project_id" IS 'Trazabilidad ISO/IEC 29110 SI.O2/SI.O7';

COMMENT ON COLUMN "istqb_assessment"."assessment_status" IS 'ENUM: Draft / Completed';

COMMENT ON COLUMN "istqb_assessment"."quality_level" IS 'ENUM: Deficiente / En proceso / Aceptable / Bueno
/ Excelente';

COMMENT ON COLUMN "istqb_assessment_item"."item_code" IS 'Catálogo ISTQB CTFL v4.0 en código, sin FK';

COMMENT ON COLUMN "istqb_assessment_item"."score" IS 'CHECK: 1..5';

ALTER TABLE "guardian" ADD FOREIGN KEY ("minor_person_id") REFERENCES "person" ("person_id") DEFERRABLE
INITIALLY IMMEDIATE;

ALTER TABLE "terms_acceptance" ADD FOREIGN KEY ("person_id") REFERENCES "person" ("person_id") DEFERRABLE
INITIALLY IMMEDIATE;

ALTER TABLE "terms_acceptance" ADD FOREIGN KEY ("guardian_id") REFERENCES "guardian" ("guardian_id")
```

```
DEFERRABLE INITIALLY IMMEDIATE;

ALTER TABLE "person" ADD FOREIGN KEY ("person_id") REFERENCES "app_user" ("person_id") DEFERRABLE INITIALLY
IMMEDIATE;

ALTER TABLE "user_session" ADD FOREIGN KEY ("user_id") REFERENCES "app_user" ("user_id") DEFERRABLE
INITIALLY IMMEDIATE;

ALTER TABLE "data_processing_consent" ADD FOREIGN KEY ("person_id") REFERENCES "person" ("person_id") ON
DELETE CASCADE DEFERRABLE INITIALLY IMMEDIATE;

ALTER TABLE "role_permission" ADD FOREIGN KEY ("role_id") REFERENCES "role" ("role_id") DEFERRABLE
INITIALLY IMMEDIATE;

ALTER TABLE "role_permission" ADD FOREIGN KEY ("permission_id") REFERENCES "permission" ("permission_id")
DEFERRABLE INITIALLY IMMEDIATE;

ALTER TABLE "user_role" ADD FOREIGN KEY ("role_id") REFERENCES "role" ("role_id") DEFERRABLE INITIALLY
IMMEDIATE;

ALTER TABLE "program" ADD FOREIGN KEY ("school_id") REFERENCES "school" ("school_id") DEFERRABLE INITIALLY
IMMEDIATE;

ALTER TABLE "academic_period" ADD FOREIGN KEY ("school_id") REFERENCES "school" ("school_id") DEFERRABLE
INITIALLY IMMEDIATE;

ALTER TABLE "cohort" ADD FOREIGN KEY ("program_id") REFERENCES "program" ("program_id") DEFERRABLE
INITIALLY IMMEDIATE;

ALTER TABLE "cohort" ADD FOREIGN KEY ("academic_period_id") REFERENCES "academic_period"
("academic_period_id") DEFERRABLE INITIALLY IMMEDIATE;

ALTER TABLE "course" ADD FOREIGN KEY ("program_id") REFERENCES "program" ("program_id") DEFERRABLE
INITIALLY IMMEDIATE;

ALTER TABLE "academic_actor" ADD FOREIGN KEY ("actor_type_id") REFERENCES "academic_actor_type"
("actor_type_id") DEFERRABLE INITIALLY IMMEDIATE;

ALTER TABLE "academic_actor" ADD FOREIGN KEY ("school_id") REFERENCES "school" ("school_id") DEFERRABLE
INITIALLY IMMEDIATE;

ALTER TABLE "enrollment" ADD FOREIGN KEY ("academic_actor_id") REFERENCES "academic_actor"
("academic_actor_id") DEFERRABLE INITIALLY IMMEDIATE;

ALTER TABLE "enrollment" ADD FOREIGN KEY ("cohort_id") REFERENCES "cohort" ("cohort_id") DEFERRABLE
INITIALLY IMMEDIATE;

ALTER TABLE "schedule_block" ADD FOREIGN KEY ("environment_id") REFERENCES "environment" ("environment_id")
DEFERRABLE INITIALLY IMMEDIATE;

ALTER TABLE "class_session" ADD FOREIGN KEY ("schedule_block_id") REFERENCES "schedule_block"
("schedule_block_id") DEFERRABLE INITIALLY IMMEDIATE;

ALTER TABLE "justification" ADD FOREIGN KEY ("attendance_record_id") REFERENCES "attendance_record"
("attendance_record_id") DEFERRABLE INITIALLY IMMEDIATE;

ALTER TABLE "justification" ADD FOREIGN KEY ("justification_type_id") REFERENCES "justification_type"
("justification_type_id") DEFERRABLE INITIALLY IMMEDIATE;

ALTER TABLE "supporting_document" ADD FOREIGN KEY ("justification_id") REFERENCES "justification"
("justification_id") DEFERRABLE INITIALLY IMMEDIATE;

ALTER TABLE "alert" ADD FOREIGN KEY ("alert_type_id") REFERENCES "alert_type" ("alert_type_id") DEFERRABLE
INITIALLY IMMEDIATE;

ALTER TABLE "quality_evaluation" ADD FOREIGN KEY ("project_id") REFERENCES "quality_project" ("project_id")
DEFERRABLE INITIALLY IMMEDIATE;

ALTER TABLE "quality_evaluation_item" ADD FOREIGN KEY ("evaluation_id") REFERENCES "quality_evaluation"
("evaluation_id") DEFERRABLE INITIALLY IMMEDIATE;

ALTER TABLE "process_assessment" ADD FOREIGN KEY ("project_id") REFERENCES "quality_project" ("project_id")
DEFERRABLE INITIALLY IMMEDIATE;

ALTER TABLE "process_assessment_rating" ADD FOREIGN KEY ("assessment_id") REFERENCES "process_assessment"
("assessment_id") DEFERRABLE INITIALLY IMMEDIATE;
```

```
ALTER TABLE "istqb_assessment" ADD FOREIGN KEY ("project_id") REFERENCES "quality_project" ("project_id")
DEFERRABLE INITIALLY IMMEDIATE;

ALTER TABLE "istqb_assessment_item" ADD FOREIGN KEY ("istqb_assessment_id") REFERENCES "istqb_assessment"
("istqb_assessment_id") DEFERRABLE INITIALLY IMMEDIATE;
```

Data model guide

1. Overview

FaceAttend-Edu is an **attendance management platform based on biometric recognition** for educational institutions. The data model supports:

- Management of people, users, and credentials
- Academic structure (campuses, programs, periods, cohorts, courses, actors, and enrollments)
- Schedules and class sessions
- Attendance recording from multiple sources (facial, manual, IoT, import)
- Justifications with document support
- Biometric template updates with an approval workflow (`biometric_update_case` unifies FACIAL/FINGERPRINT)
- Biometric templates (facial and fingerprint) stored as **NoSQL collections in MongoDB**
- Alerts and notifications
- Academic and security configuration
- Quality evaluation of the microservices (ISO/IEC 25010, ISO/IEC 29110, ISTQB) — **Quality** context

2. Bounded Contexts

2. Bounded Contexts

The FACEATTEND-EDU domain model (v6 — DBML) is organized into 9 bounded contexts, each responsible for a functional area of the business. This division makes it possible to delimit responsibilities, organize the model's entities, and differentiate the persistence mechanisms of each context.

2.1. Overview of the contexts flowchart TB ROOT(["FACEATTEND-EDU Domain model · v6"])

```
ROOT --> ID
ROOT --> AC
ROOT --> OP
ROOT --> SYS

subgraph ID["IDENTITY AND ACCESS CONTROL"]
    direction LR
    I["IDENTITY · 4 tables<br/><br/>person<br/>app_user<br/>user_session<br/>password_policy"]
    AU["AUTHORIZATION · 4 tables<br/><br/>role<br/>permission<br/>role_permission<br/>user_role"]
end

subgraph AC["ACADEMIC MANAGEMENT"]
    direction LR
    A["ACADEMIC · 8 tables<br/><br/>school · program<br/>academic_period ·
cohort<br/>course<br/>academic_actor_type<br/>academic_actor<br/>enrollment"]
    S["SCHEDULING · 3 tables<br/><br/>environment<br/>schedule_block<br/>class_session"]
end

subgraph OP["ATTENDANCE AND BIOMETRICS"]
    direction LR
    AT["ATTENDANCE · 5 tables<br/>
<br/>attendance_record<br/>justification_type<br/>justification<br/>supporting_document<br/>attendance_repo
rt"]
    BIO["BIOMETRIC · NoSQL<br/><br/>facial_emb<br/>fingerprint_emb<br/><br/>MongoDB"]
end
```

```
subgraph SYS["CONFIGURATION, NOTIFICATIONS AND QUALITY"]
    direction LR
    C["CONFIGURATION · 3 tables<br/><br/>academic_config<br/>security_config<br/>biometric_update_case"]
    N["NOTIFICATION · 2 tables<br/><br/>alert_type<br/>alert"]
    Q["QUALITY · 7 tables<br/>
<br/>quality_project<br/>quality_evaluation<br/>quality_evaluation_item<br/>process_assessment<br/>process_
assessment_rating<br/>istqb_assessment<br/>istqb_assessment_item"]
end

classDef root fill:#172554,color:#ffffff,stroke:#3b82f6,stroke-width:2px
classDef context fill:#e0f2fe,color:#0c4a6e,stroke:#0284c7,stroke-width:1px
classDef nosql fill:#fef3c7,color:#78350f,stroke:#d97706,stroke-width:1px
classDef quality fill:#f3e8ff,color:#581c87,stroke:#9333ea,stroke-width:1px

class ROOT root
class I,AU,A,S,AT,C,N context
class BIO nosql
class Q quality
```

2.2. Description of the contexts Bounded Context Tables / storage Responsibility IDENTITY 4 tables Management of people, users, sessions, and password policies. AUTHORIZATION 4 tables Administration of roles, permissions, and access assignments. ACADEMIC 8 tables Management of institutions, programs, academic periods, cohorts, courses, academic actors, and enrollments. SCHEDULING 3 tables Management of environments, schedule blocks, and class sessions. ATTENDANCE 5 tables Recording of attendance, justifications, supporting documents, and reports. BIOMETRIC NoSQL (MongoDB) Storage of facial and fingerprint biometric representations. CONFIGURATION 3 tables Administration of academic and security configurations and biometric update cases. NOTIFICATION 2 tables Management of alert types and alerts generated by the system. QUALITY 7 tables Management of projects, quality evaluations, process assessments, and ISTQB evaluations. 2.3. Relationship between users and people

In the IDENTITY context, the column app_user.person_id has a UNIQUE constraint, intended to establish a one-to-one relationship between app_user and person.

erDiagram PERSON o|--o| APP_USER : "person_id UNIQUE"

```
PERSON {
    bigint id PK
}

APP_USER {
    bigint id PK
    bigint person_id FK, UK
}
```

The one-to-one cardinality means that a person can be associated with a single user and that a user can be associated with a single person. To guarantee that every user has an associated person, person_id must also be NOT NULL; the foreign key must be defined to guarantee referential integrity.

2.4. Persistence and infrastructure considerations Relational persistence: the IDENTITY, AUTHORIZATION, ACADEMIC, SCHEDULING, ATTENDANCE, CONFIGURATION, NOTIFICATION, and QUALITY contexts are represented as relational tables. Document persistence: BIOMETRIC uses MongoDB to store the facial_emb and fingerprint_emb collections. Physical location of QUALITY: although it is an independent bounded context from the domain perspective, its 7 tables are hosted in the quality schema of the 07-ms-configuration-db database. Migration

management: there is no separate quality-liquibase service; the QUALITY tables are physically located inside the database associated with CONFIGURATION.

Consequently, the model comprises 9 business bounded contexts, although QUALITY shares persistence infrastructure with CONFIGURATION. This physical location does not mean that both contexts should be considered a single bounded context from a conceptual point of view.

3. Context: Identity

Responsibility: Identity of people, access credentials, sessions, and password policies.

Table 42

3. Context: Identity

Table	Description	PK
`person`	Base identity: document, name, contact	`person_id` (UUID)
`app_user`	Access credentials; 1:1 with `person` (`person_id` UNIQUE)	`user_id` (UUID)
`user_session`	Active/closed sessions	`session_id` (UUID)
`password_policy`	Password complexity rules	`policy_id` (INT)

Internal relationships (real FK):

- `app_user.person_id` → `person.person_id`
- `user_session.user_id` → `app_user.user_id`

Key design: `person` **does not have** `school_id`. Identity is campus-agnostic.

Historical note: the `city` table existed in this context but was **removed** (changeset `008-drop-city-table.yaml`). The geographic reference to a city is now resolved through an external catalog (countriesnow API), with no FK or dedicated table.

4. Context: Authorization

Responsibility: Role-based access control (RBAC).

Table 43

4. Context: Authorization

Table	Description	PK
`role`	Catalog of system roles	`role_id` (INT)
`permission`	Catalog of atomic permissions	`permission_id` (INT)
`role_permission`	N:N assignment of roles ↔ permissions	`(role_id, permission_id)`
`user_role`	N:N assignment of users ↔ roles	`(user_id, role_id)`

Cross-context (no FK):

- `user_role.user_id` → `Identity.app_user.user_id`

Key design: `role` **does not have** `school_id`. Roles are global.

5. Context: Academic

Responsibility: Academic structure: campuses, programs, periods, cohorts, courses, actors, and enrollments.

Table 44

5. Context: Academic

Table	Description	PK
`school`	Educational campus	`school_id` (INT)
`program`	Curricular offering of a campus	`program_id` (INT)
`academic_period`	Time window (quarter/semester)	`academic_period_id` (INT)
`cohort`	Group of students in a program + period	`cohort_id` (BIGINT)
`course`	Subject/course of a program	`course_id` (INT)
`academic_actor_type`	Catalog: STUDENT / INSTRUCTOR / ADMIN (campus administrator)	`actor_type_id` (SMALLINT)
`academic_actor`	Role that a person fulfills at a campus	`academic_actor_id` (BIGINT)
`enrollment`	Enrollment of an actor in a cohort	`enrollment_id` (BIGINT)

Internal relationships (real FK): school → program → course school → program → cohort ← academic_period academic_actor → academic_actor_type academic_actor → school enrollment → academic_actor enrollment → cohort

Cross-context (no FK):

- `academic_actor.person_id` → `Identity.person.person_id`

Key differences:

- **`program`**: Highest-level curricular container ("Technology in Data Analysis")
- **`course`**: Unit of content within a program ("Databases I")
- **`cohort`**: Specific group of students who progress together ("Cohort 2691234 — ADSI — Term 1 of 2026")

6. Context: Scheduling

Responsibility: Translates the academic structure into concrete schedules and sessions.

Table 45

6. Context: Scheduling

Table	Description	PK
`environment`	Physical space (classroom, laboratory)	`environment_id` (INT)
`schedule_block`	Recurring weekly block	`schedule_block_id` (BIGINT)
`class_session`	Materialization on a specific date	`class_session_id` (BIGINT)

Cross-context (no FK):

- `environment.school_id` → `Academic.school.school_id`
- `schedule_block.cohort_id` → `Academic.cohort.cohort_id`
- `schedule_block.course_id` → `Academic.course.course_id`

- schedule_block.instructor_actor_id → Academic.academic_actor.academic_actor_id
- class_session.opened_by / class_session.closed_by → Academic.academic_actor.academic_actor_id

Business indexes (unique):

- (environment_id, day_of_week, starts_at) → prevents double-booking the same environment.
- (instructor_actor_id, day_of_week, starts_at) → prevents double-assigning the same instructor.

Bridge to Attendance: class_session is the entity that receives attendance records.

7. Context: Attendance

Responsibility: Attendance recording, justifications, supporting documents, and reports.

Table 46

7. Context: Attendance

Table	Description	PK
attendance_record	One record per actor and session	attendance_record_id (BIGINT)
justification_type	Catalog of justification types	justification_type_id (INT)
justification	Justification for absence/tardiness	justification_id (BIGINT)
supporting_document	Attached supporting documents	supporting_document_id (BIGINT)
attendance_report	Generated attendance report (JSON snapshot)	report_id (UUID)

Internal relationships (real FK):

- justification.attendance_record_id → attendance_record.attendance_record_id
- justification.justification_type_id → justification_type.justification_type_id
- supporting_document.justification_id → justification.justification_id

Cross-context (no FK):

- attendance_record.class_session_id → Scheduling.class_session.class_session_id
- attendance_record.academic_actor_id → Academic.academic_actor.academic_actor_id
- justification.reviewed_by → Identity.app_user.user_id
- attendance_report.school_id / cohort_id → Academic
- attendance_report.generated_by → Identity.app_user.user_id

Business rule (unique index): (class_session_id, academic_actor_id) is **unique** → only **one attendance record can exist per actor per class session**.

ENUMs:

- attendance_status: Present, Absent, Late, Justified
- capture_method: FACIAL, MANUAL, IOT, IMPORT
- review_status: Pending, Approved, Rejected

8. Context: Biometric

Responsibility: Biometric templates (facial and fingerprint).

It is not part of the relational SQL schema. It is modeled as NoSQL collections in **MongoDB** (outside the relational DBML model):

Table 47

8. Context: Biometric

Collection	Key fields
`facial_embedding`	`person_id`, `template_version`, `encoding`, `model_version`, `enrolled_at`, `is_active`
`fingerprint_embedding`	`person_id`, `finger_number`, `template_version`, `encoding`, `model_version`, `enrolled_at`, `is_active`

Why NoSQL? Biometric embeddings are documents with a flexible structure and a high read/write volume due to real-time recognition.

In PostgreSQL only the empty `biometric` schema exists (`06-ms-biometric-db` only creates the schema, with no tables). The logical reference from `configuration.biometric_update_case.current_embedding_ref` points to the `_id` of the document in MongoDB (cross-paradigm, no FK).

9. Context: Configuration

Responsibility: Configurable parameters and biometric update cases.

Table 48

9. Context: Configuration

Table	Description	PK
`academic_configuration`	Parameters per campus	`configuration_id` (INT)
`security_configuration`	Global security parameters	`configuration_id` (INT)
`biometric_update_case`	Biometric update request	`case_id` (UUID)

`biometric_update_case` unifies facial and fingerprint update cases:

- `biometric_type`: FACIAL / FINGERPRINT (ENUM)
- `finger_number`: Only for FINGERPRINT (1..10), null for FACIAL
- `current_embedding_ref`: Logical reference to the active document in NoSQL

9.5 Context: Quality

Responsibility: Quality evaluation instrument (ISO/IEC 25010, ISO/IEC 29110, ISTQB) applied to the other microservices. Logical owner: `09-ms-quality`; temporary physical hosting in the `quality` schema inside `07-ms-configuration-db`, because there is no separate `quality-liquibase` service — documented debt, do not move without a compatibilized migration.

Table 49

9.5 Context: Quality

Table	Description	PK
quality_project	Evaluated software project (ISO/IEC 29110)	project_id (BIGINT)
quality_evaluation	ISO/IEC 25010 evaluation of a microservice	evaluation_id (BIGINT)
quality_evaluation_item	Score 1-5 per 25010 sub-characteristic	evaluation_item_id (BIGINT)
process_assessment	ISO/IEC 29110 process assessment (PM/SI)	assessment_id (BIGINT)
process_assessment_rating	N/P/L/F rating per 29110 objective	process_rating_id (BIGINT)
istqb_assessment	ISTQB CTFL v4.0 evaluation of a microservice	istqb_assessment_id (BIGINT)
istqb_assessment_item	Score 1-5 per ISTQB item	istqb_assessment_item_id (BIGINT)

Internal relationships (real FK):

- quality_evaluation.project_id → quality_project.project_id
- quality_evaluation_item.evaluation_id → quality_evaluation.evaluation_id
- process_assessment.project_id → quality_project.project_id
- process_assessment_rating.assessment_id → process_assessment.assessment_id
- istqb_assessment.project_id → quality_project.project_id
- istqb_assessment_item.istqb_assessment_id → istqb_assessment.istqb_assessment_id

The catalogs of sub-characteristics/objectives/items (e.g. tech-bva, PM.O1) live in code (09-ms-quality/src/domain/{iso25010,iso29110,istqb}.ts), not in tables: the instrument is immutable, so they carry no FK.

10. Context: Notification

Responsibility: Alert types and alerts generated about academic actors.

Table 50

10. Context: Notification

Table	Description	PK
alert_type	Catalog of alert types	alert_type_id (SMALLINT)
alert	Alert generated about an actor	alert_id (BIGINT)

Internal relationship (real FK):

- alert.alert_type_id → alert_type.alert_type_id

Cross-context (no FK):

- alert.academic_actor_id → Academic.academic_actor.academic_actor_id

11. Most important rule: No FKs between contexts

Every reference between different contexts is documented as a comment, never as an active `Ref:`.

This means that in the SQL/Liquibase code:

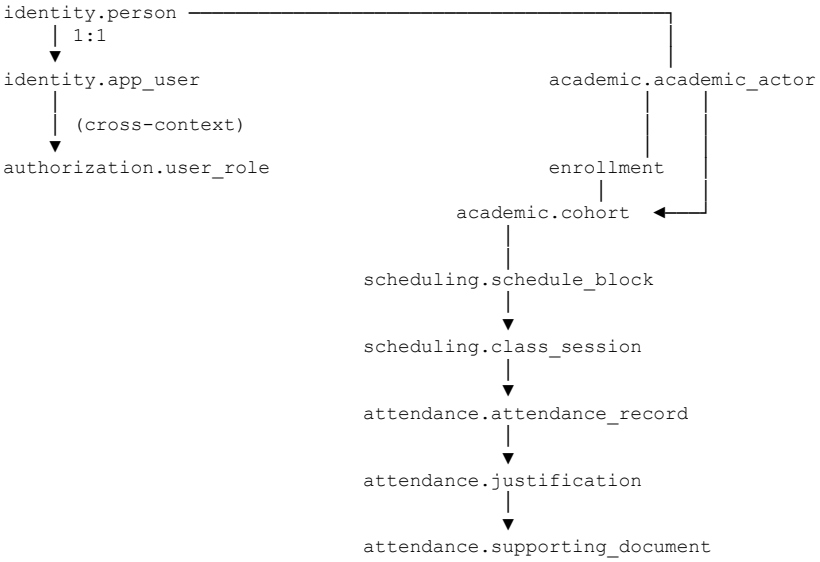
```
# CORRECT - no real FK
- column:
```

```
    name: person_id
    type: UUID
    remarks: "Cross-context reference to Identity.person (NO FK)"

# INCORRECT - real FK between contexts
- addForeignKeyConstraint:
    baseColumnNames: person_id
    referencedTableName: person
    referencedTableSchemaName: identity
Exception: References within the same context DO use real FKs (e.g. enrollment.cohort_id →
cohort.cohort_id, both in Academic).
```

12. Relationship diagram (simplified)

The following diagram summarizes the main relationships between the contexts of the data model. References between contexts are logical and do not represent physical foreign keys.



13. Model ENUMs

ENUMs are implemented as native PostgreSQL types (CREATE TYPE ... AS ENUM), not as VARCHAR columns with textual validation. Although the DBML documents them as varchar with the ENUM note, the physical implementation uses native types.

Below are the 14 actual ENUM types, with their exact values according to the Liquibase changesets.

```
Context ENUM type (PostgreSQL)  Exact values
Identity        user_session_status     Active, Closed
Identity        authentication_type     Local, Windows, External
Academic        enrollment_status       Active, Withdrawn, Completed
Scheduling      class_session_status    Open, Closed, Cancelled
Attendance      attendance_status       Present, Absent, Late, Justified
Attendance      capture_method  FACIAL, MANUAL, IOT, IMPORT
Attendance      review_status   Pending, Approved, Rejected
Configuration   biometric_type  FACIAL, FINGERPRINT
Configuration   update_status   Pending, In_Review, Approved, Rejected
Quality project_status  Planned, Active, Closed
Quality evaluation_status       Draft, Completed
Quality iso29110_process        PM, SI
Quality objective_rating        N, P, L, F
Quality quality_level   Deficiente, En proceso, Aceptable, Bueno, Excelente
```

Note: the values of `quality_level` are stored in Spanish in the database (Poor, In progress, Acceptable, Good, Excellent, respectively).

14. Auditing and cross-cutting best practices

Every table in the model includes a standard block for auditing and concurrency control.

14.1. Audit fields

```
Column  Type    Purpose
created_at      datetime        Record creation date.
updated_at      datetime        Date of the last modification.
deleted_at      datetime (nullable)     Soft delete: NULL indicates the row is active.
created_by      UUID    User who created the record.
updated_by      UUID    User who modified the record.
deleted_by      UUID    User who logically deleted the record.
row_version     BIGINT (default 1)      Optimistic concurrency control (optimistic locking).
```

14.2. Associated rules

Soft delete: the value deleted_at IS NULL identifies active rows. Partial unique indexes: unique indexes sensitive to soft delete use WHERE deleted_at IS NULL clauses; for example, uq_person_email. **ENUM types:** they are created as native PostgreSQL types during the 01-ddl/02-types/ phase of Liquibase. **Referential integrity:** real foreign keys are used only within each context, according to the rule described in section 11.

15. Physical layer (implementation)

15.1. Database topology

PostgreSQL

Database: faceattend_db. Development: one database with eight main schemas (identity, authorization, academic, scheduling, attendance, biometric, configuration, and notification), plus the quality schema, hosted in the same database. Production: serverless PostgreSQL on Neon. For more details, see DESPLIEGUE.md and database/docker-compose.neon.yml.

MongoDB

Stores the biometric embedding collections: facial_embedding fingerprint_embedding

Deployment alternative: the file database/scripts/init-multidb.sql exists, which allows a setup with eight independent databases. However, the development environment uses a single database with multiple schemas.

15.2. Folder → schema → context mapping

The following table relates the Liquibase folders to the corresponding schemas, contexts, and table counts.

```
Liquibase folder        Schema  Context Tables
01-ms-identity-db       identity        Identity        4
02-ms-authorization-db  authorization   Authorization   4
03-ms-academic-db       academic        Academic        8
04-ms-scheduling-db     scheduling      Scheduling      3
05-ms-attendance-db     attendance      Attendance      5
06-ms-biometric-db      biometric       Biometric       0 (schema only; collections in MongoDB)
07-ms-configuration-db  configuration   Configuration   3
08-ms-notification-db   notification    Notification    2
Hosted in 07-ms-configuration-db        quality Quality 7
Total                   36 SQL tables + 2 NoSQL collections
```

15.3. Liquibase migration cycle

Each domain runs its changes in the following order:

01-ddl → 02-dml → 03-dcl → 04-tcl

The phases have the following responsibilities:

```
Phase   Responsibility
01-ddl  Creation of schemas, types, tables, and triggers.
02-dml  Insertion of initial data (seeds).
03-dcl  Configuration of roles and permissions (GRANT).
04-tcl  Functions and triggers implemented in PL/pgSQL.
```

Structure per domain:

```
changelog/changelog-master.yaml: master migrations file.
01-ddl/: data definition changes, organized in subdirectories (01-schemas to 08-triggers).
02-dml/: initial data with context: seed.
03-dcl/: access control and permissions.
04-tcl/: functions and transactional logic.
05-rollbacks/: currently empty; contains databaseChangeLog: [].
```

Liquibase image version: liquibase/liquibase: 4.29.0.

15.4. Documented physical debts and limitations

Quality without its own Liquibase service: the Quality context is temporarily hosted in the quality schema of 07-ms-configuration-db. It must not be moved without a compatibilized migration.

Removal of city: the geographic reference school.city_id points to an external catalog (countriesnow API), with no dedicated table or local foreign key.

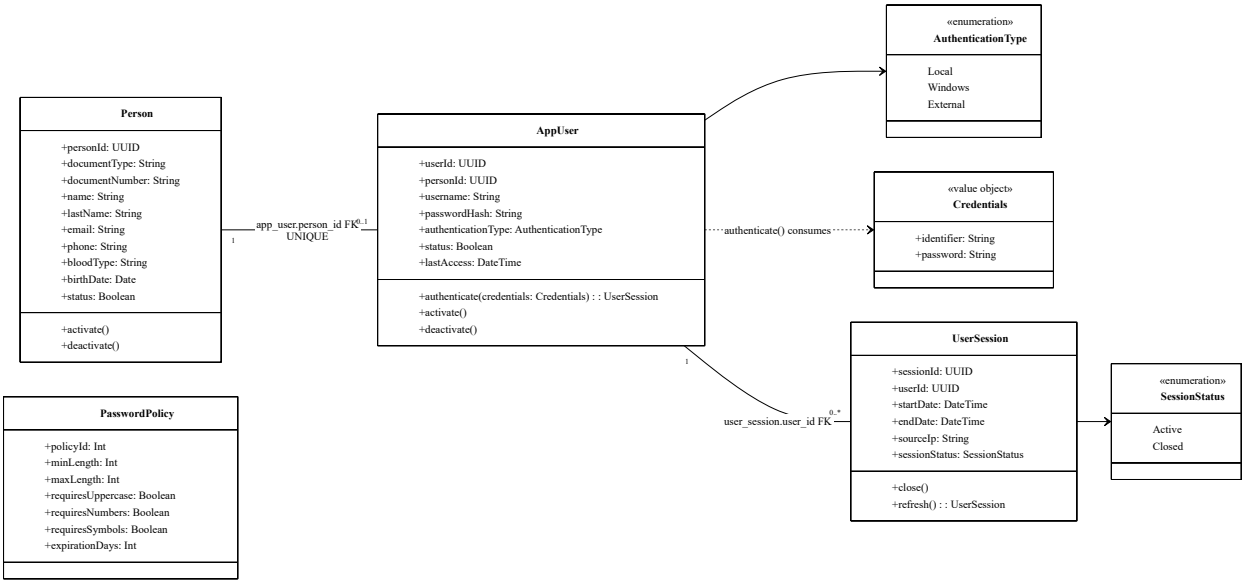
Reference between SQL and MongoDB: biometric_update_case.current_embedding_ref points to the _id of a MongoDB document. Since it is a reference across persistence paradigms, there is no physical referential integrity between the two systems.

Class diagrams

Context: identity

Figure 40

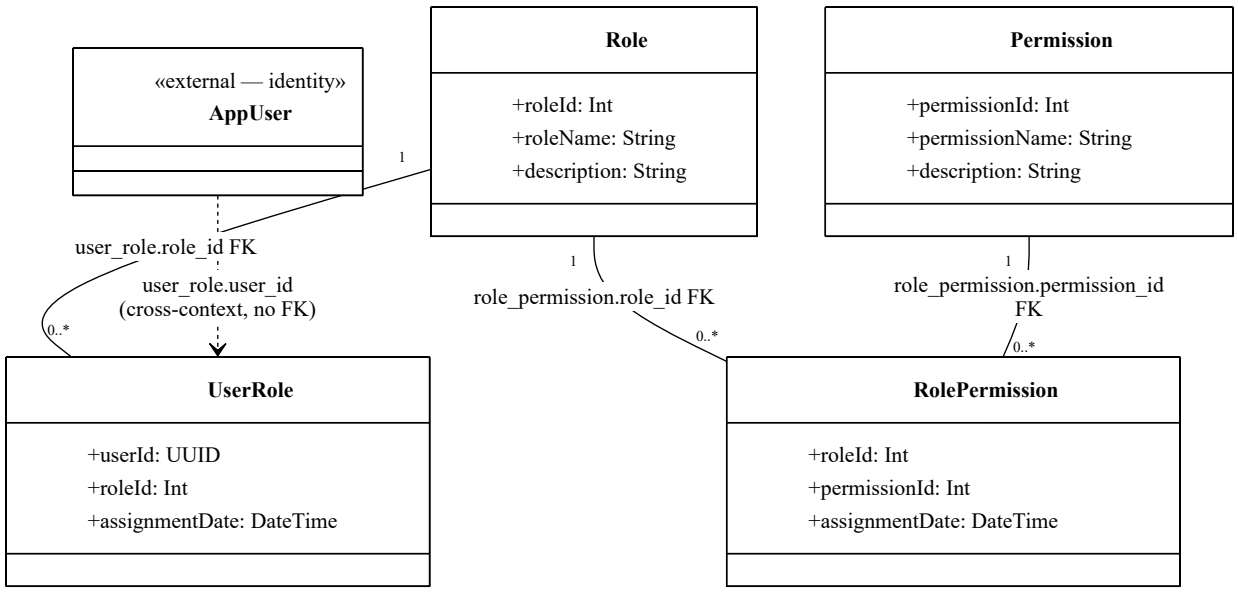
Context: identity



Context: authorization

Figure 41

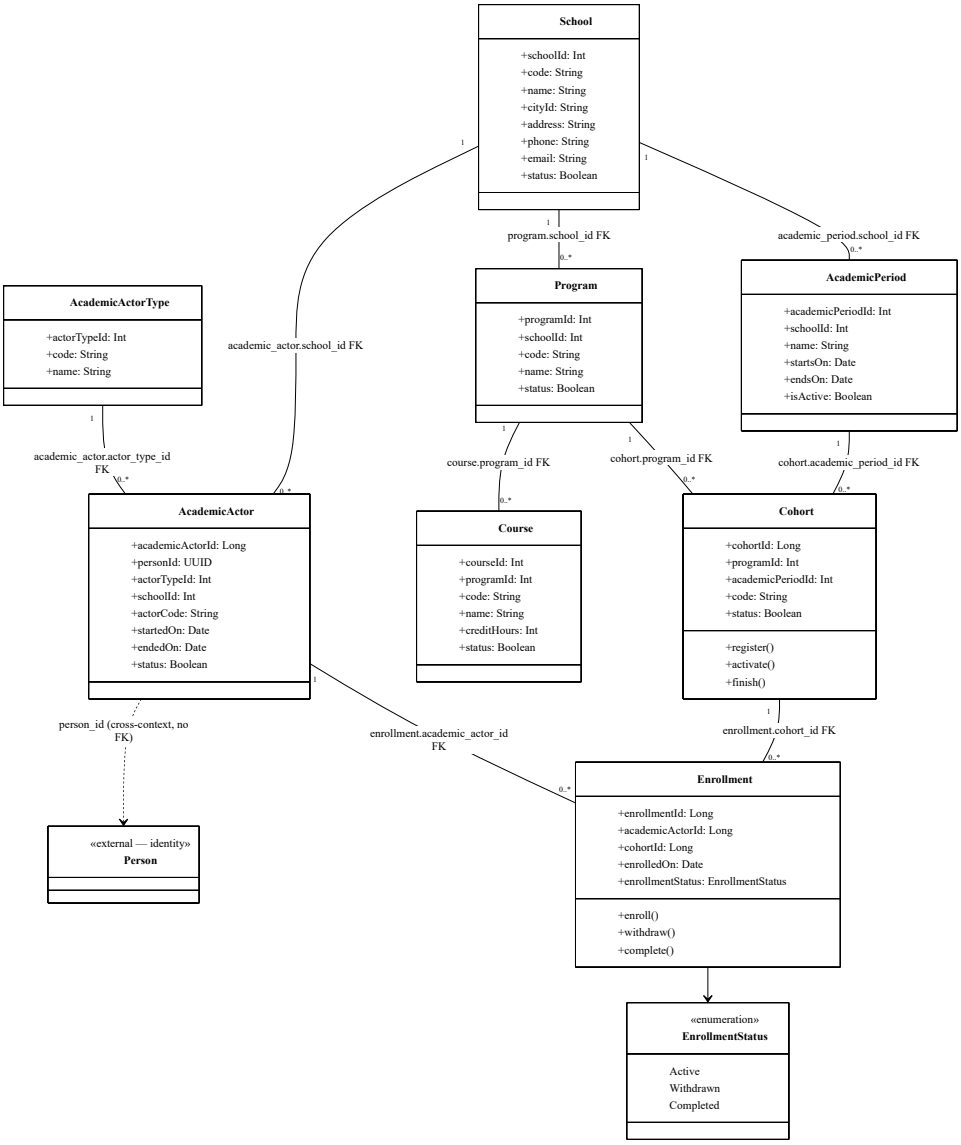
Context: authorization



Context: academic

Figure 42

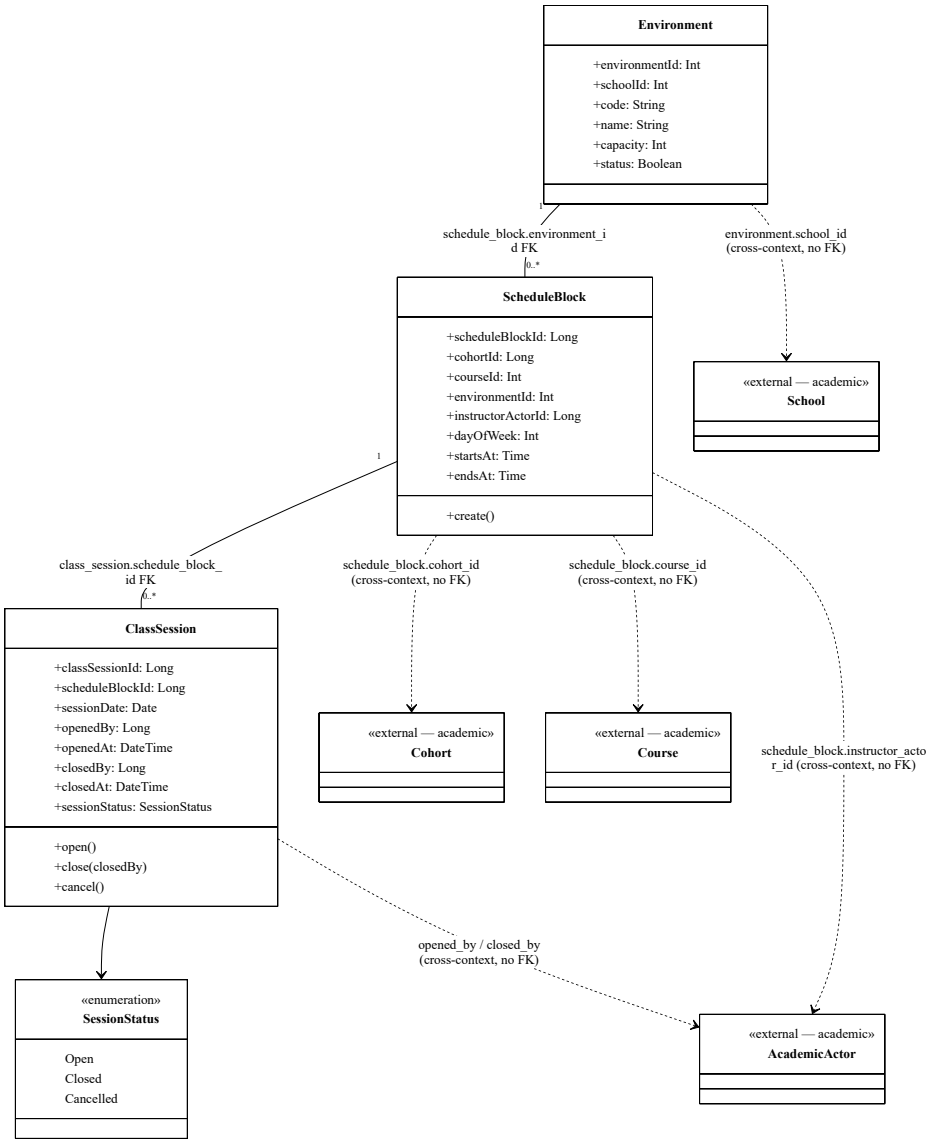
Context: academic



Context: scheduling

Figure 43

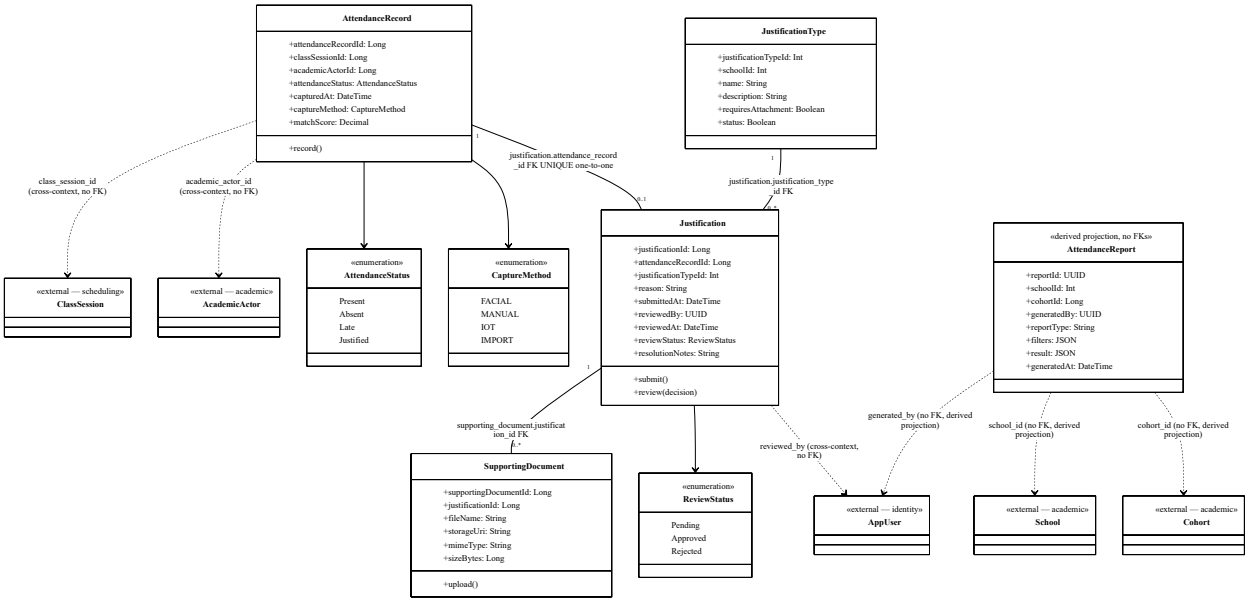
Context: scheduling



Context: attendance

Figure 44

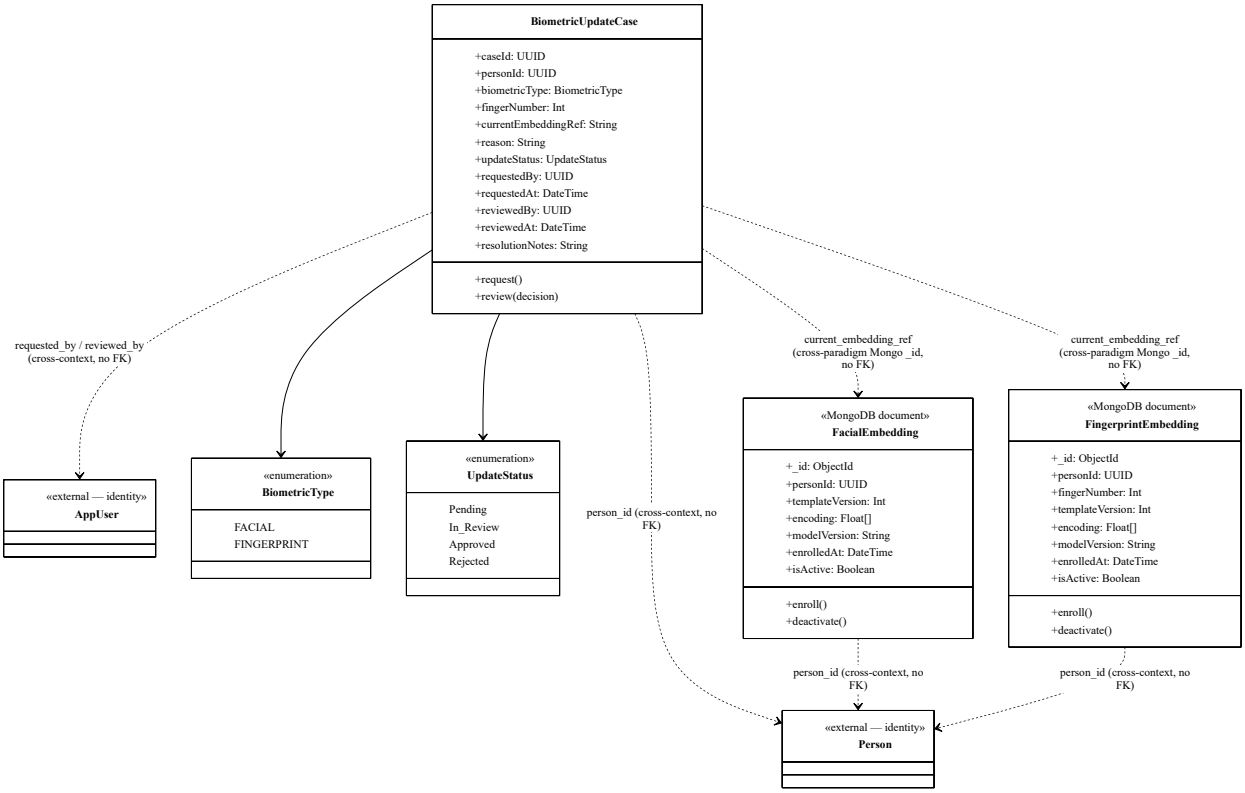
Context: attendance



Context: biometric

Figure 45

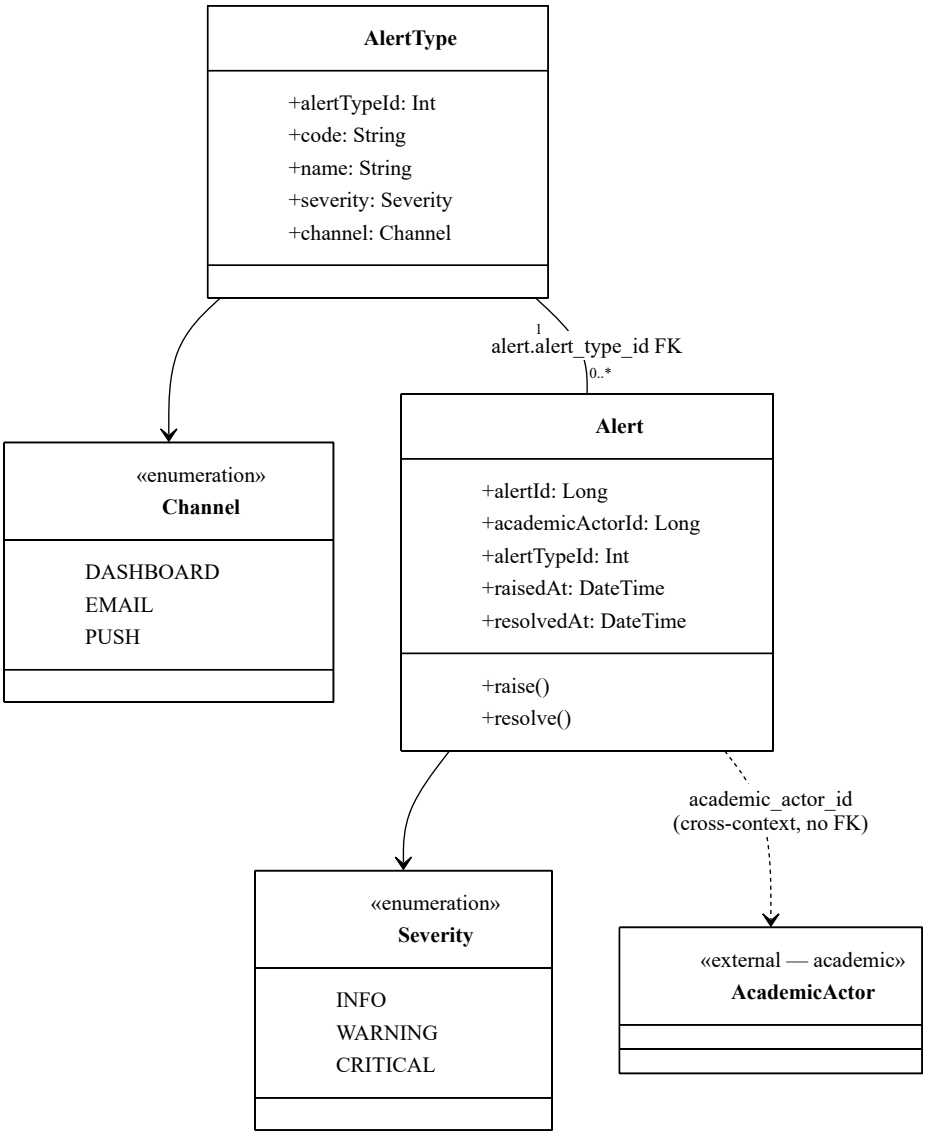
Context: biometric



Context: notification

Figure 46

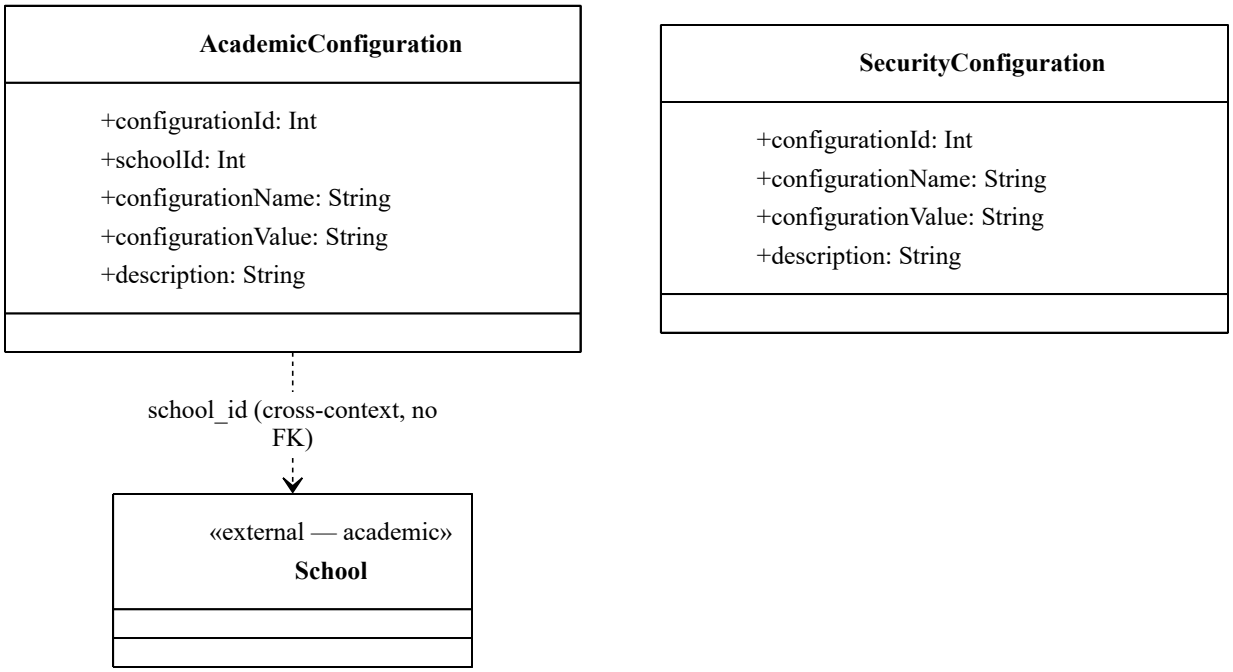
Context: notification



Context: configuration

Figure 47

Context: configuration



Data dictionary: identity

Owner: Identity bounded context **DB Engine:** PostgreSQL 17 — ACID transactions, relational integrity, multi-tenant isolation

Engine justification:

- Strong relational guarantees for person-user relationships.
- UUID primary keys for distributed ID generation across microservices.
- TIMESTAMP (no time zone) for audit fields, filled by `fn_audit_timestamps()`.
- Optimistic locking via `row_version` for concurrent update safety.

1. Table: city — DEPRECATED (legacy, still deployed)

Deprecated: `identity.city` was removed from the relational model — cities come from the external API `countriesnow.space` (see `academic.school.city_id`). The table is still created by `01-ms-identity-db/01-ddl/03-tables/001-create-city-table.yaml` and Kong still routes `GET/POST/PUT/DELETE /api/v1/cities` to `ms-identity` (marked `deprecated: true` in `07-api/contracts/openapi/01-identity-service.yaml`). Treat it as read-only legacy pending removal; do not build new features on it. `academic.school.city_id` is `VARCHAR NULL` with no FK (see `03-academic.md`).

2. Table: person

Purpose: Generic record of any person in the system (student, instructor, admin, etc.).

```
CREATE TABLE person (
  -- Business fields
  person_id         UUID        PRIMARY KEY DEFAULT gen_random_uuid(),
  document_number   VARCHAR     NOT NULL,
  name              VARCHAR     NOT NULL,
  last_name         VARCHAR     NOT NULL,
  email             VARCHAR,
  phone             VARCHAR,
  document_type     VARCHAR     NULL CHECK (document_type IN ('CC','TI','CE','PP')),
  blood_type        VARCHAR     NULL CHECK (blood_type IN ('A+','A-','B+','B-','AB+','AB-','O+','O-')),
  birth_date        DATE        NULL,
  address           VARCHAR     NULL,

  -- Status fields
  status            BOOLEAN     NOT NULL DEFAULT true,

  -- Audit (in all tables)
  created_at        TIMESTAMP NOT NULL DEFAULT NOW(),
  updated_at        TIMESTAMP NOT NULL DEFAULT NOW(),
  deleted_at        TIMESTAMP,
  created_by        UUID,
  updated_by        UUID,
  deleted_by        UUID,
  row_version       BIGINT      NOT NULL DEFAULT 1
);

-- Real uniqueness is the pair (document_type, document_number):
-- CC and TI may share numbering across different persons.
CREATE UNIQUE INDEX uq_person_document ON person (document_type, document_number);
-- Partial UNIQUE (deployed as id-008-person-email-unique-index): login resolves an email to
-- one app_user; NULLs allowed and a soft-deleted person frees its email for reuse.
CREATE UNIQUE INDEX uq_person_email ON person (email) WHERE deleted_at IS NULL;
CREATE INDEX idx_person_deleted ON person (deleted_at) WHERE deleted_at IS NULL;
```

Data dictionary:

Table 51

2. Table: person

Column	Type	Description	Example
person_id	UUID	Auto-generated unique identifier	`550e8400-e29b-41d4-a716-446655440001`
document_number	VARCHAR	National ID / document number (unique with document_type)	`1234567890`
name	VARCHAR	First name(s)	`Juan`
last_name	VARCHAR	Last name(s)	`Perez`
email	VARCHAR	Email address	`juan@example.com`
phone	VARCHAR	Phone number	`+57 300 987 6543`
document_type	VARCHAR	Document type CHECK: CC/TI/CE/PP	`CC`
blood_type	VARCHAR	Blood type CHECK: A+/A-/B+/B-/AB+/AB-/O+/O-	`O+`
birth_date	DATE	Birth date	`2000-05-10`
address	VARCHAR	Home address	`Calle 123 #45-67`
status	BOOLEAN	Active/inactive flag	`true`
created_at	TIMESTAMP	When the record was created	`2024-01-15T10:30:00Z`
updated_at	TIMESTAMP	When the record was last updated	`2024-01-15T10:30:00Z`
deleted_at	TIMESTAMP	NULL = active; with value = soft-deleted	`NULL`
created_by	UUID	User who created this record	`550e8400-...`
updated_by	UUID	User who last updated this record	`550e8400-...`
deleted_by	UUID	User who soft-deleted this record	`NULL`
row_version	BIGINT	Optimistic lock version counter	`1`

Modeling decisions:

1. Real uniqueness is `(document_type, document_number)` — CC and TI may share numbering across different persons.
2. No `school_id` — identity is school-agnostic; a person can exist in multiple schools via `academic_actor`.
3. Soft delete via `deleted_at` preserves history; `deleted_by` tracks who removed it.
4. `document_type` / `blood_type` are VARCHAR + CHECK (not native ENUM). `authentication_type` and `session_status` are native PostgreSQL ENUMs (DBML documents them as `varchar + note`).

3. Table: app_user

Purpose: Login credentials and authentication record. Linked 1:1 with Person.

```
-- Native ENUM: CREATE TYPE identity.authentication_type AS ENUM ('Local','Windows','External');
CREATE TABLE app_user (
  -- Business fields
  user_id             UUID        PRIMARY KEY DEFAULT gen_random_uuid(),
  person_id           UUID        NOT NULL UNIQUE,
  username            VARCHAR     NOT NULL,
  password_hash       VARCHAR     NOT NULL,
  authentication_type identity.authentication_type NOT NULL,
  status              BOOLEAN     NOT NULL DEFAULT true,
  last_access         TIMESTAMP,

  -- Audit (in all tables)
  created_at          TIMESTAMP NOT NULL DEFAULT NOW(),
  updated_at          TIMESTAMP NOT NULL DEFAULT NOW(),
```

```
  deleted_at          TIMESTAMP,
  created_by          UUID,
  updated_by          UUID,
  deleted_by          UUID,
  row_version         BIGINT      NOT NULL DEFAULT 1
);

CREATE UNIQUE INDEX uq_user_username ON app_user (username);
```

Data dictionary:

Table 52

3. Table: app_user

Column	Type	Description	Example
user_id	UUID	Auto-generated unique identifier	`550e8400-e29b-41d4-a716-446655440002`
person_id	UUID	Linked person (1:1, FK → person)	`550e8400-...`
username	VARCHAR	Login username, globally unique	`jperez`
password_hash	VARCHAR	Hashed password (never plaintext)	`$2b$10$...`
authentication_type	VARCHAR	Auth method: Local, Windows, External	`Local`
status	BOOLEAN	Active/inactive flag	`true`
last_access	TIMESTAMP	Last login timestamp	`2024-06-01T08:15:00Z`
created_at	TIMESTAMP	When the record was created	`2024-01-15T10:30:00Z`
updated_at	TIMESTAMP	When the record was last updated	`2024-01-15T10:30:00Z`
deleted_at	TIMESTAMP	NULL = active; with value = soft-deleted	`NULL`
created_by	UUID	User who created this record	`550e8400-...`
updated_by	UUID	User who last updated this record	`550e8400-...`
deleted_by	UUID	User who soft-deleted this record	`NULL`
row_version	BIGINT	Optimistic lock version counter	`1`

Modeling decisions:

1. 1:1 relationship with Person — each person has at most one user account.
2. `authentication_type` supports: Local (password), Windows (AD/LDAP), External (OAuth).
3. `last_access` enables inactive user detection and activity auditing.

4. Table: user_session

Purpose: Record of each user login session.

```
-- Native ENUM: CREATE TYPE identity.user_session_status AS ENUM ('Active','Closed');
CREATE TABLE user_session (
  -- Business fields
  session_id      UUID        PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id         UUID        NOT NULL,
  start_date      TIMESTAMP NOT NULL,
  end_date        TIMESTAMP,
  source_ip       VARCHAR,

  -- Status fields
  session_status  identity.user_session_status NOT NULL,

  -- Audit (in all tables)
  created_at      TIMESTAMP NOT NULL DEFAULT NOW(),
  updated_at      TIMESTAMP NOT NULL DEFAULT NOW(),
  deleted_at      TIMESTAMP,
  created_by      UUID,
```

```
  updated_by      UUID,
  deleted_by      UUID,
  row_version     BIGINT      NOT NULL DEFAULT 1
);

CREATE INDEX idx_session_user ON user_session (user_id);
CREATE INDEX idx_session_status ON user_session (session_status);
```

Data dictionary:

Table 53

4. Table: user_session

Column	Type	Description	Example
session_id	UUID	Auto-generated unique session identifier	`550e8400-...`
user_id	UUID	Session owner (FK → app_user)	`550e8400-...`
start_date	TIMESTAMP	Session start time	`2024-06-01T08:15:00Z`
end_date	TIMESTAMP	Session end time (NULL if active)	`NULL`
source_ip	VARCHAR	Client IP address	`192.168.1.100`
session_status	VARCHAR	State: Active or Closed	`Active`
created_at	TIMESTAMP	When the record was created	`2024-06-01T08:15:00Z`
updated_at	TIMESTAMP	When the record was last updated	`2024-06-01T08:15:00Z`
deleted_at	TIMESTAMP	NULL = active; with value = soft-deleted	`NULL`
created_by	UUID	User who created this record	`550e8400-...`
updated_by	UUID	User who last updated this record	`550e8400-...`
deleted_by	UUID	User who soft-deleted this record	`NULL`
row_version	BIGINT	Optimistic lock version counter	`1`

5. Table: password_policy

Purpose: System password policy (complexity requirements, expiration).

```
CREATE TABLE password_policy (
  -- Business fields
  policy_id           SERIAL      PRIMARY KEY,
  min_length          INT         NOT NULL DEFAULT 8,
  max_length          INT         NOT NULL DEFAULT 20,
  requires_uppercase  BOOLEAN     NOT NULL DEFAULT true,
  requires_numbers    BOOLEAN     NOT NULL DEFAULT true,
  requires_symbols    BOOLEAN     NOT NULL DEFAULT true,
  expiration_days     INT         NOT NULL DEFAULT 90,

  -- Audit (in all tables)
  created_at          TIMESTAMP NOT NULL DEFAULT NOW(),
  updated_at          TIMESTAMP NOT NULL DEFAULT NOW(),
  deleted_at          TIMESTAMP,
  created_by          UUID,
  updated_by          UUID,
  deleted_by          UUID,
  row_version         BIGINT      NOT NULL DEFAULT 1
);
```

Data dictionary:

Table 54

5. Table: password_policy

Column	Type	Description	Example
policy_id	SERIAL	Auto-incremental policy identifier	`1`
min_length	INT	Minimum password length	`8`
max_length	INT	Maximum password length	`20`
requires_uppercase	BOOLEAN	Requires at least one uppercase letter	`true`
requires_numbers	BOOLEAN	Requires at least one number	`true`
requires_symbols	BOOLEAN	Requires at least one symbol	`true`
expiration_days	INT	Days before password expires	`90`
created_at	TIMESTAMP	When the record was created	`2024-01-15T10:30:00Z`
updated_at	TIMESTAMP	When the record was last updated	`2024-01-15T10:30:00Z`
deleted_at	TIMESTAMP	NULL = active; with value = soft-deleted	`NULL`
created_by	UUID	User who created this record	`550e8400-...`
updated_by	UUID	User who last updated this record	`550e8400-...`
deleted_by	UUID	User who soft-deleted this record	`NULL`
row_version	BIGINT	Optimistic lock version counter	`1`

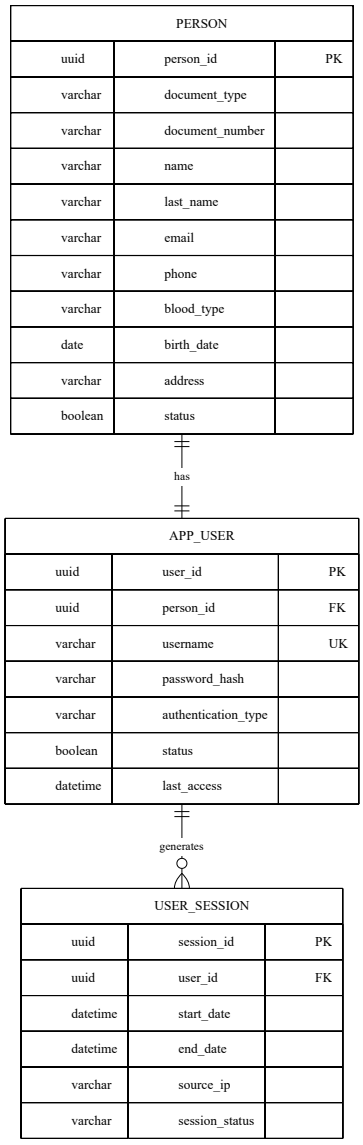
Modeling decisions:

1. Singleton table (one policy for the entire system) — not multi-tenant.
2. Sensible defaults adjustable by an administrator.

6. Relationship Diagram

Figure 48

6. Relationship Diagram



`CITY` is legacy in the deployed DB (deprecated in the DBML — cities come from the external `countriesnow.space` API). Uniqueness on person is `uq_person_document (document_type, document_number)` plus the partial unique `uq_person_email (email) WHERE deleted_at IS NULL`.

7. Cross-Context Relationships

Table 55

7. Cross-Context Relationships

From	To	Reference Type	Mechanism
Authorization.User_Role.user_id	App_User.user_id	External ID (no FK)	Cross-context reference by UUID

From	To	Reference Type	Mechanism
Academic.Academic_Actor.person_id	Person.person_id	External ID (no FK)	Cross-context reference by UUID
Attendance.Justification.reviewed_by	App_User.user_id	External ID (no FK)	Cross-context reference by UUID
Configuration.Biometric_Update_Case.requested_by	App_User.user_id	External ID (no FK)	Cross-context reference by UUID

8. Correlations

- Domain entities that map to these tables → `02-domain/entities-and-rules.md`
- Saga and Outbox pattern for distributed consistency → `05-architecture/pattern-guide.md`
- Data for each service in detail → `09-microservices/services/identity-service/data-model.md`
- How data is accessed via API → `07-api/contracts/openapi/`

Data dictionary: authorization

Owner: Authorization bounded context **DB Engine:** PostgreSQL 17 — ACID transactions, relational integrity for role-permission associations

Engine justification:

- M:N relationships between roles, permissions, and users require relational integrity.
- SERIAL primary keys for internal identifiers not exposed across services.

Design Principles

- Authorization does **NOT** depend directly on Person.
- `user_id` acts as an external reference to the Identity context.
- Roles are **global** (no `school_id`) — the same role structure applies to all schools.

1. Table: role

Purpose: Definition of available roles in the system (e.g., Admin, Instructor, Student).

```
CREATE TABLE role (
  -- Business fields
  role_id       SERIAL      PRIMARY KEY,
  role_name     VARCHAR     NOT NULL UNIQUE,
  description   VARCHAR,

  -- Audit (in all tables)
  created_at    TIMESTAMP NOT NULL DEFAULT NOW(),
  updated_at    TIMESTAMP NOT NULL DEFAULT NOW(),
  deleted_at    TIMESTAMP,
  created_by    UUID,
  updated_by    UUID,
  deleted_by    UUID,
  row_version   BIGINT      NOT NULL DEFAULT 1
);
```

Data dictionary:

Table 56

1. Table: role

Column	Type	Description	Example
role_id	SERIAL	Auto-incremental role identifier	`1`
role_name	VARCHAR	Unique role name	`ADMIN`
description	VARCHAR	Role description	`Full system administrator`
created_at	TIMESTAMP	When the record was created	`2024-01-15T10:30:00Z`
updated_at	TIMESTAMP	When the record was last updated	`2024-01-15T10:30:00Z`
deleted_at	TIMESTAMP	NULL = active; with value = soft-deleted	`NULL`
created_by	UUID	User who created this record	`550e8400-...`
updated_by	UUID	User who last updated this record	`550e8400-...`
deleted_by	UUID	User who soft-deleted this record	`NULL`
row_version	BIGINT	Optimistic lock version counter	`1`

Modeling decisions:

1. `role_name` is globally unique — roles are not per-school in v4.
2. `role_id` is SERIAL because it is a purely internal identifier.

2. Table: permission

Purpose: Global catalog of permissions available in the system.

```
CREATE TABLE permission (
  -- Business fields
  permission_id   SERIAL      PRIMARY KEY,
  permission_name VARCHAR     NOT NULL UNIQUE,
  description     VARCHAR,

  -- Audit (in all tables)
  created_at      TIMESTAMP NOT NULL DEFAULT NOW(),
  updated_at      TIMESTAMP NOT NULL DEFAULT NOW(),
  deleted_at      TIMESTAMP,
  created_by      UUID,
  updated_by      UUID,
  deleted_by      UUID,
  row_version     BIGINT      NOT NULL DEFAULT 1
);
```

Data dictionary:

Table 57

2. Table: permission

Column	Type	Description	Example
permission_id	SERIAL	Auto-incremental permission identifier	`1`
permission_name	VARCHAR	Unique permission name (namespace convention)	`attendance:read`
description	VARCHAR	Permission description	`View attendance records`
created_at	TIMESTAMP	When the record was created	`2024-01-15T10:30:00Z`
updated_at	TIMESTAMP	When the record was last updated	`2024-01-15T10:30:00Z`
deleted_at	TIMESTAMP	NULL = active; with value = soft-deleted	`NULL`
created_by	UUID	User who created this record	`550e8400-...`
updated_by	UUID	User who last updated this record	`550e8400-...`
deleted_by	UUID	User who soft-deleted this record	`NULL`
row_version	BIGINT	Optimistic lock version counter	`1`

3. Table: role_permission

Purpose: Association between roles and permissions.

```
CREATE TABLE role_permission (
  -- Business fields
  role_id         INT         NOT NULL,
  permission_id   INT         NOT NULL,
  assignment_date TIMESTAMP NOT NULL DEFAULT NOW(),

  -- Audit (in all tables)
  created_at      TIMESTAMP NOT NULL DEFAULT NOW(),
  updated_at      TIMESTAMP NOT NULL DEFAULT NOW(),
  deleted_at      TIMESTAMP,
  created_by      UUID,
  updated_by      UUID,
  deleted_by      UUID,
  row_version     BIGINT      NOT NULL DEFAULT 1,

  PRIMARY KEY (role_id, permission_id)
);
```

```
ALTER TABLE role_permission ADD CONSTRAINT fk_rp_role
  FOREIGN KEY (role_id) REFERENCES role(role_id);
ALTER TABLE role_permission ADD CONSTRAINT fk_rp_permission
  FOREIGN KEY (permission_id) REFERENCES permission(permission_id);
```

Data dictionary:

Table 58

3. Table: role_permission

Column	Type	Description	Example
role_id	INT	Role receiving the permission (FK → role)	1
permission_id	INT	Permission being assigned (FK → permission)	3
assignment_date	TIMESTAMP	When the permission was assigned	2024-01-15T10:30:00Z
created_at	TIMESTAMP	When the record was created	2024-01-15T10:30:00Z
updated_at	TIMESTAMP	When the record was last updated	2024-01-15T10:30:00Z
deleted_at	TIMESTAMP	NULL = active; with value = soft-deleted	NULL
created_by	UUID	User who created this record	550e8400-...
updated_by	UUID	User who last updated this record	550e8400-...
deleted_by	UUID	User who soft-deleted this record	NULL
row_version	BIGINT	Optimistic lock version counter	1

4. Table: user_role

Purpose: Assignment of roles to specific users.

```
CREATE TABLE user_role (
  -- Business fields
  user_id         UUID        NOT NULL,
  role_id         INT         NOT NULL,
  assignment_date TIMESTAMP NOT NULL DEFAULT NOW(),

  -- Audit (in all tables)
  created_at      TIMESTAMP NOT NULL DEFAULT NOW(),
  updated_at      TIMESTAMP NOT NULL DEFAULT NOW(),
  deleted_at      TIMESTAMP,
  created_by      UUID,
  updated_by      UUID,
  deleted_by      UUID,
  row_version     BIGINT      NOT NULL DEFAULT 1,

  PRIMARY KEY (user_id, role_id)
);

ALTER TABLE user_role ADD CONSTRAINT fk_ur_role
  FOREIGN KEY (role_id) REFERENCES role(role_id);
```

Data dictionary:

Table 59

4. Table: user_role

Column	Type	Description	Example
user_id	UUID	User receiving the role (external ID → Identity.app_user)	550e8400-...
role_id	INT	Role being assigned (FK → role)	1
assignment_date	TIMESTAMP	When the role was assigned	2024-01-15T10:30:00Z

Column	Type	Description	Example
created_at	TIMESTAMP	When the record was created	`2024-01-15T10:30:00Z`
updated_at	TIMESTAMP	When the record was last updated	`2024-01-15T10:30:00Z`
deleted_at	TIMESTAMP	NULL = active; with value = soft-deleted	`NULL`
created_by	UUID	User who created this record	`550e8400-...`
updated_by	UUID	User who last updated this record	`550e8400-...`
deleted_by	UUID	User who soft-deleted this record	`NULL`
row_version	BIGINT	Optimistic lock version counter	`1`

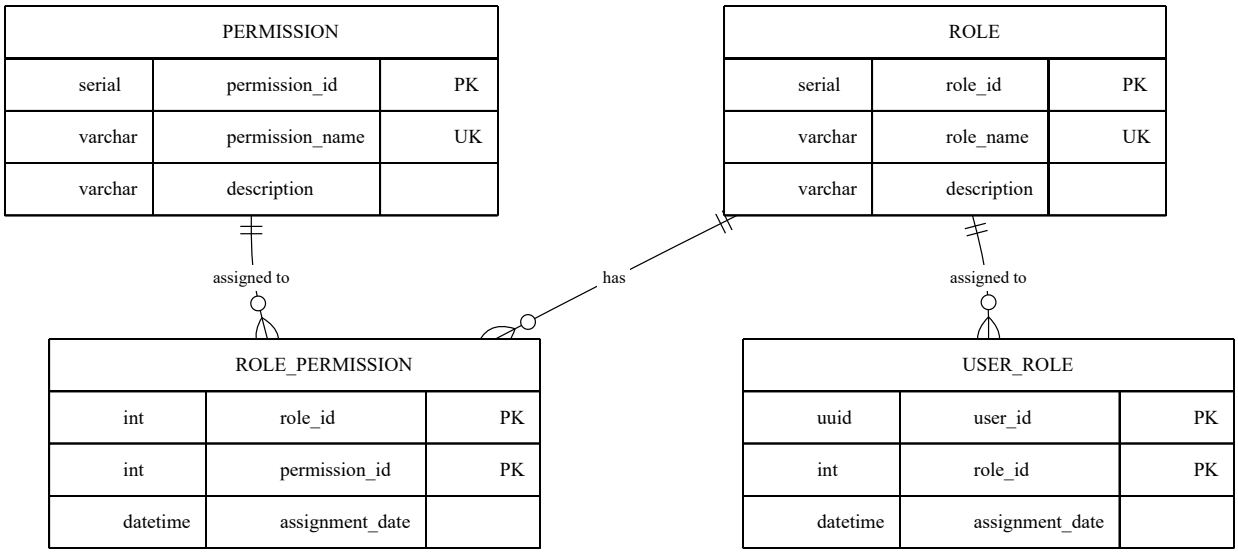
Modeling decisions:

1. `user_id` is an external reference (UUID from Identity.app_user) — **no physical FK** to respect bounded context isolation.
2. A user can have multiple roles (M:N).

5. Relationship Diagram

Figure 49

5. Relationship Diagram



6. Cross-Context Relationships

Table 60

6. Cross-Context Relationships

From	To	Reference Type	Mechanism
User_Role.user_id	Identity.App_User.user_id	External ID (no FK)	Cross-context reference by UUID

7. Correlations

- Domain entities that map to these tables → `02-domain/entities-and-rules.md`

- Saga and Outbox pattern for distributed consistency → `05-architecture/pattern-guide.md`
- Data for each service in detail → `09-microservices/services/authorization-service/data-model.md`
- How data is accessed via API → `07-api/contracts/openapi/`

Data dictionary: academic

Owner: Academic bounded context **DB Engine:** PostgreSQL 17 — ACID transactions, complex relationships, enrollment management

Engine justification:

- Complex relational model with multiple FKs for school-program-cohort-course hierarchy.
- Unified `academic_actor` replaces separate Student/Teacher tables.

Design Principles

- Academic owns: school, program, academic_period, cohort, course, academic_actor_type, academic_actor, enrollment.
- `person_id` in academic_actor is an external reference to Identity — no cross-context FK.
- `academic_actor` unifies students and instructors under a single table with `actor_type`.

1. Table: school

Purpose: Institutional record of each school in the multi-tenant platform.

```
CREATE TABLE school (
  -- Business fields
  school_id     SERIAL      PRIMARY KEY,
  code          VARCHAR     NOT NULL UNIQUE,
  name          VARCHAR     NOT NULL,
  -- External catalog via countriesnow API, no FK (DBML v6/v7: city table removed)
  city_id       VARCHAR     NULL,
  address       VARCHAR,
  district      VARCHAR     NULL,
  country       VARCHAR     NULL,
  city_name     VARCHAR     NULL,
  phone         VARCHAR,
  email         VARCHAR,

  -- Status fields
  status        BOOLEAN     NOT NULL DEFAULT true,

  -- Audit (in all tables)
  created_at    TIMESTAMP NOT NULL DEFAULT NOW(),
  updated_at    TIMESTAMP NOT NULL DEFAULT NOW(),
  deleted_at    TIMESTAMP,
  created_by    UUID,
  updated_by    UUID,
  deleted_by    UUID,
  row_version   BIGINT      NOT NULL DEFAULT 1
);
```

Data dictionary:

Table 61

1. Table: school

Column	Type	Description	Example
school_id	SERIAL	Auto-incremental school identifier	`1`
code	VARCHAR	Unique school code	`ESC-001`
name	VARCHAR	Institution name	`Escuela Nacional`
city_id	VARCHAR	External city id (countriesnow API, no FK, NULL allowed)	`CO-BOG`
address	VARCHAR	Physical address	`Calle 123 #45-67`

Column	Type	Description	Example
district	VARCHAR	District/state	`Cundinamarca`
country	VARCHAR	Country	`Colombia`
city_name	VARCHAR	Denormalized city label from external API	`Bogota`
phone	VARCHAR	Contact phone number	`+57 300 123 4567`
email	VARCHAR	Institutional email	`info@escuela.edu.co`
status	BOOLEAN	Active/inactive flag	`true`
created_at	TIMESTAMP	When the record was created	`2024-01-15T10:30:00Z`
updated_at	TIMESTAMP	When the record was last updated	`2024-01-15T10:30:00Z`
deleted_at	TIMESTAMP	NULL = active; with value = soft-deleted	`NULL`
created_by	UUID	User who created this record	`550e8400-...`
updated_by	UUID	User who last updated this record	`550e8400-...`
deleted_by	UUID	User who soft-deleted this record	`NULL`
row_version	BIGINT	Optimistic lock version counter	`1`

Modeling decisions:

1. `school_id` is SERIAL (not UUID) — internal identifier, not exposed cross-context.
2. `code` is the business identifier used in APIs and reports.
3. `city_id` is an external catalog id (VARCHAR NULL, no FK) — `identity.city` is deprecated; cities come from `countriesnow.space`. `district/country/city_name` are display fields from that API.

2. Table: program

Purpose: Academic programs offered by a school (e.g., Engineering, Medicine).

```
CREATE TABLE program (
  -- Business fields
  program_id    SERIAL      PRIMARY KEY,
  school_id     INT         NOT NULL,
  code          VARCHAR     NOT NULL,
  name          VARCHAR     NOT NULL,

  -- Status fields
  status        BOOLEAN     NOT NULL DEFAULT true,

  -- Audit (in all tables)
  created_at    TIMESTAMP NOT NULL DEFAULT NOW(),
  updated_at    TIMESTAMP NOT NULL DEFAULT NOW(),
  deleted_at    TIMESTAMP,
  created_by    UUID,
  updated_by    UUID,
  deleted_by    UUID,
  row_version   BIGINT      NOT NULL DEFAULT 1
);

CREATE UNIQUE INDEX uq_program_code_school ON program (school_id, code);
```

Data dictionary:

Table 62

2. Table: program

Column	Type	Description	Example
program_id	SERIAL	Auto-incremental program identifier	`1`

Column	Type	Description	Example
school_id	INT	Owning school (FK → school)	1
code	VARCHAR	Program code within the school	ING-SIS
name	VARCHAR	Program name	Systems Engineering
status	BOOLEAN	Active/inactive flag	true
created_at	TIMESTAMP	When the record was created	2024-01-15T10:30:00Z
updated_at	TIMESTAMP	When the record was last updated	2024-01-15T10:30:00Z
deleted_at	TIMESTAMP	NULL = active; with value = soft-deleted	NULL
created_by	UUID	User who created this record	550e8400-...
updated_by	UUID	User who last updated this record	550e8400-...
deleted_by	UUID	User who soft-deleted this record	NULL
row_version	BIGINT	Optimistic lock version counter	1

3. Table: academic_period

Purpose: Academic periods (semesters, trimesters).

```
CREATE TABLE academic_period (
  -- Business fields
  academic_period_id  SERIAL      PRIMARY KEY,
  school_id           INT         NOT NULL,
  name                VARCHAR     NOT NULL,
  starts_on           DATE        NOT NULL,
  ends_on             DATE        NOT NULL CHECK (ends_on > starts_on),

  -- Status fields
  is_active           BOOLEAN     NOT NULL DEFAULT false,

  -- Audit (in all tables)
  created_at          TIMESTAMP NOT NULL DEFAULT NOW(),
  updated_at          TIMESTAMP NOT NULL DEFAULT NOW(),
  deleted_at          TIMESTAMP,
  created_by          UUID,
  updated_by          UUID,
  deleted_by          UUID,
  row_version         BIGINT      NOT NULL DEFAULT 1,

  CONSTRAINT chk_period_dates CHECK (starts_on < ends_on)
);

CREATE UNIQUE INDEX uq_period_name_school ON academic_period (school_id, name);
```

Data dictionary:

Table 63

3. Table: academic_period

Column	Type	Description	Example
academic_period_id	SERIAL	Auto-incremental period identifier	1
school_id	INT	Owning school (FK → school)	1
name	VARCHAR	Period name	2026-S1
starts_on	DATE	Period start date	2026-01-15
ends_on	DATE	Period end date	2026-06-30
is_active	BOOLEAN	Whether this is the current active period	true
created_at	TIMESTAMP	When the record was created	2024-01-15T10:30:00Z
updated_at	TIMESTAMP	When the record was last updated	2024-01-15T10:30:00Z

Column	Type	Description	Example
deleted_at	TIMESTAMP	NULL = active; with value = soft-deleted	NULL
created_by	UUID	User who created this record	550e8400-...
updated_by	UUID	User who last updated this record	550e8400-...
deleted_by	UUID	User who soft-deleted this record	NULL
row_version	BIGINT	Optimistic lock version counter	1

4. Table: cohort

Purpose: A group of students enrolled in a program for a specific period.

```
CREATE TABLE cohort (
  -- Business fields
  cohort_id           BIGSERIAL   PRIMARY KEY,
  program_id          INT         NOT NULL,
  academic_period_id  INT         NOT NULL,
  code                VARCHAR     NOT NULL UNIQUE,

  -- Status fields
  status              BOOLEAN     NOT NULL DEFAULT true,

  -- Audit (in all tables)
  created_at          TIMESTAMP NOT NULL DEFAULT NOW(),
  updated_at          TIMESTAMP NOT NULL DEFAULT NOW(),
  deleted_at          TIMESTAMP,
  created_by          UUID,
  updated_by          UUID,
  deleted_by          UUID,
  row_version         BIGINT      NOT NULL DEFAULT 1
);
```

Data dictionary:

Table 64

4. Table: cohort

Column	Type	Description	Example
cohort_id	BIGSERIAL	Auto-incremental cohort identifier	1
program_id	INT	Program (FK → program)	1
academic_period_id	INT	Academic period (FK → academic_period)	1
code	VARCHAR	Globally unique cohort code	ING-SIS-2026-S1-01
status	BOOLEAN	Active/inactive flag	true
created_at	TIMESTAMP	When the record was created	2024-01-15T10:30:00Z
updated_at	TIMESTAMP	When the record was last updated	2024-01-15T10:30:00Z
deleted_at	TIMESTAMP	NULL = active; with value = soft-deleted	NULL
created_by	UUID	User who created this record	550e8400-...
updated_by	UUID	User who last updated this record	550e8400-...
deleted_by	UUID	User who soft-deleted this record	NULL
row_version	BIGINT	Optimistic lock version counter	1

5. Table: course

Purpose: Courses within a program, with credit hours.

```
CREATE TABLE course (
  -- Business fields
```

```
  course_id     SERIAL      PRIMARY KEY,
  program_id    INT         NOT NULL,
  code          VARCHAR     NOT NULL,
  name          VARCHAR     NOT NULL,
  credit_hours  SMALLINT    NOT NULL,

  -- Status fields
  status        BOOLEAN     NOT NULL DEFAULT true,

  -- Audit (in all tables)
  created_at    TIMESTAMP NOT NULL DEFAULT NOW(),
  updated_at    TIMESTAMP NOT NULL DEFAULT NOW(),
  deleted_at    TIMESTAMP,
  created_by    UUID,
  updated_by    UUID,
  deleted_by    UUID,
  row_version   BIGINT      NOT NULL DEFAULT 1
);

CREATE UNIQUE INDEX uq_course_code_program ON course (program_id, code);
```

Data dictionary:

Table 65

5. Table: course

Column	Type	Description	Example
course_id	SERIAL	Auto-incremental course identifier	`1`
program_id	INT	Owning program (FK → program)	`1`
code	VARCHAR	Course code within the program	`MAT-101`
name	VARCHAR	Course name	`Calculus I`
credit_hours	SMALLINT	Number of credit hours	`4`
status	BOOLEAN	Active/inactive flag	`true`
created_at	TIMESTAMP	When the record was created	`2024-01-15T10:30:00Z`
updated_at	TIMESTAMP	When the record was last updated	`2024-01-15T10:30:00Z`
deleted_at	TIMESTAMP	NULL = active; with value = soft-deleted	`NULL`
created_by	UUID	User who created this record	`550e8400-...`
updated_by	UUID	User who last updated this record	`550e8400-...`
deleted_by	UUID	User who soft-deleted this record	`NULL`
row_version	BIGINT	Optimistic lock version counter	`1`

6. Table: academic_actor_type

Purpose: Catalog of academic actor types (STUDENT, INSTRUCTOR).

```
CREATE TABLE academic_actor_type (
  -- Business fields
  actor_type_id SMALLINT    PRIMARY KEY,
  code          VARCHAR     NOT NULL UNIQUE,
  name          VARCHAR     NOT NULL,

  -- Audit (in all tables)
  created_at    TIMESTAMP NOT NULL DEFAULT NOW(),
  updated_at    TIMESTAMP NOT NULL DEFAULT NOW(),
  deleted_at    TIMESTAMP,
  created_by    UUID,
  updated_by    UUID,
  deleted_by    UUID,
  row_version   BIGINT      NOT NULL DEFAULT 1
);
```

Data dictionary:

Table 66

6. Table: academic_actor_type

Column	Type	Description	Example
actor_type_id	SMALLINT	Auto-incremental type identifier	1
code	VARCHAR	Type code (unique)	STUDENT
name	VARCHAR	Type name	Student
created_at	TIMESTAMP	When the record was created	2024-01-15T10:30:00Z
updated_at	TIMESTAMP	When the record was last updated	2024-01-15T10:30:00Z
deleted_at	TIMESTAMP	NULL = active; with value = soft-deleted	NULL
created_by	UUID	User who created this record	550e8400-...
updated_by	UUID	User who last updated this record	550e8400-...
deleted_by	UUID	User who soft-deleted this record	NULL
row_version	BIGINT	Optimistic lock version counter	1

7. Table: academic_actor

Purpose: Unified table for students and instructors. Replaces separate Student/Teacher tables.

```
CREATE TABLE academic_actor (
  -- Business fields
  academic_actor_id BIGSERIAL   PRIMARY KEY,
  person_id         UUID        NOT NULL,
  actor_type_id     SMALLINT    NOT NULL,
  school_id         INT         NOT NULL,
  actor_code        VARCHAR     NOT NULL,
  started_on        DATE        NOT NULL,
  ended_on          DATE,

  -- Status fields
  status            BOOLEAN     NOT NULL DEFAULT true,

  -- Audit (in all tables)
  created_at        TIMESTAMP NOT NULL DEFAULT NOW(),
  updated_at        TIMESTAMP NOT NULL DEFAULT NOW(),
  deleted_at        TIMESTAMP,
  created_by        UUID,
  updated_by        UUID,
  deleted_by        UUID,
  row_version       BIGINT      NOT NULL DEFAULT 1
);

CREATE UNIQUE INDEX uq_actor_code_school_type
  ON academic_actor (school_id, actor_type_id, actor_code);
CREATE UNIQUE INDEX uq_actor_person_type_school
  ON academic_actor (person_id, actor_type_id, school_id);
CREATE INDEX idx_actor_deleted ON academic_actor (deleted_at) WHERE deleted_at IS NULL;
```

Data dictionary:

Table 67

7. Table: academic_actor

Column	Type	Description	Example
academic_actor_id	BIGSERIAL	Auto-incremental actor identifier	1
person_id	UUID	External identity (→ Identity.person)	550e8400-...

Column	Type	Description	Example
actor_type_id	SMALLINT	Actor type (FK → academic_actor_type)	1
school_id	INT	School (FK → school)	1
actor_code	VARCHAR	Institutional code for this actor	EST-2024-001
started_on	DATE	When the actor started in this role	2024-01-15
ended_on	DATE	When the actor ended this role (NULL = active)	NULL
status	BOOLEAN	Active/inactive flag	true
created_at	TIMESTAMP	When the record was created	2024-01-15T10:30:00Z
updated_at	TIMESTAMP	When the record was last updated	2024-01-15T10:30:00Z
deleted_at	TIMESTAMP	NULL = active; with value = soft-deleted	NULL
created_by	UUID	User who created this record	550e8400-...
updated_by	UUID	User who last updated this record	550e8400-...
deleted_by	UUID	User who soft-deleted this record	NULL
row_version	BIGINT	Optimistic lock version counter	1

Modeling decisions:

1. Unified table: a person can be both student and instructor (different `actor_type_id`).
2. `person_id` is an external reference to Identity — no cross-context FK.
3. Two unique constraints: code per school+type, and person per type+school.

8. Table: enrollment

Purpose: Enrollment of an academic_actor in a cohort.

```
-- Native ENUM: CREATE TYPE academic.enrollment_status AS ENUM ('Active','Withdrawn','Completed');
CREATE TABLE enrollment (
  -- Business fields
  enrollment_id       BIGSERIAL   PRIMARY KEY,
  academic_actor_id   BIGINT      NOT NULL,
  cohort_id           BIGINT      NOT NULL,
  enrolled_on         DATE        NOT NULL,

  -- Status fields
  enrollment_status   academic.enrollment_status NOT NULL,

  -- Audit (in all tables)
  created_at          TIMESTAMP NOT NULL DEFAULT NOW(),
  updated_at          TIMESTAMP NOT NULL DEFAULT NOW(),
  deleted_at          TIMESTAMP,
  created_by          UUID,
  updated_by          UUID,
  deleted_by          UUID,
  row_version         BIGINT      NOT NULL DEFAULT 1
);

CREATE UNIQUE INDEX uq_enrollment_actor_cohort
  ON enrollment (academic_actor_id, cohort_id);
CREATE INDEX idx_enrollment_cohort ON enrollment (cohort_id);
CREATE INDEX idx_enrollment_deleted ON enrollment (deleted_at) WHERE deleted_at IS NULL;
```

Data dictionary:

Table 68

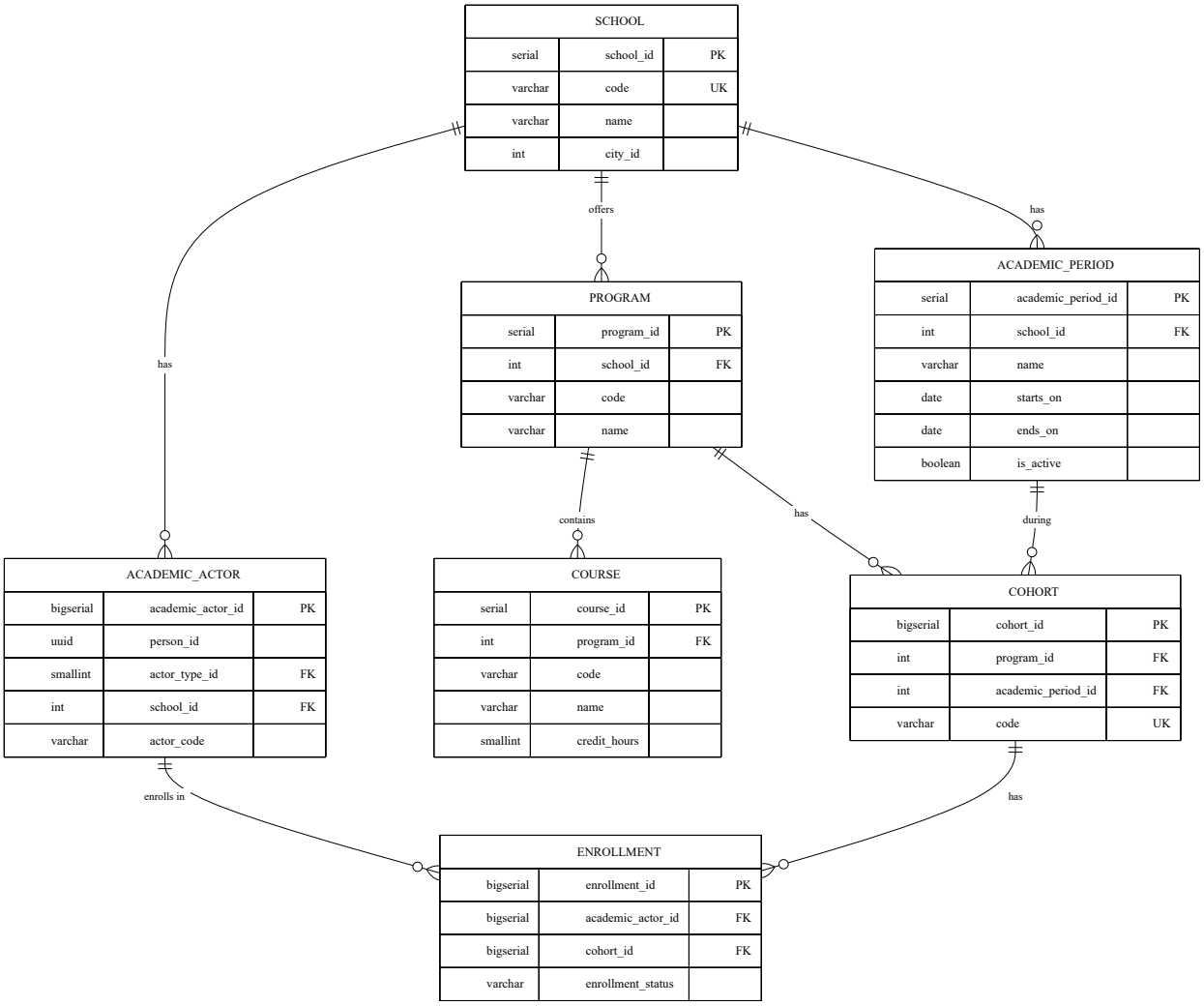
8. Table: enrollment

Column	Type	Description	Example
enrollment_id	BIGSERIAL	Auto-incremental enrollment identifier	`1`
academic_actor_id	BIGINT	Enrolled actor (FK → academic_actor)	`1`
cohort_id	BIGINT	Target cohort (FK → cohort)	`1`
enrolled_on	DATE	Enrollment date	`2024-01-15`
enrollment_status	VARCHAR	Status: Active, Withdrawn, Completed	`Active`
created_at	TIMESTAMP	When the record was created	`2024-01-15T10:30:00Z`
updated_at	TIMESTAMP	When the record was last updated	`2024-01-15T10:30:00Z`
deleted_at	TIMESTAMP	NULL = active; with value = soft-deleted	`NULL`
created_by	UUID	User who created this record	`550e8400-...`
updated_by	UUID	User who last updated this record	`550e8400-...`
deleted_by	UUID	User who soft-deleted this record	`NULL`
row_version	BIGINT	Optimistic lock version counter	`1`

9. Relationship Diagram

Figure 50

9. Relationship Diagram



10. Cross-Context Relationships

Table 69

10. Cross-Context Relationships

From	To	Reference Type	Mechanism
School.city_id	External countriesnow API	External catalog (no FK)	VARCHAR NULL display id + city_name/district/country
Academic_Actor.person_id	Identity.Person.person_id	External ID (no FK)	Cross-context reference by UUID
Scheduling.Schedule_Block.cohort_id	Academic.Cohort.cohort_id	External ID (no FK)	Cross-context reference by BIGINT

From	To	Reference Type	Mechanism
Scheduling.Schedule_Block.course_id	Academic.Course.course_id	External ID (no FK)	Cross-context reference by SERIAL
Scheduling.Class_Session.opened_by	Academic.Academic_Actor.academic_actor_id	External ID (no FK)	Cross-context reference by BIGINT
Attendance.Attendance_Record.academic_actor_id	Academic.Academic_Actor.academic_actor_id	External ID (no FK)	Cross-context reference by BIGINT

11. Correlations

- Domain entities that map to these tables → `02-domain/entities-and-rules.md`
- Saga and Outbox pattern for distributed consistency → `05-architecture/pattern-guide.md`
- Data for each service in detail → `09-microservices/services/academic-service/data-model.md`
- How data is accessed via API → `07-api/contracts/openapi/`

Data dictionary: scheduling

Owner: Scheduling bounded context **DB Engine:** PostgreSQL 17 — ACID transactions, conflict prevention for room/instructor booking

Engine justification:

- Unique constraints on (environment, day, time) and (instructor, day, time) prevent double-booking.
- Separated from Academic to allow independent evolution of timetable management.

Design Principles

- Scheduling owns: environment, schedule_block, class_session.
- `cohort_id`, `course_id`, `instructor_actor_id` are external references to Academic — no cross-context FKs.
- `environment.school_id` is an external reference to Academic.school.

1. Table: environment

Purpose: Physical spaces where classes are held (classrooms, labs, auditoriums).

```
CREATE TABLE environment (
  -- Business fields
  environment_id  SERIAL      PRIMARY KEY,
  school_id       INT         NOT NULL,
  code            VARCHAR     NOT NULL,
  name            VARCHAR     NOT NULL,
  capacity        SMALLINT    NOT NULL,

  -- Status fields
  status          BOOLEAN     NOT NULL DEFAULT true,

  -- Audit (in all tables)
  created_at      TIMESTAMP NOT NULL DEFAULT NOW(),
  updated_at      TIMESTAMP NOT NULL DEFAULT NOW(),
  deleted_at      TIMESTAMP,
  created_by      UUID,
  updated_by      UUID,
  deleted_by      UUID,
  row_version     BIGINT      NOT NULL DEFAULT 1
);

CREATE UNIQUE INDEX uq_environment_code_school ON environment (school_id, code);
```

Data dictionary:

Table 70

1. Table: environment

Column	Type	Description	Example
environment_id	SERIAL	Auto-incremental environment identifier	`1`
school_id	INT	Owning school (external ID → Academic.school)	`1`
code	VARCHAR	Environment code within the school	`SAL-101`
name	VARCHAR	Environment name	`Room 101`
capacity	SMALLINT	Maximum occupancy	`30`
status	BOOLEAN	Active/inactive flag	`true`
created_at	TIMESTAMP	When the record was created	`2024-01-15T10:30:00Z`
updated_at	TIMESTAMP	When the record was last updated	`2024-01-15T10:30:00Z`
deleted_at	TIMESTAMP	NULL = active; with value = soft-deleted	`NULL`

Column	Type	Description	Example
created_by	UUID	User who created this record	`550e8400-...`
updated_by	UUID	User who last updated this record	`550e8400-...`
deleted_by	UUID	User who soft-deleted this record	`NULL`
row_version	BIGINT	Optimistic lock version counter	`1`

2. Table: schedule_block

Purpose: A recurring time slot in the timetable (e.g., "Calculus I, Mondays 8:00-9:30, Room 101").

```
CREATE TABLE schedule_block (
  -- Business fields
  schedule_block_id   BIGSERIAL   PRIMARY KEY,
  cohort_id           BIGINT      NOT NULL,
  course_id           INT         NOT NULL,
  environment_id      INT         NOT NULL,
  instructor_actor_id BIGINT      NOT NULL,
  day_of_week         SMALLINT    NOT NULL CHECK (day_of_week BETWEEN 1 AND 7),
  starts_at           TIME        NOT NULL,
  ends_at             TIME        NOT NULL CHECK (ends_at > starts_at),

  -- Audit (in all tables)
  created_at          TIMESTAMP NOT NULL DEFAULT NOW(),
  updated_at          TIMESTAMP NOT NULL DEFAULT NOW(),
  deleted_at          TIMESTAMP,
  created_by          UUID,
  updated_by          UUID,
  deleted_by          UUID,
  row_version         BIGINT      NOT NULL DEFAULT 1,

  CONSTRAINT chk_block_day_of_week CHECK (day_of_week BETWEEN 1 AND 7),
  CONSTRAINT chk_block_time CHECK (starts_at < ends_at)
);

CREATE UNIQUE INDEX uq_block_environment_slot
  ON schedule_block (environment_id, day_of_week, starts_at);
CREATE UNIQUE INDEX uq_block_instructor_slot
  ON schedule_block (instructor_actor_id, day_of_week, starts_at);
CREATE INDEX idx_block_cohort ON schedule_block (cohort_id);
CREATE INDEX idx_block_course ON schedule_block (course_id);
```

Data dictionary:

Table 71

2. Table: schedule_block

Column	Type	Description	Example
schedule_block_id	BIGSERIAL	Auto-incremental block identifier	`1`
cohort_id	BIGINT	Target cohort (external ID → Academic.cohort)	`1`
course_id	INT	Course being taught (external ID → Academic.course)	`1`
environment_id	INT	Classroom (FK → environment)	`1`
instructor_actor_id	BIGINT	Instructor (external ID → Academic.academic_actor)	`1`
day_of_week	SMALLINT	Day of week (1..7)	`1`
starts_at	TIME	Block start time	`08:00`
ends_at	TIME	Block end time	`09:30`
created_at	TIMESTAMP	When the record was created	`2024-01-15T10:30:00Z`
updated_at	TIMESTAMP	When the record was last updated	`2024-01-15T10:30:00Z`
deleted_at	TIMESTAMP	NULL = active; with value = soft-deleted	`NULL`

Column	Type	Description	Example
created_by	UUID	User who created this record	`550e8400-...`
updated_by	UUID	User who last updated this record	`550e8400-...`
deleted_by	UUID	User who soft-deleted this record	`NULL`
row_version	BIGINT	Optimistic lock version counter	`1`

Modeling decisions:

1. Two unique constraints prevent double-booking: same room at same time, or same instructor at same time.
2. `day_of_week` is SMALLINT 1..7 ISO with `chk_block_day_of_week`; `starts_at < ends_at` with `chk_block_time`.
3. `cohort_id`, `course_id`, `instructor_actor_id` are external references — no cross-context FKs.

3. Table: class_session

Purpose: A real occurrence of a schedule_block on a specific date. Opened/closed by the instructor.

```
-- Native ENUM: CREATE TYPE scheduling.class_session_status AS ENUM ('Open','Closed','Cancelled');
CREATE TABLE class_session (
  -- Business fields
  class_session_id  BIGSERIAL   PRIMARY KEY,
  schedule_block_id BIGINT      NOT NULL,
  session_date      DATE        NOT NULL,
  opened_by         BIGINT,
  opened_at         TIMESTAMP,
  closed_by         BIGINT,
  closed_at         TIMESTAMP,

  -- Status fields
  session_status    scheduling.class_session_status NOT NULL,

  -- Audit (in all tables)
  created_at        TIMESTAMP NOT NULL DEFAULT NOW(),
  updated_at        TIMESTAMP NOT NULL DEFAULT NOW(),
  deleted_at        TIMESTAMP,
  created_by        UUID,
  updated_by        UUID,
  deleted_by        UUID,
  row_version       BIGINT      NOT NULL DEFAULT 1
);

CREATE UNIQUE INDEX uq_session_block_date ON class_session (schedule_block_id, session_date);
CREATE INDEX idx_session_date ON class_session (session_date);
```

Data dictionary:

Table 72

3. Table: class_session

Column	Type	Description	Example
class_session_id	BIGSERIAL	Auto-incremental session identifier	`1`
schedule_block_id	BIGINT	Parent schedule block (FK → schedule_block)	`1`
session_date	DATE	Date of this session	`2024-06-01`
opened_by	BIGINT	Instructor who opened the class (external ID → Academic.academic_actor)	`1`
opened_at	TIMESTAMP	When the instructor opened the class	`2024-06-01T08:00:00Z`
closed_by	BIGINT	Instructor who closed the class (external ID)	`1`

Column	Type	Description	Example
closed_at	TIMESTAMP	When the instructor closed the class	`2024-06-01T09:30:00Z`
session_status	VARCHAR	State: Open, Closed, Cancelled	`Open`
created_at	TIMESTAMP	When the record was created	`2024-06-01T08:00:00Z`
updated_at	TIMESTAMP	When the record was last updated	`2024-06-01T08:00:00Z`
deleted_at	TIMESTAMP	NULL = active; with value = soft-deleted	`NULL`
created_by	UUID	User who created this record	`550e8400-...`
updated_by	UUID	User who last updated this record	`550e8400-...`
deleted_by	UUID	User who soft-deleted this record	`NULL`
row_version	BIGINT	Optimistic lock version counter	`1`

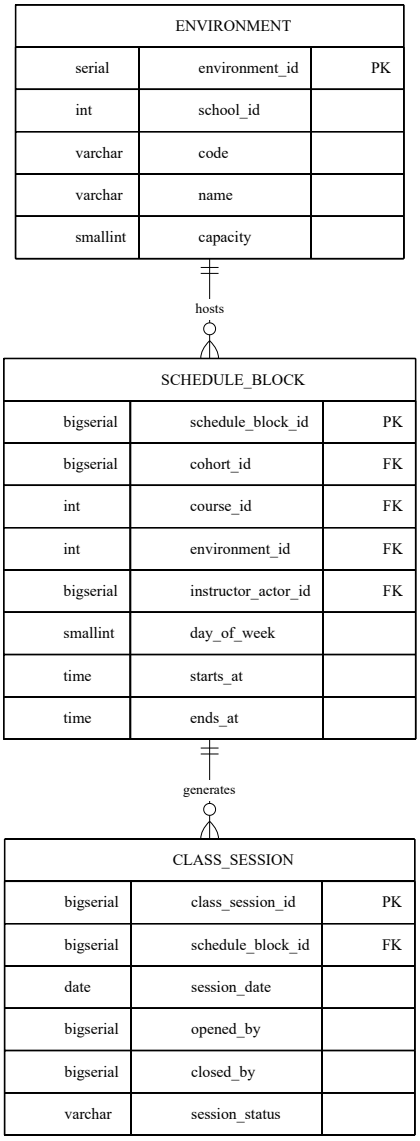
Modeling decisions:

1. One session per block per date — no duplicates.
2. `opened_by`/`closed_by` can differ from the scheduled instructor (substitution).
3. Sessions are created on-demand when the instructor starts class.

4. Relationship Diagram

Figure 51

4. Relationship Diagram



5. Cross-Context Relationships

Table 73

5. Cross-Context Relationships

From	To	Reference Type	Mechanism
Environment.school_id	Academic.School.school_id	External ID (no FK)	Cross-context reference by SERIAL
Schedule_Block.cohort_id	Academic.Cohort.cohort_id	External ID (no FK)	Cross-context reference by BIGINT

From	To	Reference Type	Mechanism
Schedule_Block.course_id	Academic.Course.course_id	External ID (no FK)	Cross-context reference by SERIAL
Schedule_Block.instructor_actor_id	Academic.Academic_Actor.academic_actor_id	External ID (no FK)	Cross-context reference by BIGINT
Class_Session.opened_by	Academic.Academic_Actor.academic_actor_id	External ID (no FK)	Cross-context reference by BIGINT
Class_Session.closed_by	Academic.Academic_Actor.academic_actor_id	External ID (no FK)	Cross-context reference by BIGINT
Attendance.Attendance_Record.class_session_id	Scheduling.Class_Session.class_session_id	External ID (no FK)	Cross-context reference by BIGINT

6. Correlations

- Domain entities that map to these tables → `02-domain/entities-and-rules.md`
- Saga and Outbox pattern for distributed consistency → `05-architecture/pattern-guide.md`
- Data for each service in detail → `09-microservices/services/scheduling-service/data-model.md`
- How data is accessed via API → `07-api/contracts/openapi/`

Data dictionary: attendance

Owner: Attendance bounded context **DB Engine:** PostgreSQL 17 — ACID transactions, session-based attendance tracking

Engine justification:

- Attendance records require transactional integrity (one record per actor per session).
- Justification workflow (Pending/Approved/Rejected) requires ACID guarantees.
- Supporting documents tracked separately for flexibility.

Design Principles

- Attendance has **NO** FK toward Scheduling or Academic.
- `class_session_id` and `academic_actor_id` are external IDs.
- Attendance can only be recorded while the associated session is `Open`.

1. Table: attendance_record

Purpose: Individual attendance record of an academic_actor in a class_session.

```
-- Native ENUMs: CREATE TYPE attendance.attendance_status AS ENUM ('Present','Absent','Late','Justified');
-- CREATE TYPE attendance.capture_method AS ENUM ('FACIAL','MANUAL','IOT','IMPORT');
CREATE TABLE attendance_record (
  -- Business fields
  attendance_record_id  BIGSERIAL   PRIMARY KEY,
  class_session_id      BIGINT      NOT NULL,
  academic_actor_id     BIGINT      NOT NULL,
  captured_at           TIMESTAMP,

  -- Status fields
  attendance_status     attendance.attendance_status NOT NULL,
  capture_method        attendance.capture_method NOT NULL,
  match_score           DECIMAL(5,4),

  -- Audit (in all tables)
  created_at            TIMESTAMP NOT NULL DEFAULT NOW(),
  updated_at            TIMESTAMP NOT NULL DEFAULT NOW(),
  deleted_at            TIMESTAMP,
  created_by            UUID,
  updated_by            UUID,
  deleted_by            UUID,
  row_version           BIGINT      NOT NULL DEFAULT 1
);

CREATE UNIQUE INDEX uq_attendance_session_actor
  ON attendance_record (class_session_id, academic_actor_id);
CREATE INDEX idx_attendance_actor ON attendance_record (academic_actor_id);
CREATE INDEX idx_attendance_deleted ON attendance_record (deleted_at) WHERE deleted_at IS NULL;
```

Data dictionary:

Table 74

1. Table: attendance_record

Column	Type	Description	Example
attendance_record_id	BIGSERIAL	Auto-incremental record identifier	`1`
class_session_id	BIGINT	Class session (external ID → Scheduling.class_session)	`1`
academic_actor_id	BIGINT	Actor being tracked (external ID → Academic.academic_actor)	`1`
captured_at	TIMESTAMP	When attendance was captured	`2024-06-01T08:05:00Z`

Column	Type	Description	Example
attendance_status	ENUM	Status: Present, Absent, Late, Justified	`Present`
capture_method	ENUM	Method: FACIAL, MANUAL, IOT, IMPORT	`FACIAL`
match_score	DECIMAL(5,4)	Biometric match score (0.0000..1.0000)	`0.9876`
created_at	TIMESTAMP	When the record was created	`2024-06-01T08:05:00Z`
updated_at	TIMESTAMP	When the record was last updated	`2024-06-01T08:05:00Z`
deleted_at	TIMESTAMP	NULL = active; with value = soft-deleted	`NULL`
created_by	UUID	User who created this record	`550e8400-...`
updated_by	UUID	User who last updated this record	`550e8400-...`
deleted_by	UUID	User who soft-deleted this record	`NULL`
row_version	BIGINT	Optimistic lock version counter	`1`

Modeling decisions:

1. One record per actor per session (uniqueness on class_session_id, academic_actor_id).
2. `match_score` stores biometric confidence for FACIAL captures.
3. `capture_method` replaces the old `source` field with a controlled vocabulary.

2. Table: justification_type

Purpose: Catalog of justification types for absences.

```
CREATE TABLE justification_type (
  -- Business fields
  justification_type_id  SERIAL      PRIMARY KEY,
  -- NULL = global type, NOT NULL = per-school custom type
  school_id              INT         NULL,
  name                   VARCHAR     NOT NULL,
  description            VARCHAR,
  requires_attachment    BOOLEAN     NOT NULL DEFAULT false,

  -- Status fields
  status                 BOOLEAN     NOT NULL DEFAULT true,

  -- Audit (in all tables)
  created_at             TIMESTAMP NOT NULL DEFAULT NOW(),
  updated_at             TIMESTAMP NOT NULL DEFAULT NOW(),
  deleted_at             TIMESTAMP,
  created_by             UUID,
  updated_by             UUID,
  deleted_by             UUID,
  row_version            BIGINT      NOT NULL DEFAULT 1
);

CREATE UNIQUE INDEX uq_justification_type_school_name ON justification_type (school_id, name);
-- In PostgreSQL NULL != NULL: the composite UNIQUE does not cover globals.
CREATE UNIQUE INDEX uq_justification_type_global_name ON justification_type (name) WHERE school_id IS NULL;
CREATE INDEX idx_justification_type_school ON justification_type (school_id);
```

Data dictionary:

Table 75

2. Table: justification_type

Column	Type	Description	Example
justification_type_id	SERIAL	Auto-incremental type identifier	`1`
school_id	INT	NULL = global, NOT NULL = per-school custom	`NULL`

Column	Type	Description	Example
name	VARCHAR	Type name (unique per school + partial unique for globals)	`Medical leave`
description	VARCHAR	Type description	`Validated medical incapacity`
requires_attachment	BOOLEAN	Whether supporting document is required	`true`
status	BOOLEAN	Active/inactive flag	`true`
created_at	TIMESTAMP	When the record was created	`2024-01-15T10:30:00Z`
updated_at	TIMESTAMP	When the record was last updated	`2024-01-15T10:30:00Z`
deleted_at	TIMESTAMP	NULL = active; with value = soft-deleted	`NULL`
created_by	UUID	User who created this record	`550e8400-...`
updated_by	UUID	User who last updated this record	`550e8400-...`
deleted_by	UUID	User who soft-deleted this record	`NULL`
row_version	BIGINT	Optimistic lock version counter	`1`

3. Table: justification

Purpose: Justification request for a specific absence. Linked 1:1 to attendance_record.

```
-- Native ENUM: CREATE TYPE attendance.review_status AS ENUM ('Pending','Approved','Rejected');
CREATE TABLE justification (
  -- Business fields
  justification_id        BIGSERIAL   PRIMARY KEY,
  attendance_record_id    BIGINT      NOT NULL UNIQUE,
  justification_type_id   INT         NOT NULL,
  reason                  TEXT        NOT NULL,
  submitted_at            TIMESTAMP NOT NULL,

  -- Status fields
  review_status           attendance.review_status NOT NULL,
  resolution_notes        TEXT        NULL,

  -- Audit (in all tables)
  created_at              TIMESTAMP NOT NULL DEFAULT NOW(),
  updated_at              TIMESTAMP NOT NULL DEFAULT NOW(),
  deleted_at              TIMESTAMP,
  created_by              UUID,
  updated_by              UUID,
  deleted_by              UUID,
  row_version             BIGINT      NOT NULL DEFAULT 1,

  -- Extra
  reviewed_by             UUID,
  reviewed_at             TIMESTAMP
);

CREATE INDEX idx_justification_type ON justification (justification_type_id);
CREATE INDEX idx_justification_reviewer ON justification (reviewed_by);
```

Data dictionary:

Table 76

3. Table: justification

Column	Type	Description	Example
justification_id	BIGSERIAL	Auto-incremental justification identifier	`1`
attendance_record_id	BIGINT	Attendance record (FK → attendance_record, UNIQUE)	`1`
justification_type_id	INT	Type of justification (FK → justification_type)	`1`

Column	Type	Description	Example
reason	TEXT	Detailed justification reason	`Medical appointment`
submitted_at	TIMESTAMP	When the justification was submitted	`2024-06-01T10:00:00Z`
review_status	ENUM	Status: Pending, Approved, Rejected	`Pending`
resolution_notes	TEXT	Review resolution notes	`Approved — certificate verified`
created_at	TIMESTAMP	When the record was created	`2024-06-01T10:00:00Z`
updated_at	TIMESTAMP	When the record was last updated	`2024-06-01T10:00:00Z`
deleted_at	TIMESTAMP	NULL = active; with value = soft-deleted	`NULL`
created_by	UUID	User who created this record	`550e8400-...`
updated_by	UUID	User who last updated this record	`550e8400-...`
deleted_by	UUID	User who soft-deleted this record	`NULL`
row_version	BIGINT	Optimistic lock version counter	`1`
reviewed_by	UUID	Reviewing user (external ID → Identity.app_user)	`550e8400-...`
reviewed_at	TIMESTAMP	When the justification was reviewed	`2024-06-02T14:00:00Z`

4. Table: supporting_document

Purpose: Files attached to a justification (medical certificates, etc.).

```
CREATE TABLE supporting_document (
  -- Business fields
  supporting_document_id  BIGSERIAL   PRIMARY KEY,
  justification_id        BIGINT      NOT NULL,
  file_name               VARCHAR     NOT NULL,
  storage_uri             VARCHAR     NOT NULL,
  mime_type               VARCHAR     NOT NULL,
  size_bytes              BIGINT      NOT NULL,

  -- Audit (in all tables)
  created_at              TIMESTAMP NOT NULL DEFAULT NOW(),
  updated_at              TIMESTAMP NOT NULL DEFAULT NOW(),
  deleted_at              TIMESTAMP,
  created_by              UUID,
  updated_by              UUID,
  deleted_by              UUID,
  row_version             BIGINT      NOT NULL DEFAULT 1
);

CREATE INDEX idx_document_justification ON supporting_document (justification_id);
```

5. Table: attendance_report (derived projection, outside 3NF core)

Derived projection (JSONB, no FKs): kept in the full model but outside the 3NF core. Cross-context ids are logical only.

```
CREATE TABLE attendance_report (
  -- Business fields
  report_id     UUID        PRIMARY KEY DEFAULT gen_random_uuid(),
  school_id     INT         NULL,
  cohort_id     BIGINT      NULL,
  generated_by  UUID        NULL,
  report_type   VARCHAR     NOT NULL,
  filters       JSONB       NULL,
  result        JSONB       NULL,
  generated_at  TIMESTAMP NOT NULL,

  -- Audit (in all tables)
  created_at    TIMESTAMP NOT NULL DEFAULT NOW(),
  updated_at    TIMESTAMP NOT NULL DEFAULT NOW(),
  deleted_at    TIMESTAMP,
```

```
  created_by    UUID,
  updated_by    UUID,
  deleted_by    UUID,
  row_version   BIGINT      NOT NULL DEFAULT 1
);

CREATE INDEX idx_report_school ON attendance_report (school_id);
CREATE INDEX idx_report_cohort ON attendance_report (cohort_id);
```

Data dictionary (attendance_report):

Table 77

5. Table: attendance_report (derived projection, outside 3NF core)

Column	Type	Description	Example
report_id	UUID	Report identifier	`550e8400-...`
school_id	INT	Cross-context to Academic.school (no FK)	`1`
cohort_id	BIGINT	Cross-context to Academic.cohort (no FK)	`1`
generated_by	UUID	Cross-context to Identity.app_user (no FK)	`550e8400-...`
report_type	VARCHAR	Report type	`cohort-summary`
filters	JSONB	Input filters	`{"cohort_id":1}`
result	JSONB	Computed result	`{"present":25}`
generated_at	TIMESTAMP	Generation time	`2024-06-02T14:00:00Z`

Data dictionary (supporting_document):

Table 78

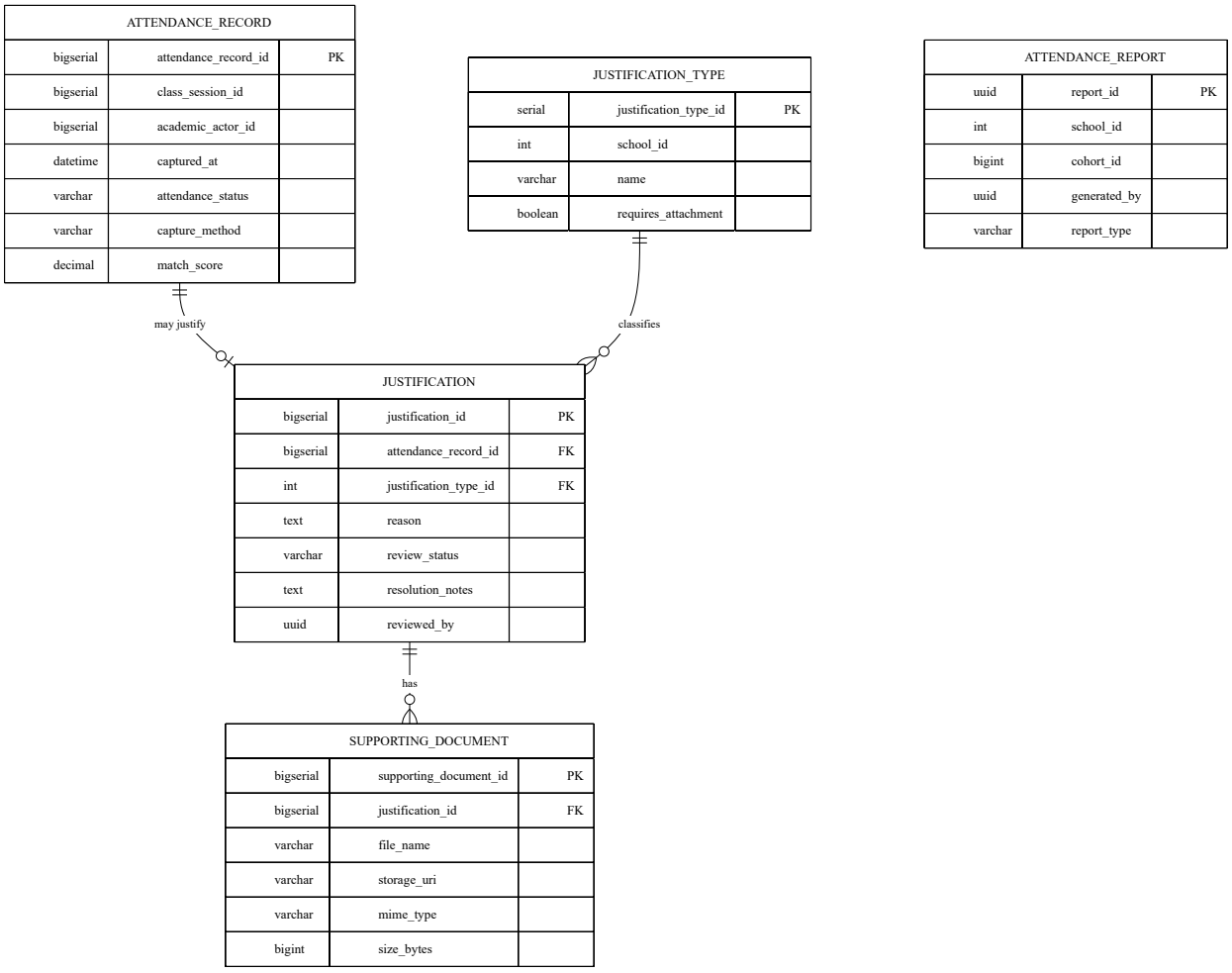
5. Table: attendance_report (derived projection, outside 3NF core)

Column	Type	Description	Example
supporting_document_id	BIGSERIAL	Auto-incremental document identifier	`1`
justification_id	BIGINT	Parent justification (FK → justification)	`1`
file_name	VARCHAR	Original file name	`medical_cert.pdf`
storage_uri	VARCHAR	Storage path or URL	`/docs/justifications/001/medical_cert.pdf`
mime_type	VARCHAR	MIME type of the file	`application/pdf`
size_bytes	BIGINT	File size in bytes	`245760`
created_at	TIMESTAMP	When the record was created	`2024-06-01T10:00:00Z`
updated_at	TIMESTAMP	When the record was last updated	`2024-06-01T10:00:00Z`
deleted_at	TIMESTAMP	NULL = active; with value = soft-deleted	`NULL`
created_by	UUID	User who created this record	`550e8400-...`
updated_by	UUID	User who last updated this record	`550e8400-...`
deleted_by	UUID	User who soft-deleted this record	`NULL`
row_version	BIGINT	Optimistic lock version counter	`1`

6. Relationship Diagram

Figure 52

6. Relationship Diagram



`attendance_report` has no FKs (derived projection). Cross-context ids are logical only.

7. Cross-Context Relationships

Table 79

7. Cross-Context Relationships

From	To	Reference Type	Mechanism
Attendance_Record.class_session_id	Scheduling.Class_Session.class_session_id	External ID (no FK)	Cross-context reference by BIGINT
Attendance_Record.academic_actor_id	Academic.Academic_Actor.academic_actor_id	External ID (no FK)	Cross-context reference by BIGINT
Justification.reviewed_by	Identity.App_User.user_id	External ID (no FK)	Cross-context reference by UUID
Attendance_Report.school_id / cohort_id / generated_by	Academic / Identity	External ID (no FK)	Derived projection, no FKs

8. Correlations

- Domain entities that map to these tables → `02-domain/entities-and-rules.md`
- Saga and Outbox pattern for distributed consistency → `05-architecture/pattern-guide.md`
- Data for each service in detail → `09-microservices/services/attendance-service/data-model.md`
- How data is accessed via API → `07-api/contracts/openapi/`

Data dictionary: biometric

Owner: Biometric bounded context **DB Engine:** MongoDB 7 for embeddings + one SQL workflow table (`biometric_update_case` hosted in `configuration`)

Engine justification:

- Facial and fingerprint embeddings are variable-length vectors — document storage is natural.
- MongoDB supports vector search indexes for similarity matching.
- Schema flexibility accommodates different embedding model versions.

Design Principles

- Biometric does **NOT** know Person — `person_id` represents an external identity.
- Embeddings persist in **NoSQL** (MongoDB); the re-enrollment workflow persists in SQL.
- ID references are resolved at the application level.
- `biometric_update_case` is logically owned by Biometric (DBML v7) and references these collections by logical ID (`current_embedding_ref`, cross-paradigm, no FK). It is still physically hosted in the `configuration` schema transiently for Liquibase/service compatibility — moving it requires a compatibility migration.

1. Collection: facial_embeddings

Purpose: Storage of facial embeddings (numeric vectors representing a face).

```
// MongoDB Collection: facial_embeddings
{
  _id: ObjectId,

  // Business fields
  person_id: UUID,             // External identity (→ Identity.person)
  encoding: [Float],           // Facial embedding vector
  model_version: String,       // AI model version used
  template_version: Int,       // Version counter for this person's facial templates
  enrolled_at: Date,           // When this embedding was captured

  // Status fields
  is_active: Boolean,

  // Audit (in all documents)
  created_at: Date,
  updated_at: Date,
  deleted_at: Date,
  created_by: UUID,
  updated_by: UUID,
  deleted_by: UUID
}
```

Data dictionary:

Table 80

1. Collection: facial_embeddings

Field	Type	Description	Example
_id	ObjectId	MongoDB document identifier	`507f1f77bcf86cd799439011`
person_id	UUID	External identity (→ Identity.person)	`550e8400-e29b-41d4-a716-446655440001`

Field	Type	Description	Example
encoding	Array[Float]	Facial embedding vector (512/1024 dimensions)	`[0.12, -0.45, ...]`
model_version	String	AI model version used	`face_net_v2.1`
template_version	Int	Version counter for this person	`1`
enrolled_at	Date	When this embedding was captured	`2024-01-15T10:30:00Z`
is_active	Boolean	Whether this is the current active embedding	`true`
created_at	Date	When the document was created	`2024-01-15T10:30:00Z`
updated_at	Date	When the document was last updated	`2024-01-15T10:30:00Z`
deleted_at	Date	NULL = active; with value = soft-deleted	`null`
created_by	UUID	User who created this document	`550e8400-...`
updated_by	UUID	User who last updated this document	`550e8400-...`
deleted_by	UUID	User who soft-deleted this document	`null`

Indexes:

Table 81

1. Collection: facial_embeddings

Name	Fields	Type	Justification
idx_embedding_person_active	(person_id, is_active)	Compound	Find active embeddings per person
idx_embedding_deleted	deleted_at	Sparse	Filter out soft-deleted documents

2. Collection: fingerprint_embeddings

Purpose: Storage of fingerprint biometric templates.

```
// MongoDB Collection: fingerprint_embeddings
{
  _id: ObjectId,

  // Business fields
  person_id: UUID,             // External identity (→ Identity.person)
  finger_number: Int,          // 1..10 (finger identifier)
  encoding: [Float],           // Fingerprint template vector
  model_version: String,       // AI model version used
  template_version: Int,       // Version counter for this person's finger templates
  enrolled_at: Date,           // When this template was captured

  // Status fields
  is_active: Boolean,

  // Audit (in all documents)
  created_at: Date,
  updated_at: Date,
  deleted_at: Date,
  created_by: UUID,
  updated_by: UUID,
  deleted_by: UUID
}
```

Data dictionary:

Table 82

2. Collection: fingerprint_embeddings

Field	Type	Description	Example
_id	ObjectId	MongoDB document identifier	`507f1f77bcf86cd799439012`
person_id	UUID	External identity (→ Identity.person)	`550e8400-...`
finger_number	Int	Finger identifier (1..10)	`1`
encoding	Array[Float]	Fingerprint template vector	`[0.23, -0.67, ...]`
model_version	String	AI model version used	`fingerprint_v1.3`
template_version	Int	Version counter for this person+finger	`1`
enrolled_at	Date	When this template was captured	`2024-01-15T10:30:00Z`
is_active	Boolean	Whether this is the current active template	`true`
created_at	Date	When the document was created	`2024-01-15T10:30:00Z`
updated_at	Date	When the document was last updated	`2024-01-15T10:30:00Z`
deleted_at	Date	NULL = active; with value = soft-deleted	`null`
created_by	UUID	User who created this document	`550e8400-...`
updated_by	UUID	User who last updated this document	`550e8400-...`
deleted_by	UUID	User who soft-deleted this document	`null`

Indexes:

Table 83

2. Collection: fingerprint_embeddings

Name	Fields	Type	Justification
idx_fingerprint_person_finger_active	(person_id, finger_number, is_active)	Compound	Find active template by person and finger
idx_fingerprint_deleted	deleted_at	Sparse	Filter out soft-deleted documents

3. Cross-Context Relationships

Table 84

3. Cross-Context Relationships

From	To	Reference Type	Mechanism
Facial_Embedding.person_id	Identity.Person.person_id	External ID (no FK)	Cross-context reference by UUID
Fingerprint_Embedding.person_id	Identity.Person.person_id	External ID (no FK)	Cross-context reference by UUID
Configuration.Biometric_Update_Case.current_embedding_ref	Facial/Fingerprint_Embedding._id	External ID (no FK)	Cross-paradigm reference (SQL → NoSQL)

Note: Embeddings persist in MongoDB with no physical FKs. The `biometric_update_case` SQL workflow table is logically owned by Biometric and physically hosted in `configuration`; all references are resolved at the application level.

4. Correlations

- Domain entities that map to these collections → `02-domain/entities-and-rules.md`
- Saga and Outbox pattern for distributed consistency → `05-architecture/pattern-guide.md`
- Data for each service in detail → `09-microservices/services/biometric-service/data-model.md`
- How data is accessed via API → `07-api/contracts/openapi/`

Data dictionary: notification

Owner: Notification bounded context **DB Engine:** PostgreSQL 17 — ACID transactions, alert lifecycle management

Engine justification:

- Alerts require reliable state tracking (raised → resolved).
- Alert types are catalog-driven for extensibility.

Design Principles

- Notification owns: alert_type, alert.
- `academic_actor_id` is an external reference to Academic — no cross-context FK.

1. Table: alert_type

Purpose: Catalog of alert types (e.g., absence threshold reached, schedule conflict).

```
CREATE TABLE alert_type (
  -- Business fields
  alert_type_id SMALLINT    PRIMARY KEY,
  code          VARCHAR     NOT NULL UNIQUE,
  name          VARCHAR     NOT NULL,
  severity      VARCHAR     NOT NULL CHECK (severity IN ('INFO','WARNING','CRITICAL')),
  channel       VARCHAR     NOT NULL DEFAULT 'DASHBOARD' CHECK (channel IN ('DASHBOARD','EMAIL','PUSH')),

  -- Audit (in all tables)
  created_at    TIMESTAMP NOT NULL DEFAULT NOW(),
  updated_at    TIMESTAMP NOT NULL DEFAULT NOW(),
  deleted_at    TIMESTAMP,
  created_by    UUID,
  updated_by    UUID,
  deleted_by    UUID,
  row_version   BIGINT      NOT NULL DEFAULT 1
);
```

Data dictionary:

Table 85

1. Table: alert_type

Column	Type	Description	Example
alert_type_id	SMALLINT	Auto-incremental type identifier	`1`
code	VARCHAR	Unique type code	`ABSENCE_THRESHOLD`
name	VARCHAR	Type name	`Absence threshold reached`
severity	VARCHAR	INFO / WARNING / CRITICAL	`WARNING`
channel	VARCHAR	DASHBOARD / EMAIL / PUSH	`DASHBOARD`
created_at	TIMESTAMP	When the record was created	`2024-01-15T10:30:00Z`
updated_at	TIMESTAMP	When the record was last updated	`2024-01-15T10:30:00Z`
deleted_at	TIMESTAMP	NULL = active; with value = soft-deleted	`NULL`
created_by	UUID	User who created this record	`550e8400-...`
updated_by	UUID	User who last updated this record	`550e8400-...`
deleted_by	UUID	User who soft-deleted this record	`NULL`
row_version	BIGINT	Optimistic lock version counter	`1`

2. Table: alert

Purpose: An alert raised for an academic_actor.

```
CREATE TABLE alert (
  -- Business fields
  alert_id          BIGSERIAL   PRIMARY KEY,
  academic_actor_id BIGINT      NOT NULL,
  alert_type_id     SMALLINT    NOT NULL,
  raised_at         TIMESTAMP NOT NULL,
  resolved_at       TIMESTAMP,

  -- Audit (in all tables)
  created_at        TIMESTAMP NOT NULL DEFAULT NOW(),
  updated_at        TIMESTAMP NOT NULL DEFAULT NOW(),
  deleted_at        TIMESTAMP,
  created_by        UUID,
  updated_by        UUID,
  deleted_by        UUID,
  row_version       BIGINT      NOT NULL DEFAULT 1
);

CREATE INDEX idx_alert_actor ON alert (academic_actor_id);
CREATE INDEX idx_alert_type ON alert (alert_type_id);
```

Data dictionary:

Table 86

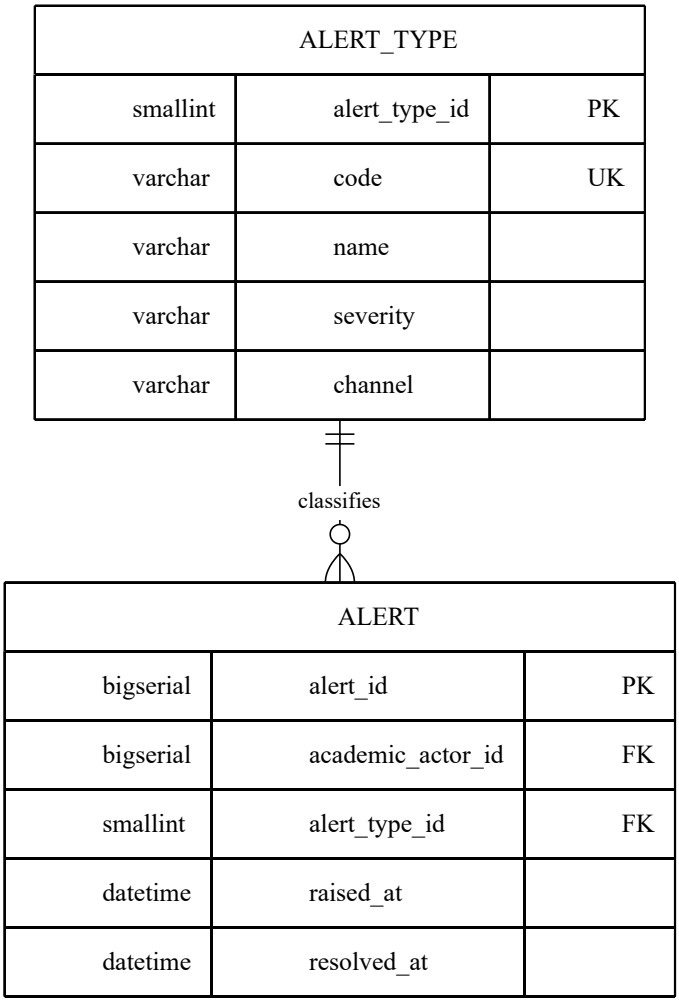
2. Table: alert

Column	Type	Description	Example
alert_id	BIGSERIAL	Auto-incremental alert identifier	`1`
academic_actor_id	BIGINT	Actor this alert is about (external ID → Academic.academic_actor)	`1`
alert_type_id	SMALLINT	Alert type (FK → alert_type)	`1`
raised_at	TIMESTAMP	When the alert was raised	`2024-06-01T10:00:00Z`
resolved_at	TIMESTAMP	When the alert was resolved (NULL = unresolved)	`NULL`
created_at	TIMESTAMP	When the record was created	`2024-06-01T10:00:00Z`
updated_at	TIMESTAMP	When the record was last updated	`2024-06-01T10:00:00Z`
deleted_at	TIMESTAMP	NULL = active; with value = soft-deleted	`NULL`
created_by	UUID	User who created this record	`550e8400-...`
updated_by	UUID	User who last updated this record	`550e8400-...`
deleted_by	UUID	User who soft-deleted this record	`NULL`
row_version	BIGINT	Optimistic lock version counter	`1`

3. Relationship Diagram

Figure 53

3. Relationship Diagram



4. Cross-Context Relationships

Table 87

4. Cross-Context Relationships

From	To	Reference Type	Mechanism
Alert.academic_actor_id	Academic.Academic_Actor.academic_actor_id	External ID (no FK)	Cross-context reference by BIGINT

5. Correlations

- Domain entities that map to these tables → `02-domain/entities-and-rules.md`
- Saga and Outbox pattern for distributed consistency → `05-architecture/pattern-guide.md`
- Data for each service in detail → `09-microservices/services/notification-service/data-model.md`

- How data is accessed via API → `07-api/contracts/openapi/`

Data dictionary: configuration

Owner: Configuration bounded context **DB Engine:** PostgreSQL 17 — key-value configuration storage with relational structure

Engine justification:

- Configuration parameters require ACID transactions for safe updates.
- `biometric_update_case` requires relational state tracking (Pending/Approved/Rejected).

Design Principles

- Configuration is cross-cutting — it does not directly relate to User/Person.
- `biometric_update_case` is the unified table for both facial and fingerprint update requests.
- **Logical owner is Biometric (DBML v7):** this schema hosts it only transiently for Liquibase/service compatibility. See `06-biometric.md`.

1. Table: academic_configuration

Purpose: Configurable parameters per school that affect academic behavior.

```
CREATE TABLE academic_configuration (
  -- Business fields
  configuration_id    SERIAL      PRIMARY KEY,
  school_id           INT         NOT NULL,
  configuration_name  VARCHAR     NOT NULL,
  configuration_value VARCHAR     NOT NULL,
  description         VARCHAR,

  -- Audit (in all tables)
  created_at          TIMESTAMP NOT NULL DEFAULT NOW(),
  updated_at          TIMESTAMP NOT NULL DEFAULT NOW(),
  deleted_at          TIMESTAMP,
  created_by          UUID,
  updated_by          UUID,
  deleted_by          UUID,
  row_version         BIGINT      NOT NULL DEFAULT 1
);

CREATE UNIQUE INDEX uq_academic_configuration
  ON academic_configuration (school_id, configuration_name);
```

Data dictionary:

Table 88

1. Table: academic_configuration

Column	Type	Description	Example
configuration_id	SERIAL	Auto-incremental configuration identifier	`1`
school_id	INT	Owning school (external ID → Academic.school)	`1`
configuration_name	VARCHAR	Parameter name	`tardy_tolerance_minutes`
configuration_value	VARCHAR	Parameter value (stored as string)	`15`
description	VARCHAR	Parameter description	`Minutes after start before marking late`
created_at	TIMESTAMP	When the record was created	`2024-01-15T10:30:00Z`
updated_at	TIMESTAMP	When the record was last updated	`2024-01-15T10:30:00Z`

Column	Type	Description	Example
deleted_at	TIMESTAMP	NULL = active; with value = soft-deleted	`NULL`
created_by	UUID	User who created this record	`550e8400-...`
updated_by	UUID	User who last updated this record	`550e8400-...`
deleted_by	UUID	User who soft-deleted this record	`NULL`
row_version	BIGINT	Optimistic lock version counter	`1`

2. Table: security_configuration

Purpose: System-wide security configuration parameters (not per-school).

```
CREATE TABLE security_configuration (
  -- Business fields
  configuration_id    SERIAL      PRIMARY KEY,
  configuration_name  VARCHAR     NOT NULL UNIQUE,
  configuration_value VARCHAR     NOT NULL,
  description         VARCHAR,

  -- Audit (in all tables)
  created_at          TIMESTAMP NOT NULL DEFAULT NOW(),
  updated_at          TIMESTAMP NOT NULL DEFAULT NOW(),
  deleted_at          TIMESTAMP,
  created_by          UUID,
  updated_by          UUID,
  deleted_by          UUID,
  row_version         BIGINT      NOT NULL DEFAULT 1
);
```

Data dictionary:

Table 89

2. Table: security_configuration

Column	Type	Description	Example
configuration_id	SERIAL	Auto-incremental configuration identifier	`1`
configuration_name	VARCHAR	Unique parameter name	`max_login_attempts`
configuration_value	VARCHAR	Parameter value	`5`
description	VARCHAR	Parameter description	`Max failed logins before lockout`
created_at	TIMESTAMP	When the record was created	`2024-01-15T10:30:00Z`
updated_at	TIMESTAMP	When the record was last updated	`2024-01-15T10:30:00Z`
deleted_at	TIMESTAMP	NULL = active; with value = soft-deleted	`NULL`
created_by	UUID	User who created this record	`550e8400-...`
updated_by	UUID	User who last updated this record	`550e8400-...`
deleted_by	UUID	User who soft-deleted this record	`NULL`
row_version	BIGINT	Optimistic lock version counter	`1`

3. Table: biometric_update_case — logical owner Biometric, physical host Configuration (transient)

Purpose: Unified cases for requesting facial or fingerprint embedding updates.

```
-- Native ENUMs (logical owner Biometric):
-- CREATE TYPE biometric.biometric_type AS ENUM ('FACIAL','FINGERPRINT');
-- CREATE TYPE biometric.update_status AS ENUM ('Pending','In_Review','Approved','Rejected');
CREATE TABLE biometric_update_case (
  -- Business fields
  case_id                UUID        PRIMARY KEY,
  person_id              UUID        NOT NULL,
```

```
  biometric_type          biometric.biometric_type NOT NULL,
  finger_number           SMALLINT    NULL CHECK (finger_number BETWEEN 1 AND 10),
  current_embedding_ref   VARCHAR,
  reason                  TEXT        NOT NULL,

  -- Status fields
  update_status           biometric.update_status NOT NULL DEFAULT 'Pending',

  -- Audit (in all tables)
  created_at              TIMESTAMP NOT NULL DEFAULT NOW(),
  updated_at              TIMESTAMP NOT NULL DEFAULT NOW(),
  deleted_at              TIMESTAMP,
  created_by              UUID,
  updated_by              UUID,
  deleted_by              UUID,
  row_version             BIGINT      NOT NULL DEFAULT 1,

  -- Workflow
  requested_by            UUID        NOT NULL,
  requested_at            TIMESTAMP NOT NULL,
  reviewed_by             UUID,
  reviewed_at             TIMESTAMP,
  resolution_notes        TEXT,

  CONSTRAINT chk_finger_number CHECK (
    (biometric_type = 'FINGERPRINT' AND finger_number BETWEEN 1 AND 10)
    OR (biometric_type = 'FACIAL' AND finger_number IS NULL)
  )
);

CREATE INDEX idx_biometric_case_person ON biometric_update_case (person_id);
CREATE INDEX idx_biometric_case_person_type ON biometric_update_case (person_id, biometric_type);
CREATE INDEX idx_biometric_case_status ON biometric_update_case (update_status);
```

Data dictionary:

Table 90

3. Table: biometric_update_case — logical owner Biometric, physical host Configuration (transient)

Column	Type	Description	Example
case_id	UUID	Auto-generated unique case identifier	`550e8400-...`
person_id	UUID	Person (external ID → Identity.person)	`550e8400-...`
biometric_type	ENUM	FACIAL or FINGERPRINT	`FACIAL`
finger_number	SMALLINT	Finger ID (1..10), only if FINGERPRINT, else NULL	`1`
current_embedding_ref	VARCHAR	Logical _id in MongoDB (cross-paradigm, no FK)	`507f1f77bcf86cd799439011`
reason	TEXT	Reason for the update request	`Significant weight change`
update_status	ENUM	Status: Pending, In_Review, Approved, Rejected	`Pending`
created_at	TIMESTAMP	When the record was created	`2024-06-01T10:00:00Z`
updated_at	TIMESTAMP	When the record was last updated	`2024-06-01T10:00:00Z`
deleted_at	TIMESTAMP	NULL = active; with value = soft-deleted	`NULL`
created_by	UUID	User who created this record	`550e8400-...`
updated_by	UUID	User who last updated this record	`550e8400-...`
deleted_by	UUID	User who soft-deleted this record	`NULL`
row_version	BIGINT	Optimistic lock version counter	`1`
requested_by	UUID	Requesting user, mandatory (external ID → Identity.app_user)	`550e8400-...`
requested_at	TIMESTAMP	When the request was submitted, mandatory	`2024-06-01T10:00:00Z`
reviewed_by	UUID	Reviewing user (external ID → Identity.app_user)	`550e8400-...`
reviewed_at	TIMESTAMP	When the request was reviewed	`2024-06-02T14:00:00Z`

Column	Type	Description	Example
resolution_notes	TEXT	Notes from the review	`Approved — new photo captured`

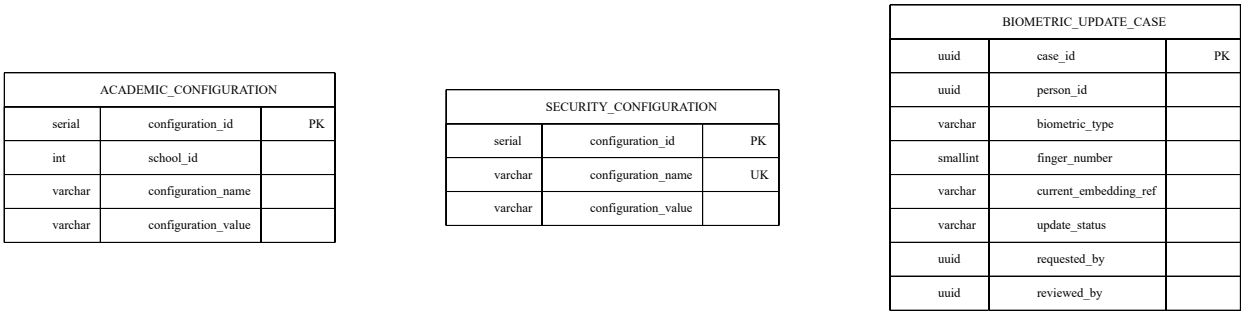
Modeling decisions:

1. Unified table: `biometric_type` distinguishes FACIAL vs FINGERPRINT cases.
2. `finger_number` is only populated when `biometric_type = FINGERPRINT` (`chk_finger_number`).
3. `current_embedding_ref` is a logical reference to the MongoDB document (cross-paradigm).
4. State flow: Pending → In_Review → Approved/Rejected. `requested_by/requested_at` are mandatory.

4. Relationship Diagram

Figure 54

4. Relationship Diagram



5. Cross-Context Relationships

Table 91

5. Cross-Context Relationships

From	To	Reference Type	Mechanism
Academic_Configuration.school_id	Academic.School.school_id	External ID (no FK)	Cross-context reference by SERIAL
Biometric_Update_Case.person_id	Identity.Person.person_id	External ID (no FK)	Cross-context reference by UUID
Biometric_Update_Case.current_embedding_ref	Biometric.Facial/Fingerprint_Embedding._id	External ID (no FK)	Cross-paradigm reference (SQL → NoSQL)
Biometric_Update_Case.requested_by	Identity.App_User.user_id	External ID (no FK)	Cross-context reference by UUID
Biometric_Update_Case.reviewed_by	Identity.App_User.user_id	External ID (no FK)	Cross-context reference by UUID

6. Correlations

- Domain entities that map to these tables → `02-domain/entities-and-rules.md`
- Saga and Outbox pattern for distributed consistency → `05-architecture/pattern-guide.md`
- Data for each service in detail → `09-microservices/services/configuration-service/data-model.md`

- How data is accessed via API → `07-api/contracts/openapi/`